Darshana Opticals (Pvt) Ltd.

Final Development Report

By

E.K.A.K. AMALKA

SE/2021/008

SANDARU MIHIRAN

SE/2021/010

T.H. IMASHA

SE/2021/053

D.S.U. WEERAKKODY

SE/2021/018

A report submitted in partial fulfillment of the requirements for the degree of Bachelor of Science Honors in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Contents

CHAPTER 1 — INTRODUCTION 15
1.1 Background of the Project 15
1.2 Problem Statement 16
1.3 Aim of the Project 16
1.4 Objectives of the Project 17
1.4.1 Digitize Retail and E-Commerce Operations 17
1.4.2 Improve Inventory Management 17
1.4.3 Facilitate Online Appointment Scheduling 17
1.4.4 Improve Customer Information Management 17
1.4.5 Support Secure Clinical Prescription Management 17
1.4.6 Enhance Customer Engagement 17
1.4.7 Support Business Administration and Reporting 17
1.4.8 Apply Software Quality and Security Practices 18
1.5 Scope of the Project 18
1.5.1 Retail and E-Commerce Integration 18
1.5.2 Clinical Appointment Scheduling 18
1.5.3 Multi-Branch Inventory Management 18
1.5.4 Customer Relationship and Retention 19
1.5.5 Administrative Reporting 19
1.6 Scope Exclusions 19
1.7 Project Stakeholders 20
1.8 Proposed Solution 21
1.9 Significance of the Project 21
1.10 Organization of the Report 22
CHAPTER 2 : SYSTEM ANALYSIS 23
2.1 Introduction 23
2.2 Analysis of the Existing System 23
2.2.1 Manual Inventory Tracking 23
2.2.2 Traditional Appointment Scheduling 24
2.2.3 Limited Digital Retail Accessibility 24
2.2.4 Customer Information Management 24
2.2.5 Business Reporting and Decision-Making 24

2.3 Analysis of the Proposed System 25
2.4 Stakeholder Analysis 25
2.4.1 Internal Stakeholders 25
2.4.2 External Stakeholders 26
2.4.3 Stakeholder Role Refinement 26
2.5 Functional Requirements Analysis 27
2.5.2 Profile Management — FR-002 28
2.5.3 Digital Product Catalogue — FR-003 28
2.5.4 Shopping Cart and Checkout — FR-004 28
2.5.5 Stock Synchronization — FR-005 28
2.5.6 Automated Reorder Alerts — FR-006 28
2.5.7 Online Appointment Booking — FR-007 28
2.5.8 Automated Appointment Reminders — FR-008 29
2.5.9 Payment Gateway Processing — FR-009 29
2.5.10 Loyalty Management — FR-010 29
2.5.11 AI Chatbot Assistance — FR-011 29
2.5.12 Sales and Inventory Reporting — FR-012 29
2.5.13 Clinical Prescription Recording — FR-013 29
2.5.14 Order Tracking — FR-014 30
2.6 Non-Functional Requirements Analysis 30
2.6.1 Performance — NFR-001 30
2.6.2 Reliability — NFR-002 31
2.6.3 Usability — NFR-003 31
2.6.4 Security — NFR-004 31
2.6.5 Scalability — NFR-005 31
2.6.6 Maintainability — NFR-006 31
2.6.7 Portability — NFR-007 31
2.7 Business Rules Analysis 32
2.7.1 Appointment Management Rules 32
2.7.2 Inventory and Sales Rules 32
2.7.3 Loyalty and Customer Rules 32
2.7.4 User Access Rules 33
2.8 System Constraints and Assumptions 33

2.8.1 Technical Constraints 33
2.8.2 Regulatory and Legal Constraints 33
2.8.3 Operational Constraints 33
2.8.4 Assumptions 34
2.9 Use Case Analysis 34
2.9.1 Overall Use Case Model 34
2.9.2 Use Case UC-03 — Book Eye Test 35
2.9.3 Use Case UC-05 — Manage Multi-Branch Inventory 36
2.10 Activity Analysis 37
2.10.1 Customer Activities 38
2.10.2 Staff Activities 38
2.10.3 Administrative Activities 38
2.11 Data Requirements Analysis 38
2.12 Alternative Solutions Considered 39
2.12.1 Continuing the Manual System 39
2.12.2 Native Desktop Application 39
2.12.3 Off-the-Shelf Retail Software 39
2.12.4 Alternative Technology Stacks 39
2.13 Feasibility Analysis 40
2.13.1 Technical Feasibility 40
2.13.2 Economic Feasibility 40
2.13.3 Operational Feasibility 40
2.13.4 Schedule Feasibility 40
2.14 Risk Analysis 41
2.14.2 Payment Integration Risk — RSK-002 41
2.14.3 Development Personnel Availability — RSK-003 41
2.14.4 Requirement Changes — RSK-004 42
2.14.5 Unauthorized Clinical Data Access — RSK-005 42
2.14.6 Staff Resistance — RSK-006 42
2.15 Approved Clarifications During Development 42
2.15.1 Role-Based Access Control 42
2.15.2 Shopping Cart and Checkout 42
2.15.3 Appointment Scheduling 42

2.15.4 Other Domain Decisions 43
2.16 Chapter Summary 43
CHAPTER 3 : SYSTEM DESIGN 44
3.1 Introduction 44
3.2 System Architecture 45
3.2.1 Architectural Overview 45
3.2.2 Presentation Layer 47
3.2.3 Application Layer 47
3.2.4 Data Layer 48
3.3 Architectural Design Considerations 49
3.3.1 Separation of Concerns 49
3.3.2 Maintainability 49
3.3.3 Scalability 49
3.3.4 Reliability and Data Consistency 50
3.3.5 Security 50
3.4 Database Design 50
3.4.1 Database Architecture 50
3.4.2 Core Database Entities 51
3.4.3 Entity Relationships 53
3.4.4 Data Integrity and Consistency 54
3.5 Component Design 54
3.5.1 Authentication and User Management Component 57
3.5.2 Product Catalogue and Inventory Component 57
3.5.3 Shopping Cart and Checkout Component 57
3.5.4 Order and Payment Component 57
3.5.5 Appointment and Doctor Session Component 58
3.5.6 Prescription Component 58
3.5.7 Loyalty Component 58
3.5.8 AI Chatbot Component 58
3.5.9 Reporting and Audit Component 59
3.6 Authentication and Authorization Design 59
3.6.1 Authentication Architecture 59
3.6.2 Role-Based Access Control 59

3.7 Major System Workflow Design 60
3.7.1 Customer Registration and Login 60
3.7.2 Shopping Cart Management 63
3.7.3 Checkout and Payment Processing 63
3.7.4 Appointment Booking 65
3.7.5 Order Lifecycle Management 67
3.7.6 Clinical Prescription Management 67
3.7.7 AI Chatbot Interaction 68
3.8 User Interface and User Experience Design 70
3.8.1 Customer Interface Design 70
3.8.2 Staff Interface Design 70
3.8.3 Administrative Interface Design 70
3.8.4 Accessibility Considerations 70
3.9 Security Design 71
3.9.1 Authentication Security 71
3.9.2 Authorization Security 71
3.9.3 Data Protection 71
3.9.4 Input Validation and Error Handling 71
3.9.5 Payment Security 72
3.9.6 Audit Logging 72
3.9.7 Security Verification Boundaries 72
3.10 External Service Integration Design 73
3.10.1 PayHere Integration 73
3.10.2 Cloudinary Integration 73
3.10.3 Gemini AI Integration 73
3.10.4 Email Integration 73
3.11 Deployment Architecture 74
3.11.1 Cloud Deployment Overview 74
3.11.2 Frontend Hosting 75
3.11.3 Backend Hosting 76
3.11.4 Database Hosting 76
3.11.5 Environment Configuration 76
3.11.6 Continuous Integration and Deployment 76

3.12 Architectural Decisions and Design Evolution 77
3.13 Design Limitations 78
3.14 Chapter Summary 79
CHAPTER 4 : SYSTEM IMPLEMENTATION 80
4.1 Introduction 80
4.2 Technology Stack and Justification 80
4.2.1 Frontend Technologies 81
4.2.2 Backend Technologies 82
4.2.3 Database Technologies 83
4.2.4 External Services 83
4.2.5 Deployment and Development Tools 84
4.3 Development Environment and Repository Organization 86
4.3.1 Development Environment 86
4.3.2 Repository Organization 86
4.3.3 Backend Code Organization 87
4.4 Frontend Implementation 88
4.4.1 React-Based Application 88
4.4.2 Authentication and Protected Navigation 89
4.4.3 Product Discovery Implementation 89
4.4.4 Shopping Cart Implementation 89
4.4.5 Checkout and Payment Interfaces 90
4.4.6 Appointment Booking Interface 90
4.4.7 Role-Specific Operational Interfaces 90
4.4.8 Frontend Verification 91
4.5 Backend Implementation 91
4.5.1 REST API Architecture 91
4.5.2 Authentication Implementation 91
4.5.3 JWT Implementation 92
4.5.4 Role-Based Authorization 93
4.5.5 Product and Inventory Implementation 93
4.5.6 Checkout and Order Creation 94
4.5.7 Order Lifecycle Implementation 95
4.5.8 PayHere Payment Integration 96

4.5.9 Doctor Session and Appointment Implementation 97
4.5.10 Clinical Prescription Implementation 97
4.5.11 Loyalty Implementation 98
4.5.12 AI Chatbot Implementation 98
4.5.13 Reporting and Audit Implementation 98
4.6 Database Implementation 100
4.6.1 MongoDB and Mongoose Integration 100
4.6.2 Database Collections 100
4.6.3 Data Relationships 100
4.6.4 Database Indexing 101
4.6.5 Database Transactions 101
4.6.6 Data Validation and Security 101
4.6.7 Data Retention and Temporary Records 102
4.7 External Service Integration 102
4.7.1 Payment Gateway 102
4.7.2 Cloudinary Image Management 102
4.7.3 Gemini AI Chatbot 103
4.7.4 Email Notifications 103
4.8 Continuous Integration and Deployment 103
4.8.1 Version Control and Collaboration 103
4.8.2 Backend Continuous Integration 103
4.8.3 Frontend Continuous Integration 105
4.8.4 Frontend Deployment 106
4.8.5 Backend Deployment 107
4.8.6 Deployment Architecture Evolution 107
4.8.7 Deployment Verification and Quality Gates 107
4.9 Implementation Challenges and Engineering Decisions 107
4.9.1 Maintaining a Consistent Authorization Model 107
4.9.2 Maintaining Checkout and Inventory Consistency 108
4.9.3 Handling Payment Verification 108
4.9.4 Supporting Appointment Capacity 108
4.9.5 Separating Clinical and Operational Responsibilities 109
4.9.6 Managing Inventory Images 109

4.9.7 Controlling AI Chatbot Usage 109
4.9.8 Adapting the Deployment Strategy 109
4.10 Software Quality and Implementation Verification 110
4.11 Chapter Summary 111
CHAPTER 5 : TESTING 113
5.1 Introduction 113
5.2 Testing Objectives 113
5.3 Testing Strategy and Methodology 114
5.3.1 Testing Levels 114
5.3.2 Requirement-Based Testing 115
5.3.3 Risk-Based Testing 115
5.4 Testing Environment 115
5.4.1 Backend Environment 115
5.4.2 Frontend Environment 116
5.4.3 Test Data and Isolation 116
5.5 Backend Automated Testing 117
5.5.1 Backend Testing Scope 117
5.5.2 Backend Test Execution Results 117
5.6 Frontend Automated Testing 118
5.6.1 Frontend Testing Scope 118
5.6.2 Frontend Test Execution Results 118
5.7 Consolidated Automated Testing Results 119
5.8 Test Case Register and Requirement Traceability 120
5.8.1 Final Test Case Register 120
5.8.2 Selected Representative Test Cases 120
5.8.3 Requirement Traceability Limitations 121
5.9 Code Coverage Analysis 122
5.9.1 Coverage Metrics 122
5.9.2 Critical Business Logic Branch Coverage 122
5.9.3 Interpretation of Code Coverage 123
5.10 API Integration Coverage 124
5.10.1 API Route Inventory 124
5.10.2 Confirmed Integration Test Source References 124

5.11 System Integration and End-to-End Testing 124
5.11.1 Integration Testing Scope 124
5.11.2 Recorded E2E Evidence 125
5.11.3 Hosted PayHere Verification Limitation 125
5.12 Security Testing 125
5.12.1 Security Assessment 125
5.12.2 Supporting Adversarial Test Results 126
5.12.3 Residual Security Risks 127
5.13 Performance Testing 127
5.13.1 Performance Evaluation Method 127
5.13.2 Baseline API Performance 127
5.13.3 Controlled Load Testing 128
5.13.4 Memory Usage 128
5.14 Database Optimization and Concurrency Testing 129
5.14.1 Database Query Optimization 129
5.14.2 Concurrent Checkout Verification 130
5.14.3 Concurrent Appointment Booking Verification 130
5.15 Non-Functional Requirements Evaluation 131
5.16 Defect Management and Regression Testing 132
5.17 Testing Limitations and Outstanding Verification 132
5.18 Chapter Summary 133
CHAPTER 6 : EVALUATION 135
6.1 Introduction 135
6.2 Evaluation of Project Objectives 135
6.2.1 Digitalization of Retail and E-Commerce Operations 135
6.2.2 Improvement of Inventory Management 136
6.2.3 Online Appointment Scheduling 136
6.2.4 Customer Information and Clinical Prescription Management 137
6.2.5 Customer Assistance and Engagement 137
6.2.6 Business Administration and Reporting 138
6.2.7 Software Quality, Security, and Maintainability 138
6.3 Functional Requirements Achievement Assessment 139
6.4 Non-Functional Requirements Evaluation 141

6.4.1 Performance 142
6.4.2 Reliability 143
6.4.3 Usability 143
6.4.4 Security 143
6.4.5 Scalability 144
6.4.6 Maintainability 144
6.4.7 Portability 144
6.5 Evaluation of Architectural and Engineering Decisions 145
6.5.1 Role-Based Access Control 145
6.5.2 Shopping Cart and Checkout 145
6.5.3 Appointment Management 145
6.5.4 Payment Processing 145
6.5.5 Cloud Hosting 146
6.6 Client Feedback and User Acceptance 146
6.7 Limitations of the Implemented System 146
6.7.1 Product Catalogue Limitations 147
6.7.2 Inventory Management Limitations 147
6.7.3 Appointment Communication Limitations 147
6.7.4 Loyalty Functionality Limitations 147
6.7.5 Reporting Limitations 147
6.7.6 External Integration Limitations 148
6.7.7 Deployment and Operational Limitations 148
6.7.8 Security and Compliance Limitations 148
6.8 Outstanding Requirements and Known Defects 149
6.9 Future Work 151
6.9.1 Enhanced Product Discovery 151
6.9.2 Advanced Inventory Operations 151
6.9.3 Extended Loyalty Management 151
6.9.4 Automated Appointment Communication 151
6.9.5 Advanced Business Analytics 151
6.9.6 Improved Operational Reliability 151
6.9.7 Extended Security Verification 152
6.9.8 Enhanced Cross-Browser and Usability Evaluation 152

6.10 Overall Project Evaluation 152
6.11 Chapter Summary 153
CHAPTER 7 : CONCLUSION 154
7.1 Introduction 154
7.2 Summary of Project Achievements 154
7.3 Achievement of Project Objectives 155
7.4 Software Engineering Contributions 156
7.5 Implementation Challenges and Lessons Learned 156
7.6 Testing and Quality Evaluation 157
7.7 Project Limitations 158
7.8 Future Development Opportunities 159
7.9 Overall Conclusion 160
CHAPTER 8 — REFERENCES 161
8.1 Academic and Technical References 161
8.2 Project Evidence References 162
Original Design and Requirements Documentation 162
Architectural Decision Records 162
Testing and Quality Assurance Evidence 164
Deployment and Continuous Integration Evidence 164
Source Code Repositories 165
CHAPTER 9 : APPENDICES 166
9.1 Introduction 166
Appendix A — Requirements and Design Documentation 166
A.1 Original Software Requirements Specification 166
A.2 Original Software Design Specification 167
A.3 Final Software Requirements Specification 167
A.4 Final Software Design Specification 167
Appendix B — System Architecture and Design Diagrams 168
B.1 System Context Diagram 168
B.2 High-Level Three-Tier Architecture 168
B.3 Logical Entity Relationship Diagram 169
B.4 Core Domain Model Class Diagram 169
B.5 Application Service Operations Diagram 169

B.6 Major Sequence Diagrams 169
B.7 Deployment Architecture 170
Appendix C — User Interface Screenshots 171
Appendix D — Source Code and Repository References 179
D.1 GitHub Repository Organization 179
D.2 Selected Backend Implementation Files 179
Table D.2: Selected Backend Implementation Files 179
D.3 Selected Frontend Implementation References 180
D.4 Source Revision Information 180
Appendix E — Architectural Decision Records 181
Appendix F — Final Test Case Register and Quality Assurance Evidence 182
F.1 Final Test Case Register 182
F.2 Consolidated Automated Testing Results 183
F.3 Automated Test Coverage Report 183
F.4 Selected Critical-Module Coverage 184
F.5 API Integration Test Evidence 185
F.6 End-to-End Test Evidence 185
F.7 Performance Test Report 185
F.8 Transaction Concurrency Evidence 186
F.9 Security Evidence 186
F.10 Final QA Evidence Limitations 187
Appendix G — CI/CD and Deployment Evidence 187
G.1 Backend Continuous Integration 187
G.2 Frontend Continuous Integration 188
G.3 Frontend Deployment 188
G.4 Backend Deployment 188
G.5 Database Configuration 188
Database Configuration 188
G.6 Deployment Verification 189
Appendix H — Project Management and Development Evidence 190
H.1 GitHub Issue Management 190
H.2 Pull Request Reviews 191
Pull Request Evidence Summary 191

H.3 Sprint Management 194
H.4 Development Standards 194
Appendix I — User Acceptance and Client Feedback Evidence 194
I.1 User Acceptance Testing 194
I.2 Client Feedback 195
I.3 Final Acceptance or Sign-Off 195
Appendix J — Supplementary Limitations and Outstanding Work 195
J.1 Outstanding Quality Assurance Verification 196
9.2 Appendix Summary 197

CHAPTER 1 — INTRODUCTION

1.1 Background of the Project

The rapid development of information technology and web-based services has transformed how retail businesses manage their daily operations and interact with customers. Digital platforms provide opportunities to improve operational efficiency, maintain consistent business records, extend customer accessibility, and reduce the limitations associated with manual processes.

The optical retail industry combines conventional retail activities with specialized eye-care services. In addition to selling eyewear products such as frames, lenses, and sunglasses, optical businesses may need to manage customer appointments, eye examination records, clinical prescriptions, product inventories, and customer orders. These activities require coordination among customers, clinical personnel, operational staff, and business management.

Darshana Opticals (Pvt) Ltd. is an optical retail business operating in Sri Lanka. According to the original Software Requirements Specification (SRS v1.0), the organization sought to transition from a primarily manual physical-store operating model towards a centralized digital platform. The proposed transformation aimed to improve the management of retail and clinical operations while introducing online services for customers.

To address these requirements, the **Optical Shop Management System (OSMS)** was proposed as an integrated web-based application.

OSMS combines a digital product catalogue, online shopping facilities, appointment scheduling, inventory management, customer account management, clinical prescription records, loyalty functionality, an AI-driven chatbot, and administrative reporting within a common information system.

The project was initially analyzed and designed during SENG 31242 and subsequently continued through the SENG 34213 System Development Project. The original SRS and Software Design Specification (SDS) established the approved requirements and proposed design. During development, additional architectural decisions were documented to clarify business rules, system responsibilities, and technical implementation boundaries.

The project adopts a web-based architecture based on the MERN technology stack, consisting of MongoDB, Express.js, React.js, and Node.js. This approach supports the development of an interactive frontend, a centralized application layer, and persistent storage of business information.

The OSMS project therefore represents an effort to integrate optical retail management and customer-facing services through modern software engineering practices.

1.2 Problem Statement

The existing business environment described in the original requirements analysis relies on manual or conventional processes for several optical retail and service-related operations. These processes introduce limitations when managing information across different business activities and physical locations.

Inventory management is one of the significant challenges. Maintaining accurate stock information across multiple branches requires continuous coordination. Manual tracking can introduce errors, delay stock updates, and make it difficult to identify products that require replenishment.

Appointment scheduling presents another challenge. Customers may need to contact the business or visit a branch to arrange an eye examination. Manual booking procedures can make appointment availability difficult to communicate and increase the possibility of scheduling conflicts.

Customer information and clinical prescription records also require reliable storage and retrieval. Maintaining these records through separate processes can make it difficult to access historical information efficiently while ensuring that sensitive clinical details remain protected.

Traditional physical-store operations further limit customers' ability to browse available eyewear products, place orders, and monitor order progress remotely.

From a management perspective, the absence of an integrated reporting environment can reduce the availability of consistent sales, inventory, and operational information for decision-making.

These limitations establish the need for a centralized information system capable of integrating business processes while supporting secure access, consistent records, and convenient customer interactions.

The central problem addressed by the OSMS project is therefore:

How can Darshana Opticals integrate its retail, inventory, appointment, clinical, and customer-service operations into a secure, accessible, and maintainable web-based management system?

1.3 Aim of the Project

The main aim of this project is to design, develop, and evaluate an integrated, web-based Optical Shop Management System for Darshana Opticals (Pvt) Ltd. that supports the digitalization of core retail and clinical operations, improves accessibility to customer services, and enables the centralized management of business information.

1.4 Objectives of the Project

The objectives of OSMS are derived from the functional requirements and project scope established in the original SRS.

1.4.1 Digitize Retail and E-Commerce Operations

To provide a web-based digital storefront that allows customers to browse eyewear products, review product information, manage shopping-cart selections, and complete supported online purchasing workflows.

1.4.2 Improve Inventory Management

To support the centralized management of inventory records across branches, provide timely stock information, and generate reorder-related alerts according to defined business rules.

1.4.3 Facilitate Online Appointment Scheduling

To provide an online appointment-booking facility that enables customers to access available eye examination sessions and manage relevant appointment activities.

1.4.4 Improve Customer Information Management

To maintain customer accounts and provide authorized access to personal information, purchase history, appointment information, and clinical prescription history.

1.4.5 Support Secure Clinical Prescription Management

To provide an appropriate mechanism for recording, storing, and retrieving clinical prescription information while applying the necessary access restrictions.

1.4.6 Enhance Customer Engagement

To introduce customer-support capabilities, including an AI-driven chatbot, customer loyalty functionality, and approved notification services.

1.4.7 Support Business Administration and Reporting

To provide authorized staff and management users with tools for managing relevant business operations and reviewing sales and inventory information.

1.4.8 Apply Software Quality and Security Practices

To develop the system using appropriate security controls, modular software architecture, validation mechanisms, version control, testing procedures, and documented engineering decisions.

The achievement of these objectives will be evaluated against the approved requirements and available testing evidence in Chapter 6.

1.5 Scope of the Project

The original SRS defines the scope of OSMS around five major business domains.

1.5.1 Retail and E-Commerce Integration

The system provides an online platform for displaying eyewear products, including frames, lenses, and sunglasses.

Customers are intended to be able to browse the product catalogue, view relevant product information, filter products based on supported attributes, and complete online purchasing activities.

Secure payment integration forms part of the approved e-commerce scope.

1.5.2 Clinical Appointment Scheduling

OSMS aims to replace manual appointment scheduling with a web-based booking process.

Customers can access available eye examination sessions and submit appointment requests through the application.

The original requirements also include automated appointment reminders. The final scope of communication and reminder functionality is governed by the approved architectural decisions.

1.5.3 Multi-Branch Inventory Management

The system supports centralized inventory management across branches.

The original scope includes stock monitoring, inventory updates, reorder alerts, and appropriate visibility of inventory information.

Detailed inventory permissions, reorder thresholds, and alert behavior were subsequently clarified through approved architectural decisions.

1.5.4 Customer Relationship and Retention

OSMS maintains customer-related information, including relevant account details, purchase information, and prescription history.

Customer engagement functionality includes an AI-driven chatbot and a loyalty program.

The system aims to improve customer accessibility and support continued interaction with the business.

1.5.5 Administrative Reporting

The system includes reporting capabilities intended to support business management and operational decision-making.

The original scope identifies sales analysis, appointment-related information, and inventory reporting as relevant management needs.

The final reporting definitions and access permissions are further governed by approved architectural decisions.

1.6 Scope Exclusions

The original SRS explicitly excludes the following activities from the initial project scope.

Medical Diagnostic Hardware Integration: OSMS manages relevant eye examination and prescription information but does not directly interface with medical diagnostic hardware.

Physical Delivery and Fleet Management: The system includes order-status tracking, but the physical transportation of goods and management of delivery fleets are outside the project scope.

Corporate Financial Auditing and Tax Filing: OSMS supports relevant sales and payment-related workflows but does not replace comprehensive accounting, statutory financial auditing, or corporate tax filing systems.

The original requirements also specify English-language support for the initial application. Sinhala and Tamil localization are outside the initial development scope.

Any further approved exclusions, deferred functionality, or implementation limitations will be discussed in Chapter 6.

1.7 Project Stakeholders

The original requirements analysis identified several stakeholder groups involved in the optical retail environmen

Stakeholder	Primary Interest
System Administrator	System-level management, configuration, and security
Branch Manager	Branch operations, inventory information, and business reporting
Optometrist / Medical Staff	Clinical services and prescription information
Inventory Manager	Stock tracking, inventory coordination, and replenishment
Sales Assistant / Cashier	Sales-related activities and customer assistance
Customers	Online purchases, appointments, and personal records
Darshana Opticals Management	Business oversight and strategic decision-making
Third-Party Logistics Providers	External physical delivery operations

Figure 1.1: Original Project Stakeholder Analysis

During implementation, the initial operational role definitions were refined through ADR-002 to establish an authoritative five-role application model:

- CUSTOMER
- SYSTEM_ADMIN
- BRANCH_MANAGER
- BRANCH_ASSISTANT
- OPTOMETRIST

This change consolidated overlapping operational responsibilities without removing the corresponding underlying business activities from the project scope.

The distinction between original stakeholders and final application authorization roles is discussed further in Chapter 2.

1.8 Proposed Solution

The proposed solution is a centralized web-based management platform that connects customers, authorized staff, and administrative users to the relevant services of Darshana Opticals.

The system uses a layered architecture consisting of a presentation layer, an application layer, and a data layer.

The presentation layer provides the user interface through which customers and staff interact with the system.

The application layer processes requests, enforces business rules, manages authentication and authorization, and coordinates communication with external services.

The data layer maintains persistent records relating to customers, staff, products, inventory, orders, appointments, prescriptions, and other supported business entities.

This architectural separation is intended to improve maintainability, support controlled access to data, and provide a foundation for future system enhancements.

The detailed design and actual implemented architecture are presented in Chapter 3.

1.9 Significance of the Project

The OSMS project is significant because it addresses several operational and service-related needs within an optical retail environment.

The system aims to improve access to product information and customer services through a web-based platform.

It also provides a centralized approach to maintaining inventory, orders, appointment information, and clinical records.

The integration of these functions can reduce the need to coordinate information through separate manual procedures and provide more consistent access to operational information.

Role-based authorization is particularly important because the system involves customer, managerial, administrative, and clinical information with different access requirements.

From a software engineering perspective, the project demonstrates the application of requirements engineering, system architecture, database design, iterative implementation, testing, collaborative development, and deployment-related practices.

The project also provides an opportunity to evaluate the extent to which the proposed solution satisfies its original objectives and identifies practical improvements for future development.

1.10 Organization of the Report

This report is organized according to Section 10.4 of the SENG 34213 System Development Project guideline.

Chapter 1 : introduces the project background, problem statement, objectives, scope, stakeholders, and significance.

Chapter 2 : presents the original system analysis, including business requirements, stakeholder needs, functional and non-functional requirements, operational rules, feasibility considerations, and project risks.

Chapter 3 : explains the system design, updated to reflect the final implemented architecture and approved architectural decisions.

Chapter 4 : describes system implementation, including selected technologies, major components, database implementation, CI/CD processes, and engineering challenges.

Chapter 5 : presents the testing approach and available evidence concerning functionality, coverage, performance, security, and reliability.

Chapter 6 : evaluates the achievement of project objectives, available client feedback, limitations, known defects, and future improvements.

Chapter 7 : presents the conclusion.

Chapter 8 : contains the references.

Chapter 9 : provides supporting appendices, including the Test Case Register and other relevant evidence.

CHAPTER 2 : SYSTEM ANALYSIS

2.1 Introduction

System analysis focuses on identifying the business problems, stakeholder expectations, functional requirements, operational constraints, and quality characteristics that define a proposed software solution.

The system analysis of the Optical Shop Management System (OSMS) was originally conducted during the SENG 31242 system analysis and design phase.

The analysis established the scope of the proposed application, identified the needs of Darshana Opticals (Pvt) Ltd., and defined the requirements documented in the original Software Requirements Specification (SRS v1.0).

The original analysis serves as the foundation for the development project. In accordance with the SENG 34213 Final Report guidelines, the approved system analysis is retained where the business requirements remain valid.

During implementation, certain aspects of the original requirements and design were clarified through approved Architectural Decision Records (ADRs). These clarifications are identified separately to maintain traceability between the original analysis and the implementation.

2.2 Analysis of the Existing System

According to the original SRS, Darshana Opticals sought to transform its manual physical-store operating model into a centralized digital platform.

The existing operational environment involved managing physical retail activities, customer information, inventory records, eye examination appointments, and sales-related processes.

The original requirements analysis identified limitations associated with manual records and the absence of an integrated customer-facing digital platform.

2.2.1 Manual Inventory Tracking

Maintaining product stock information across multiple branches requires coordination and accurate recordkeeping.

Manual processes can introduce inconsistencies in stock quantities, delay inventory updates, and reduce visibility of available products.

These limitations create a need for centralized inventory information and automated low-stock alerts.

2.2.2 Traditional Appointment Scheduling

Manual appointment arrangements may require customers to contact a branch or interact directly with staff to obtain an available appointment.

This approach can make it difficult to communicate real-time availability and coordinate appointments efficiently.

The original analysis therefore identified online appointment booking and automated communication as important system requirements.

2.2.3 Limited Digital Retail Accessibility

A business operating primarily through physical stores cannot provide the same level of remote product browsing and purchasing convenience as an online storefront.

The original requirements identified a need for a digital product catalogue, filtering options, shopping-cart functionality, and secure checkout.

2.2.4 Customer Information Management

Customer information may include personal details, purchase history, appointment information, and clinical prescriptions.

These records need to be accurately associated with individual customers and made available only to authorized users.

The proposed system therefore requires centralized storage and controlled access to customer-related information.

2.2.5 Business Reporting and Decision-Making

Management requires relevant information about sales performance, inventory availability, and other operational activities.

Without integrated reporting, obtaining timely and consistent business information can be difficult.

The requirements analysis identified comprehensive sales and inventory reporting as part of the proposed solution.

2.3 Analysis of the Proposed System

The Optical Shop Management System was proposed as a centralized web-based platform for managing the primary retail and clinical operations of Darshana Opticals.

The proposed application integrates customer-facing services with operational and administrative functionality.

The system is organized around five major business domains identified in the original SRS.

Business Domain	Proposed Capability
Retail and E-Commerce	Online product catalogue, shopping cart, checkout, and payments
Clinical Appointments	Eye examination booking, availability management, and communication
Multi-Branch Inventory	Centralized stock information, inventory operations, and reorder alerts
Customer Relationship Management	Customer profiles, prescription history, loyalty, and chatbot
Administration and Reporting	Authorized operational management and business reports

Figure 2.1: Proposed OSMS Business Domains

The system is intended to improve operational coordination while supporting secure and convenient access to relevant services.

2.4 Stakeholder Analysis

The original SRS identified several internal and external stakeholders.

2.4.1 Internal Stakeholders

System Administrator: Responsible for system-level administration, security-related management, and authorized configuration activities.

Branch Manager: Responsible for overseeing branch-level operations, inventory information, and business reporting.

Optometrist / Medical Staff: Responsible for clinical activities involving eye examinations and prescription information.

Inventory Manager: Originally identified as responsible for inventory records, stock control, purchase orders, and stock transfers.

Sales Assistant / Cashier: Originally identified as responsible for retail transactions, customer inquiries, and relevant point-of-sale activities.

2.4.2 External Stakeholders

Customers: Use the application to browse eyewear products, place orders, book appointments, and access their permitted personal information.

Darshana Opticals Management: Requires business information and relevant analytical reports for decision-making.

Third-Party Logistics Providers: Responsible for physical delivery-related activities that remain outside the direct system implementation scope.

2.4.3 Stakeholder Role Refinement

During implementation, ADR-002 established an authoritative five-role RBAC model.

The original Inventory Manager and Sales Assistant/Cashier responsibilities were reassigned to the Branch Manager and Branch Assistant roles.

The final application roles are CUSTOMER, SYSTEM_ADMIN, BRANCH_MANAGER, BRANCH_ASSISTANT, and OPTOMETRIST.

This refinement preserves the underlying business needs while simplifying authorization responsibilities.

A Doctor maintained within the appointment directory is treated as a service-provider record rather than an additional authenticated application role.

2.5 Functional Requirements Analysis

The original SRS defines 14 functional requirements.

These requirements describe the primary services expected from OSMS.

Requirement ID	Requirement Description	Original Priority
FR-01	User Registration and Authentication	High
FR-02	Profile Management and Prescription History	Medium
FR-03	Digital Product Catalogue and Filtering	High
FR-04	Shopping Cart and Checkout	High
FR-05	Real-Time Stock Synchronization	High
FR-06	Automated Inventory Reorder Alerts	Medium
FR-07	Online Appointment Booking	High
FR-08	Automated Appointment Reminders	Medium
FR-09	Secure Payment Gateway Processing	High
FR-10	Loyalty Points Calculation and Updates	Low
FR-11	AI-Driven Chatbot Assistance	Low
FR-12	Sales and Inventory Reporting	High
FR-13	Clinical Prescription Recording	High
FR-14	Order Tracking	Medium

Figure 2.2: Original Functional Requirements Summary

2.5.1 User Registration and Authentication — FR-001

The system must provide secure account registration and authentication for applicable customers and staff.

Authentication establishes user identity, while authorization controls access to permitted operations.

The final authorization model is governed by ADR-002.

2.5.2 Profile Management — FR-002

Users must be able to access and maintain relevant personal profile information.

Customers must be able to view permitted prescription history associated with their accounts.

2.5.3 Digital Product Catalogue — FR-003

The system must maintain a digital catalogue of eyewear products.

Customers must be able to browse products and use applicable filtering options, including brand, style, and price.

2.5.4 Shopping Cart and Checkout — FR-004

The system must provide a shopping-cart and checkout process for online eyewear purchases.

Cart persistence, authentication requirements, and checkout boundaries were subsequently clarified by ADR-003.

2.5.5 Stock Synchronization — FR-005

The application must support accurate stock information across relevant branches.

Inventory information must remain consistent with authorized stock updates and applicable order-processing operations.

2.5.6 Automated Reorder Alerts — FR-006

The system must support reorder alerts when inventory quantities meet the defined threshold conditions.

The detailed alert rules and role responsibilities are defined by ADR-006.

2.5.7 Online Appointment Booking — FR-007

Customers must be able to book eye examination appointments through an interactive scheduling interface.

The original requirement identifies available appointments and booking confirmation as essential functionality.

The scheduling design was subsequently refined through ADR-007 and the approved Doctor/Doctor Session model.

2.5.8 Automated Appointment Reminders — FR-008

The original SRS requires automated appointment reminders through email or SMS.

The final delivery mechanism, supported notification events, and any deferred functionality must follow the subsequently approved communication policy.

2.5.9 Payment Gateway Processing — FR-009

The system must support secure online payment processing through an integrated payment provider.

The approved payment lifecycle is governed by ADR-011.

Payment completion must be evaluated according to the actual configured and tested environment.

2.5.10 Loyalty Management — FR-010

The system must support loyalty points calculated according to approved purchase eligibility and business rules.

The final calculation and persistence design is governed by ADR-005.

2.5.11 AI Chatbot Assistance — FR-011

The system must provide chatbot assistance for supported customer inquiries.

The chatbot's approved knowledge, integration, usage, and lifecycle requirements are clarified through ADR-010.

2.5.12 Sales and Inventory Reporting — FR-012

The application must provide reporting functionality to authorized business users.

Reporting calculations, scope restrictions, and permitted management access are defined by ADR-014.

2.5.13 Clinical Prescription Recording — FR-013

The system must provide secure recording and storage of clinical prescription information.

Prescription records must be associated with the correct customer and protected by appropriate authorization controls.

The clinical data model is governed by ADR-001.

2.5.14 Order Tracking — FR-014

Customers must be able to access relevant order-status information.

Authorized staff must be able to perform supported order-tracking operations.

Order lifecycle rules and permitted status changes are governed by ADR-004.

2.6 Non-Functional Requirements Analysis

The original SRS defines seven measurable non-functional requirements.

These requirements establish the quality expectations against which the system should be evaluated.

ID	Quality Attribute	Original Measurable Criterion
NFR-001	Performance	95% of database search queries must be processed and displayed within 2 seconds
NFR-002	Reliability	99.9% uptime, excluding scheduled maintenance
NFR-003	Usability	New staff members should complete a sales transaction within 10 minutes of initial training
NFR-004	Security	Sensitive data transmissions must use SSL/TLS and passwords must be hashed using bcrypt
NFR-005	Scalability	Support a 50% increase in concurrent users and transactions without response-time degradation
NFR-006	Maintainability	90% of minor bug fixes should be implemented and deployed within 24 hours
NFR-007	Portability	Support the latest versions of Google Chrome, Mozilla Firefox, and Safari

Figure 2.3: Original Non-Functional Requirements Summary

2.6.1 Performance — NFR-001

The system is expected to respond efficiently under standard operational workloads.

The original target requires at least 95% of database search queries to be processed and displayed within two seconds.

Actual compliance must be established through measured performance testing.

2.6.2 Reliability — NFR-002

The original requirements specify a system availability target of 99.9%, excluding scheduled maintenance.

Achievement of this target requires operational monitoring and reliable deployment evidence.

2.6.3 Usability — NFR-003

The application is intended to be usable by staff members with limited technical experience.

The original measurable criterion requires newly trained staff to complete a full sales transaction within ten minutes of initial training.

2.6.4 Security — NFR-004

The system must protect sensitive customer and prescription information.

The original requirements specify encrypted communication using SSL/TLS and password hashing using bcrypt.

Additional authorization and application-security requirements are defined in the design and relevant architectural decisions.

2.6.5 Scalability — NFR-005

The original SRS requires the architecture to support a 50% increase in concurrent users and transactions without degradation in response time.

This remains a target rather than a verified achievement unless supported by appropriate testing.

2.6.6 Maintainability — NFR-006

The system must use an appropriate modular architecture to support defect correction and updates.

The original measurable target is to implement and deploy 90% of minor bug fixes within 24 hours.

2.6.7 Portability — NFR-007

The original requirements specify compatibility with current versions of Google Chrome, Mozilla Firefox, and Safari.

Browser compatibility must be confirmed using actual testing evidence.

2.7 Business Rules Analysis

The original SRS defines specific operational rules in Section 10.

These rules establish the business constraints that must be considered during implementation.

2.7.1 Appointment Management Rules

The original requirements establish the following rules:

- Appointment bookings must be made at least 24 hours in advance.
- Cancellations without penalty must occur at least 12 hours before the scheduled appointment.
- Only one patient may be assigned to an optometrist within a single 20-minute time slot.

These rules represent the original approved baseline. Detailed scheduling decisions and any formally approved changes must be interpreted using ADR-007 and the final appointment implementation.

2.7.2 Inventory and Sales Rules

The original SRS specifies that:

- Reorder requests are triggered when the stock quantity of a particular SKU falls below its branch-specific safety stock threshold.
- Promotional discounts are applied only when the relevant items are not already classified as clearance stock.
- Prescription-lens orders require an associated eye test result dated within the preceding 24 months.

The final implementation status of these specific rules must be verified against the relevant modules and approved changes.

2.7.3 Loyalty and Customer Rules

The original SRS specifies:

- One loyalty point is earned for every LKR 100 spent on eligible frames and accessories.
- Clinical eye examination fees are excluded from loyalty point accrual.
- Loyalty point redemption is permitted only when the customer balance exceeds 500 points.

- Returns of eyewear frames are permitted within seven days of purchase, subject to the original-condition and receipt requirements.

The detailed authoritative loyalty architecture is documented in ADR-005.

The presence of a rule in the original SRS does not independently establish that the associated functionality was fully implemented.

2.7.4 User Access Rules

The original requirements state that inventory adjustments and manual price changes require the appropriate managerial or system-administrator clearance.

They also define a 30-minute inactivity timeout for user sessions.

The later RBAC design clarifies authorization roles and permissions. Actual session-timeout enforcement must be verified before the project claims compliance with this rule.

2.8 System Constraints and Assumptions

2.8.1 Technical Constraints

The original SRS identifies the MERN technology stack as the selected implementation approach.

The system requires modern web-browser support, cloud-based deployment, and network connectivity for centralized operations.

Offline sales processing is excluded from the original version.

2.8.2 Regulatory and Legal Constraints

The original requirements identify the Sri Lankan Personal Data Protection Act (PDPA) as a relevant privacy requirement.

They also identify payment-provider compliance considerations and record-retention expectations.

Actual legal and security compliance must be assessed against implemented controls and available evidence.

2.8.3 Operational Constraints

The original application scope is limited to English-language support.

The interface is expected to follow Darshana Opticals' corporate branding and provide a consistent user experience.

The project is also constrained by the approved academic development schedule and available resources.

2.8.4 Assumptions

The original analysis assumes that:

- The business has access to suitable computing equipment and network connectivity.
- Staff members possess basic computer skills and receive appropriate training.
- Existing business records are verified before migration.
- Customers have access to internet-enabled devices.
- External payment and communication services are sufficiently available.
- Relevant stakeholders provide timely feedback during development and user acceptance testing.

These assumptions were used during the original system planning and should be evaluated against the actual project environment where necessary.

2.9 Use Case Analysis

The original SRS includes use case diagrams and descriptions of important business interactions.

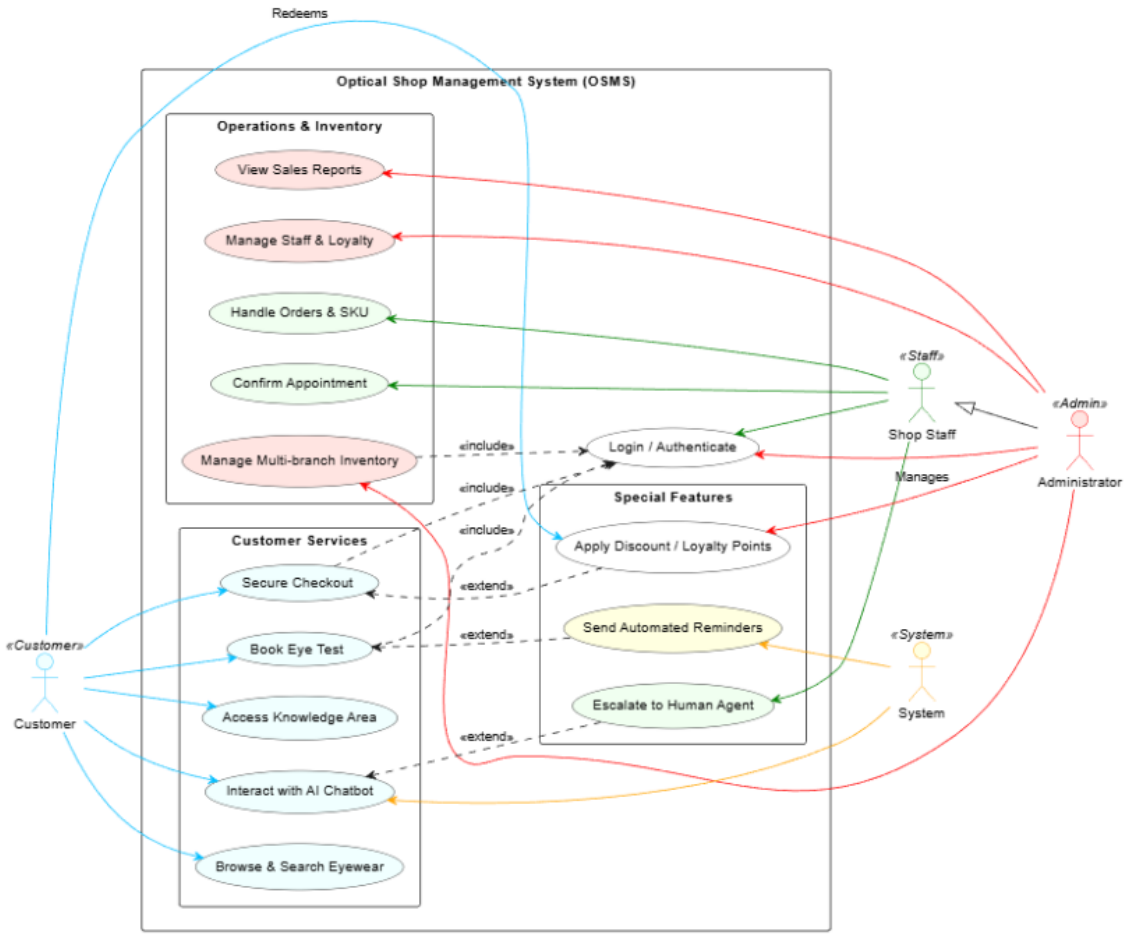
These models establish how users interact with OSMS and describe the required system responses.

2.9.1 Overall Use Case Model

The original use case diagram groups functionality into three main areas:

- Operations and Inventory.
- Customer Services.
- Special Features.

The diagram includes customer activities such as product browsing, checkout, and appointment booking, together with staff and administrative operations.



The diagram represents the initial requirements baseline. Role-related differences introduced through ADR-002 should be identified in the accompanying explanation.

2.9.2 Use Case UC-03 — Book Eye Test

The original SRS defines UC-03 as the process of booking an eye examination.

Primary actor: Customer.

Preconditions:

- The customer has a valid authenticated account.
- Appointment availability information is accessible.

Main Success Scenario:

1. The customer opens the appointment booking interface.
2. The system displays available branches.
3. The customer selects the preferred service provider.
4. Available appointment slots are displayed.
5. The customer selects a suitable date and time.
6. Required contact information is provided.
7. The booking request is submitted.
8. The system validates and records the appointment.
9. A booking confirmation is generated.

Alternative Scenarios:

- The selected appointment slot is unavailable.
- Mandatory information is incomplete or invalid.

The original use case references direct optometrist selection. The final appointment-provider architecture was subsequently refined to use a Doctor directory and Doctor Sessions. The current model must be described accurately in the final design chapter.

2.9.3 Use Case UC-05 — Manage Multi-Branch Inventory

The original SRS defines UC-05 as the process of monitoring and updating inventory across branch locations.

Original actors: Administrator and Inventory Manager.

Preconditions:

- An authorized user is authenticated.
- Appropriate inventory permissions are available.

Main Outcome:

Relevant inventory records are updated according to authorized operations. Applicable reorder alerts are generated when inventory threshold conditions are met.

The original Inventory Manager responsibilities were subsequently consolidated under the approved Branch Manager authorization model.

This change affects role ownership rather than eliminating the inventory-management requirement.

2.10 Activity Analysis

The original SRS includes an overall system activity diagram describing the interaction between Customer, Staff, and Administrator functions.

The major activity areas include customer authentication, product browsing, appointment booking, staff operations, inventory management, and administrative reporting.

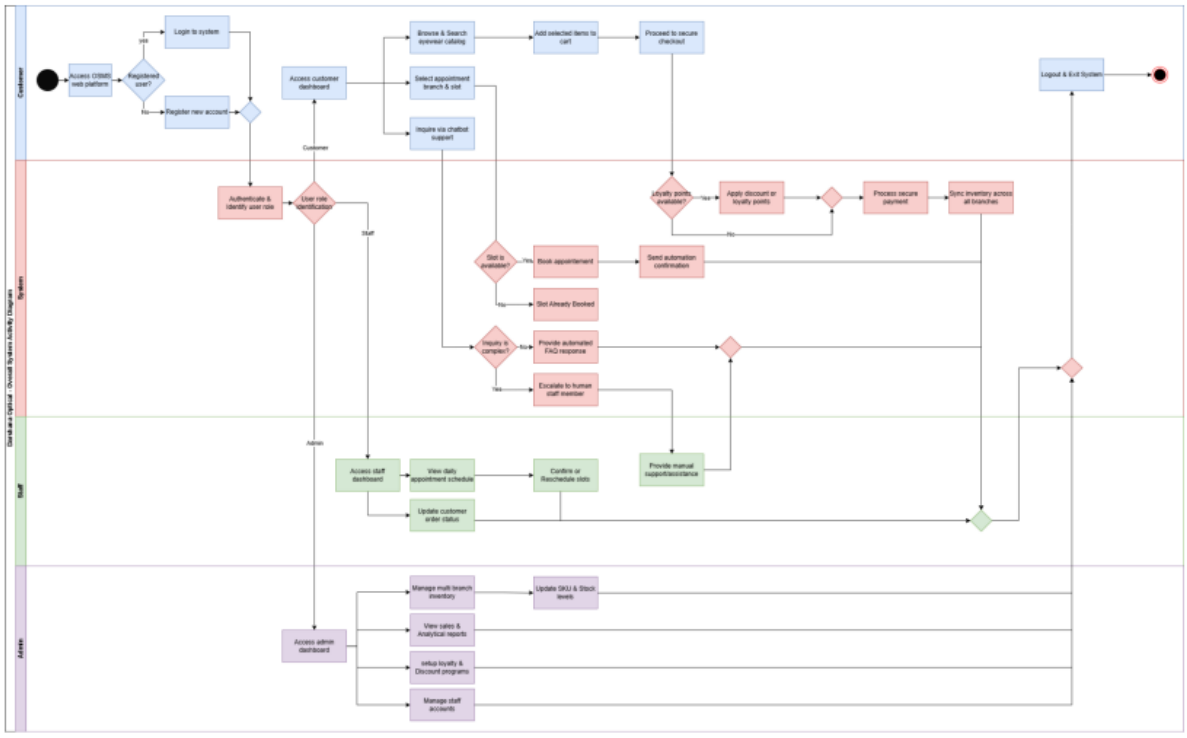


Figure 2.4: Overall System Activity Diagram

2.10.1 Customer Activities

Customer activities include accessing the web application, registering or logging in, viewing the product catalogue, managing shopping-cart selections, placing orders, and booking appointments.

2.10.2 Staff Activities

Staff activities include performing authorized branch operations, managing supported appointment functions, accessing appropriate order information, and handling relevant inventory operations.

2.10.3 Administrative Activities

Administrative activities include approved user-management operations, relevant configuration tasks, and permitted business reporting functions.

The final assignment of operations to specific application roles follows ADR-002 and the implemented authorization model.

2.11 Data Requirements Analysis

The original SRS identifies centralized data management as a core requirement of OSMS.

The system must maintain records relating to customer accounts, staff, inventory, orders, appointments, prescriptions, and relevant operational transactions.

MongoDB was selected as the primary data-storage technology.

The original data analysis identifies several important entity relationships:

- Customers and their associated orders.
- Orders and purchased items.
- Branches and related inventory.
- Customers and appointment records.
- Customers and clinical prescription records.
- Orders and payment information.

The final database design is addressed in Chapter 3 because additional approved collections and relationships were introduced during development.

Data integrity and access control remain central considerations because some business operations involve multiple related records.

2.12 Alternative Solutions Considered

The original SRS evaluates alternative solutions before selecting a web-based MERN architecture.

2.12.1 Continuing the Manual System

Maintaining the existing manual process was considered.

This alternative was rejected because it would not adequately address centralized inventory visibility, online service accessibility, and the operational limitations identified during requirements analysis.

2.12.2 Native Desktop Application

Developing a desktop application for installation at individual branches was considered.

This approach was not selected because it would introduce additional deployment and maintenance responsibilities and would not independently satisfy the requirement for a customer-facing online storefront.

2.12.3 Off-the-Shelf Retail Software

Existing retail-management software was evaluated as an alternative.

The original analysis identified limitations in adapting generic solutions to the specialized appointment, prescription, chatbot, and loyalty requirements of the optical retail domain.

2.12.4 Alternative Technology Stacks

Traditional web development approaches, including PHP/MySQL-based stacks, were considered.

The MERN stack was selected because it supported a JavaScript-based development environment, a modern single-page frontend, and centralized API-based interaction with a document database.

2.13 Feasibility Analysis

The original requirements documentation evaluates the project through four feasibility dimensions.

2.13.1 Technical Feasibility

The original technical feasibility assessment considered the availability of established MERN technologies, REST API development methods, authentication mechanisms, database services, and third-party integration techniques.

The availability of technical documentation and the development team's experience supported the selection of the proposed technology stack.

2.13.2 Economic Feasibility

The original economic feasibility analysis considered the use of open-source development technologies and opportunities to reduce manual operational effort.

The proposed online storefront and customer engagement services were also expected to provide opportunities for improved customer reach and retention.

These benefits were project expectations rather than independently measured financial outcomes.

2.13.3 Operational Feasibility

The proposed system was assessed in terms of its ability to support existing optical retail and clinical workflows.

The original analysis emphasized suitable user interfaces, staff training, and alignment with operational activities.

2.13.4 Schedule Feasibility

The original project plan followed a six-month development schedule.

The work was divided into manageable development phases covering requirements, design, implementation, and testing.

The incremental approach was intended to support progress tracking and timely integration of system modules.

2.14 Risk Analysis

The original SRS identifies six principal project risks.

Risk ID	Risk Description	Original Risk Level
RSK-001	Data loss during migration from manual records	Medium
RSK-002	Delays in third-party payment integration	High
RSK-003	Unavailability of key development personnel	Medium
RSK-004	Late functional requirement changes	High
RSK-005	Unauthorized access to sensitive prescription data	Medium
RSK-006	Staff resistance due to application complexity	Medium

Figure 2.5: Original Project Risk Summary

2.14.1 Data Migration Risk — RSK-001

The original analysis identifies the possibility of losing or corrupting information during migration from manual records to MongoDB.

The proposed mitigation involves controlled migration procedures and maintaining appropriate backups.

2.14.2 Payment Integration Risk — RSK-002

External payment-gateway integration may introduce development delays due to provider requirements and configuration dependencies.

The original mitigation strategy recommends beginning integration activities early and identifying alternative providers where appropriate.

2.14.3 Development Personnel Availability — RSK-003

The unavailability of key team members may affect implementation progress and knowledge continuity.

The original mitigation emphasizes documentation and knowledge-sharing practices.

2.14.4 Requirement Changes — RSK-004

Changes to approved functionality after requirements sign-off may introduce additional development effort and schedule risks.

The original requirements recommend formal change management and communication with relevant stakeholders.

2.14.5 Unauthorized Clinical Data Access — RSK-005

Clinical prescription information presents security and privacy concerns.

The original mitigation includes stronger authentication and encryption-related controls.

The final implementation of access-control and security measures must be verified through the project's security testing evidence.

2.14.6 Staff Resistance — RSK-006

Staff members may experience difficulty adapting to new digital workflows.

The original mitigation involves user-oriented interface design and appropriate training.

2.15 Approved Clarifications During Development

Although the original business scope remained the foundation of OSMS, several design areas required clarification during implementation.

2.15.1 Role-Based Access Control

ADR-002 established the five-role authorization model and clarified overlapping operational responsibilities.

2.15.2 Shopping Cart and Checkout

ADR-003 addressed cart persistence and checkout boundaries that were not fully defined in the original SDS.

2.15.3 Appointment Scheduling

ADR-007 provided further scheduling and availability rules, while the approved Doctor/Doctor Session approach refined the appointment-provider design.

2.15.4 Other Domain Decisions

Additional ADRs clarified clinical prescriptions, order lifecycles, loyalty calculations, inventory alerts, notification policies, payment integration, chatbot architecture, audit logging, and reporting definitions.

These records provide traceability for approved implementation-related decisions.

The detailed technical consequences of these decisions are presented in Chapter 3.

2.16 Chapter Summary

The original system analysis established the business foundation for developing the Optical Shop Management System.

The analysis identified the limitations of manual and disconnected operating processes and defined the need for centralized information management, online customer services, appointment scheduling, inventory control, and relevant administrative functions.

The original SRS specified 14 functional requirements and seven measurable non-functional requirements, together with business rules, constraints, assumptions, use cases, feasibility considerations, and project risks.

During implementation, approved architectural decisions clarified selected responsibilities and business workflows while retaining the original requirements as the historical baseline.

This analysis forms the foundation for the final System Design, which is presented in Chapter 3.

CHAPTER 3 : SYSTEM DESIGN

3.1 Introduction

System design translates the identified functional and non-functional requirements into an architectural and technical solution that guides the development of the proposed software application. It defines the structure of the system, the responsibilities of its components, the relationships among data entities, and the mechanisms used to ensure secure and reliable operation.

The Optical Shop Management System (OSMS) was designed to integrate the core retail, customer-service, clinical, and administrative operations of Darshana Opticals (Pvt) Ltd. into a centralized web-based platform. The system design was initially established during the SENG 31242 design phase and documented in the Software Design Specification (SDS v1.0).

During subsequent development, certain architectural components and business workflows were refined to address implementation requirements, technical constraints, and approved changes. These refinements were documented through Architectural Decision Records (ADR-001 to ADR-015) and incorporated into the implementation-aligned SDS v1.1.

The final design follows a three-tier layered architecture based on the MERN technology stack, consisting of MongoDB, Express.js, React.js, and Node.js. It separates presentation, application processing, and data persistence responsibilities while integrating external services for payment processing, image storage, AI-powered customer assistance, and supported email communication.

This chapter presents the final system design, including the architectural structure, component organization, database design, authentication and authorization mechanisms, major functional workflows, security architecture, user interface design, and cloud deployment architecture.

Detailed diagrams, schema definitions, and technical design specifications are maintained in the final SDS.

3.2 System Architecture

3.2.1 Architectural Overview

The Optical Shop Management System adopts a three-tier layered architecture to organize the application into independent functional layers.

The three layers are:

1. Presentation Layer — React.js frontend.
2. Application Layer — Node.js and Express.js backend.
3. Data Layer — MongoDB Atlas database.

The architectural design separates user interaction from business logic and persistent data storage. This separation supports modularity, maintainability, controlled information access, and independent management of application components.

Users interact with the system through a web browser. The React frontend sends requests to the backend through RESTful APIs. The backend processes authentication, authorization, business validation, database operations, and communication with external service providers.

MongoDB Atlas stores the system's operational and clinical records, while external services are accessed through defined integration boundaries.

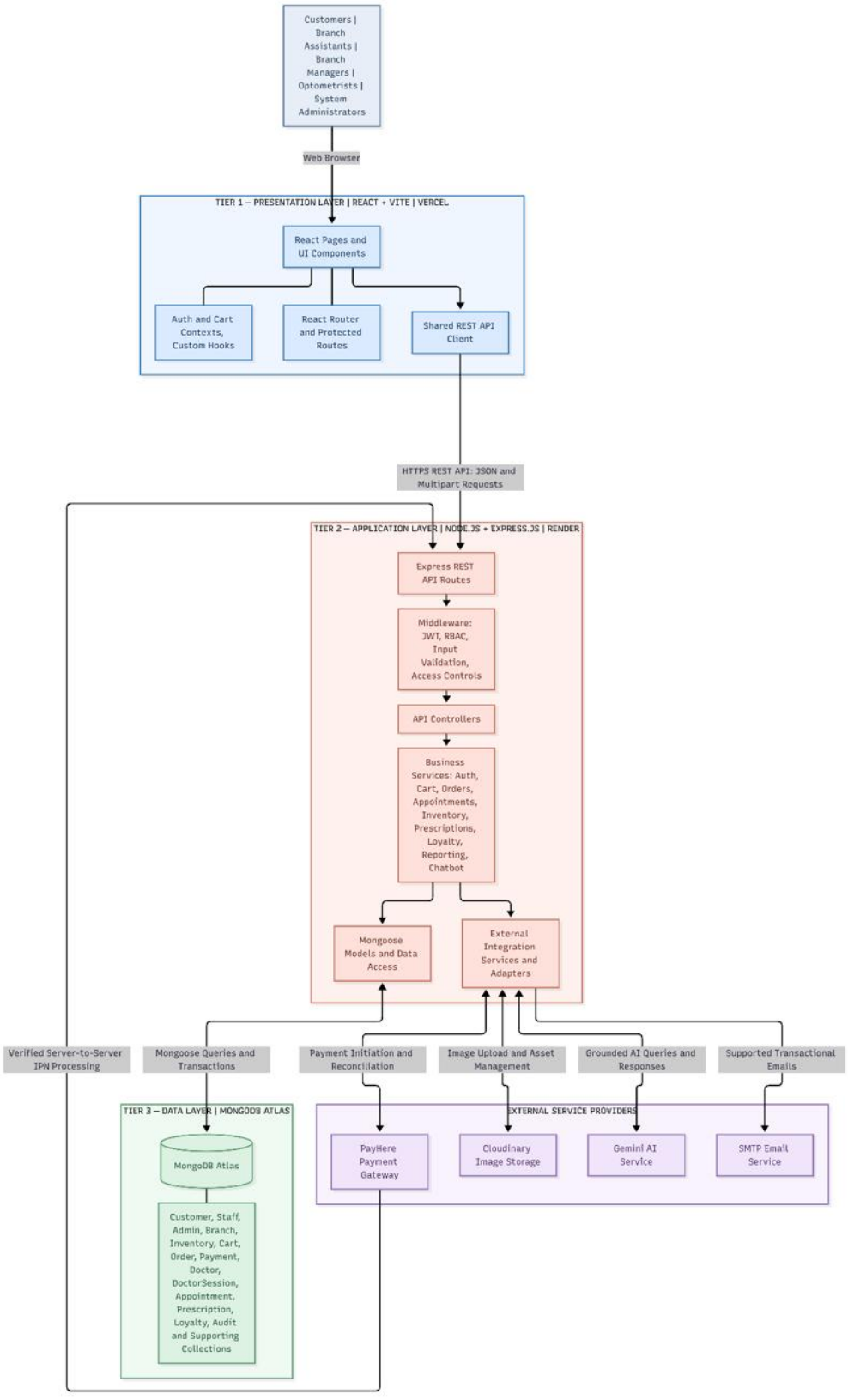


Figure 3.1: High-Level Three-Tier Architecture of OSMS

3.2.2 Presentation Layer

The presentation layer is responsible for the graphical user interface and user interactions.

OSMS uses React.js with Vite and React Router to implement a component-based Single Page Application (SPA).

The frontend provides different interfaces based on the authenticated user's role and supported operations.

Its responsibilities include:

- Rendering customer-facing and staff-facing interfaces.
- Managing client-side navigation.
- Handling form inputs and client-side validation.
- Maintaining application state through React components, contexts, and hooks.
- Displaying product catalogues, shopping carts, order details, appointment information, and administrative interfaces.
- Communicating with backend REST APIs.
- Displaying loading, success, validation, and error states.

The frontend includes protected routes to control navigation based on authenticated roles. However, these controls are not treated as the authoritative security boundary.

Backend authorization remains responsible for enforcing access restrictions on protected resources and operations.

The frontend communicates with the backend through a shared API client using HTTPS in the deployed environment.

3.2.3 Application Layer

The application layer contains the system's core business logic and application services.

It is developed using Node.js and Express.js, with Mongoose providing database integration.

The backend is responsible for:

- User authentication and JWT generation.
- Role-based authorization.
- Customer and staff account management.
- Input validation and business-rule enforcement.
- Product catalogue and inventory operations.
- Shopping-cart persistence and checkout processing.

- Order creation and lifecycle management.
- Payment initiation and verification.
- Doctor Session management and appointment booking.
- Clinical prescription recording and retrieval.
- Loyalty point calculations.
- AI chatbot request processing.
- Sales and inventory reporting.
- Audit-event generation.
- Integration with external service providers.

The backend separates these responsibilities into routes, controllers, services, models, middleware, validation components, and provider adapters.

This modular structure reduces direct dependencies between different functional areas and supports targeted testing and maintenance.

Business-critical operations are executed through backend services rather than relying on client-submitted calculations or unauthorized frontend state changes.

3.2.4 Data Layer

The data layer provides centralized storage and retrieval of business and clinical information.

OSMS uses MongoDB Atlas as its document-oriented database and Mongoose for schema definitions, validation, indexing, and data access.

The database maintains information associated with:

- Customer, Staff, and Admin accounts.
- Branches and inventory.
- Shopping carts and orders.
- Order items and payments.
- Doctors, Doctor Sessions, and appointments.
- Clinical prescriptions.
- Loyalty transactions.
- Reorder alerts.
- AI chatbot knowledge and usage records.
- Audit events and notification records.

MongoDB document references are used to maintain relationships between relevant entities. Embedded subdocuments are used where appropriate for data such as shopping-cart items, order-status histories, and purchase-time information.

The system also uses indexes and transactions to support data integrity and coordinated operations.

For example, checkout transactions coordinate order creation and inventory reservation, while appointment-booking transactions maintain consistency between confirmed appointments and Doctor Session capacity.

3.3 Architectural Design Considerations

The three-tier architecture was selected to satisfy important system requirements and software quality objectives.

3.3.1 Separation of Concerns

The architecture assigns distinct responsibilities to the presentation, application, and data layers.

Frontend components focus on user interaction, backend services execute business rules, and database models manage persistent information.

This separation reduces unnecessary coupling and allows different parts of the application to be modified independently.

3.3.2 Maintainability

The modular backend structure supports targeted changes to individual business modules.

React components and shared frontend services support interface reuse and consistent API communication.

Architectural decisions, source-code organization, automated tests, and continuous integration processes support the system's maintainability objectives.

3.3.3 Scalability

The separation between frontend hosting, backend processing, and database storage allows these infrastructure components to be managed independently.

Vercel provides managed frontend delivery, Render.com hosts the backend service, and MongoDB Atlas provides managed database resources.

However, the architecture's ability to satisfy the original NFR-005 scalability requirement depends on the configured infrastructure and measured workload behavior. Automatic horizontal scaling and the complete original scalability target have not been independently established.

3.3.4 Reliability and Data Consistency

The backend uses validation, controlled business operations, database indexes, and transactions to maintain consistent records.

Transactions are particularly important for workflows involving inventory reservations, payment-related operations, and appointment capacity management.

The architecture supports reliable operation but does not independently guarantee the original 99.9% availability target defined in NFR-002.

3.3.5 Security

Protected application operations require authentication and authorization.

Sensitive business and clinical information is processed through backend-controlled interfaces rather than through direct browser access to the database.

The system also applies password hashing, input validation, role-based restrictions, ownership checks, and HTTPS-based communication.

The implementation of these controls supports the approved security design, while complete security and regulatory compliance remain dependent on verification evidence.

3.4 Database Design

3.4.1 Database Architecture

The OSMS database follows a document-oriented design using MongoDB Atlas.

The original SDS defined the initial core entities required to represent customers, employees, inventory, orders, appointments, payments, and chatbot-related information.

During development, additional collections and relationships were introduced to support approved architectural decisions and more detailed business workflows.

The final database design combines separate collections, document references, embedded structures, schema validation, indexes, and transactions.

Although relational normalization concepts were considered during data modelling, the implementation does not use a conventional relational Third Normal Form schema.

Instead, selected information is embedded or preserved as historical snapshots where required for operational efficiency and business consistency.

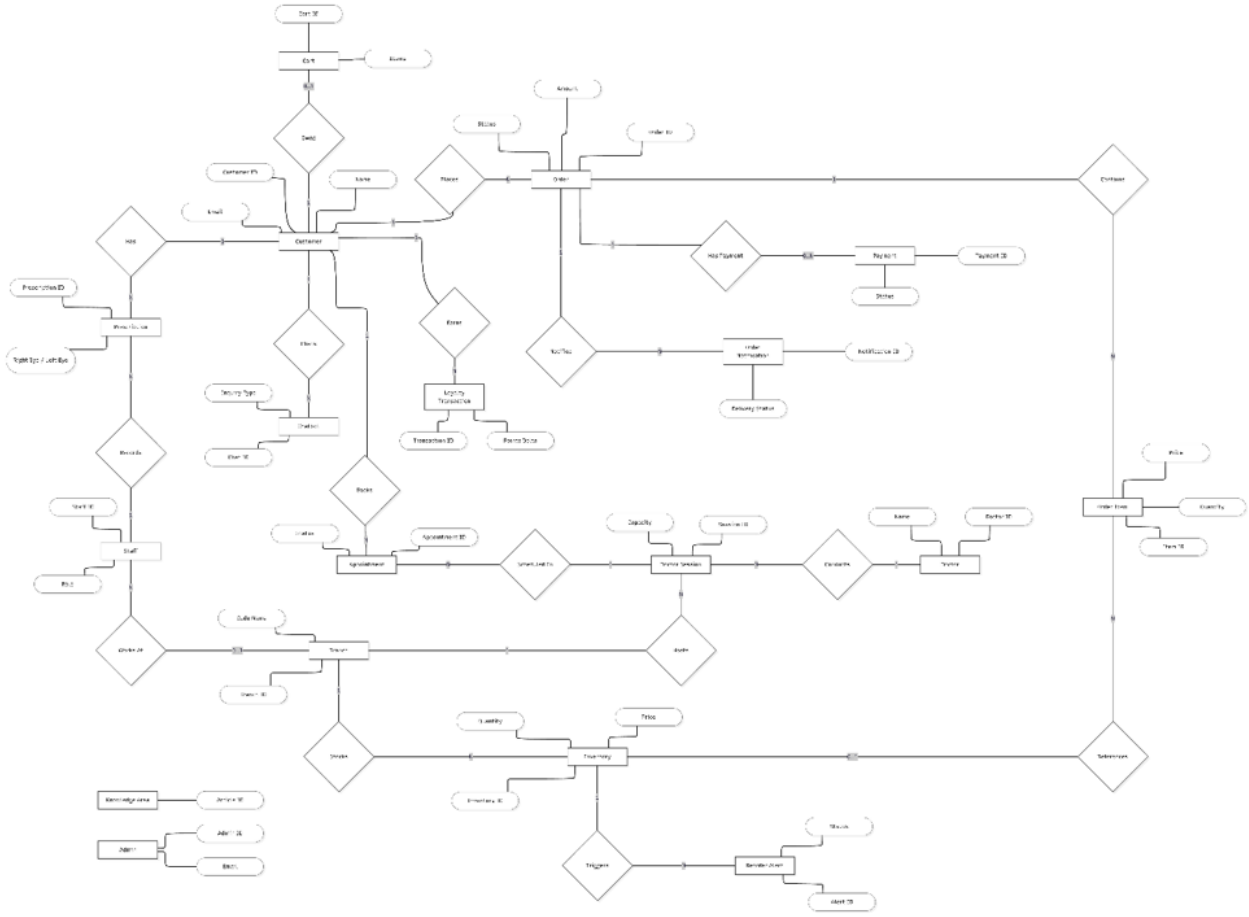


Figure 3.2: Logical Entity Relationship Diagram of OSMS

3.4.2 Core Database Entities

The principal database collections and their responsibilities are summarized below.

Table 3.1: Core Database Collections and Responsibilities

Collection	Responsibility
Customer	Stores customer identity, account, address, and loyalty information
Staff	Stores authorized staff information, roles, and branch associations
Admin	Stores System Administrator identity and authentication information
Branch	Maintains branch-related business information

Inventory	Stores branch-specific product and stock records
Cart	Maintains persisted customer shopping-cart items
Order	Records orders, statuses, reservations, and fulfillment details
OrderItem	Stores purchased items and purchase-time snapshots
Payment	Maintains payment attempts and relevant payment states
Doctor	Maintains the clinical service-provider directory
DoctorSession	Defines examination sessions and patient capacity
Appointment	Maintains customer examination bookings
Prescription	Stores customer clinical prescription records
LoyaltyTransaction	Records loyalty point earnings and approved reversals
ReorderAlert	Tracks inventory items requiring replenishment
InventoryConfig	Maintains relevant inventory configuration values
KnowledgeArea	Stores approved chatbot knowledge content
Chatbot	Maintains chatbot interaction metadata
ChatbotUsage	Tracks customer usage-window information
ImageUpload	Maintains inventory image-upload lifecycle information
OrderNotification	Records supported order-email notification activities

AuditEvent	Stores relevant critical business and administrative audit records

Additional collections support payment verification, reconciliation, manual refund evidence, and other operational requirements.

3.4.3 Entity Relationships

Several relationships are essential to the OSMS database architecture.

Customer and Order Relationship

A customer may place multiple orders, while each order is associated with its owning customer.

The OrderItem collection maintains the products purchased within an order, including relevant price and classification snapshots.

Branch and Inventory Relationship

Inventory records are associated with individual branches.

This relationship allows the application to enforce branch-specific inventory management and access restrictions.

Customer and Cart Relationship

Each customer can maintain one active persisted shopping cart.

Cart items are embedded within the Cart document and reference corresponding Inventory records.

Doctor Session and Appointment Relationship

Doctor Sessions reference Doctor records and branch information.

An Appointment is associated with a customer and an available Doctor Session.

A single Doctor Session may accommodate multiple customers according to its configured capacity.

Customer and Prescription Relationship

Prescription records are associated with the customer to whom the clinical information belongs.

Each prescription also identifies the authorized Optometrist responsible for recording the information.

Order and Payment Relationship

An order may have multiple payment attempts according to the approved payment lifecycle.

Payment records maintain provider-related references and processing states without storing customer card credentials.

3.4.4 Data Integrity and Consistency

The database architecture uses several mechanisms to maintain reliable information.

Schema Validation: Mongoose schemas define required fields, supported data types, permitted values, and relevant validation rules.

Indexes: Unique and compound indexes support identity constraints, lookup operations, payment processing, and scheduling requirements.

Transactions: MongoDB transactions coordinate operations that require consistent updates across multiple documents.

Document References: ObjectId references maintain relationships between related records.

Historical Snapshots: OrderItem records preserve purchase-time information, while order-status histories and payment events provide traceability.

Temporary Data Expiration: Chatbot usage records support expiration through a MongoDB Time To Live (TTL) index.

Audit Records: AuditEvent records preserve relevant critical activities through the approved append-only audit architecture.

The complete collection schemas and important constraints are documented in Chapter 2 of the final SDS.

3.5 Component Design

The component design organizes OSMS functionality into logical application modules and services.

The backend uses separate services for significant business operations, while the frontend uses reusable components, routes, contexts, and API integration modules.

The final SDS represents these responsibilities using domain-model and application-service diagrams.

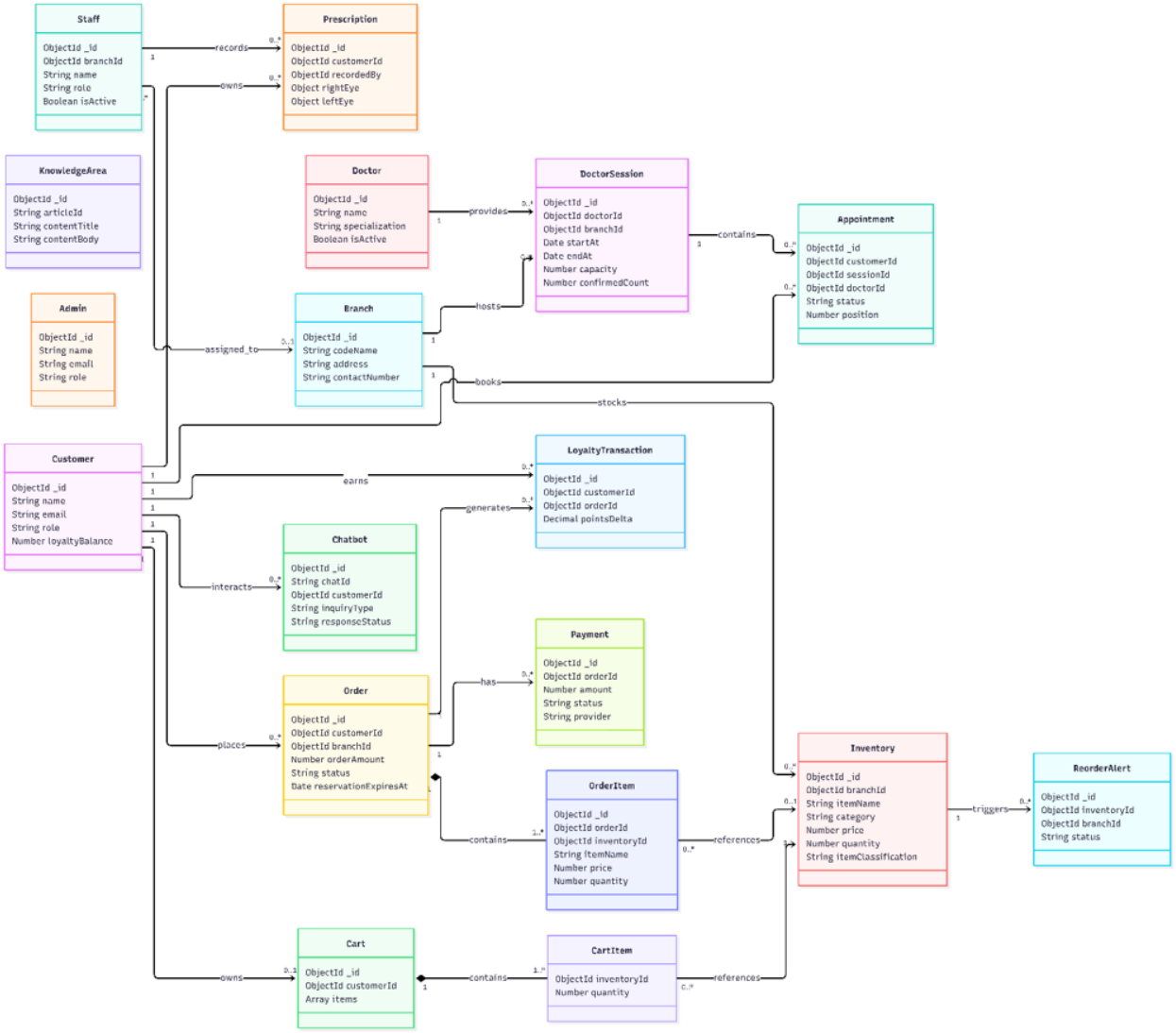


Figure 3.3: Core Domain Model Class Diagram

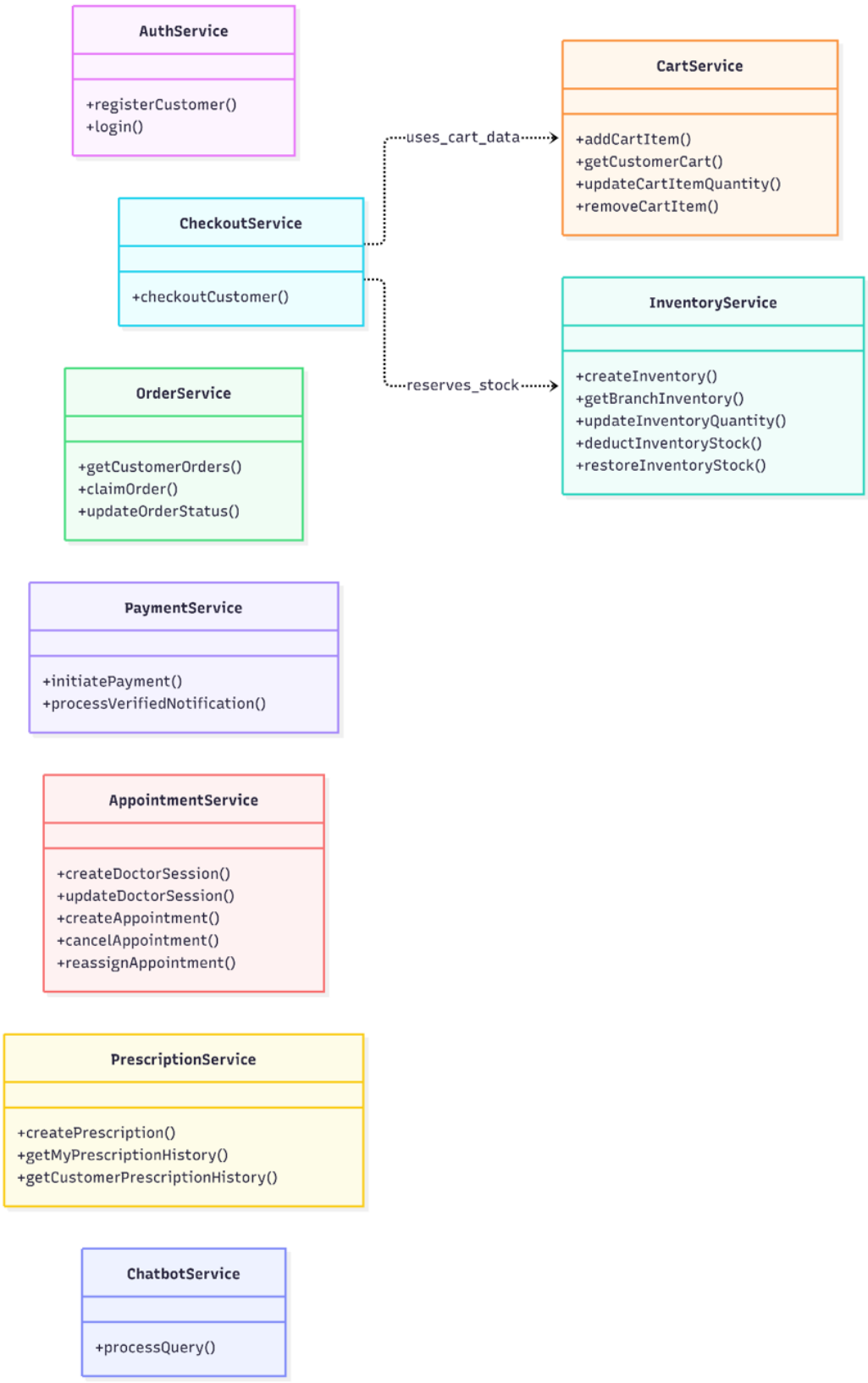


Figure 3.4: Application Service Operations Diagram

3.5.1 Authentication and User Management Component

The authentication component manages customer registration, login, password verification, token generation, password recovery, and protected access.

Customer, Staff, and Admin identities are maintained in separate MongoDB collections.

Public registration creates Customer accounts only. Staff and administrative account provisioning is performed through authorized administrative processes.

The backend generates signed JWT credentials after successful authentication.

3.5.2 Product Catalogue and Inventory Component

The product catalogue component retrieves available inventory information for customer-facing product discovery.

The inventory component manages branch-specific product records, stock quantities, prices, and supported product images.

Authorized inventory operations are restricted according to the user's role and branch.

The inventory design also integrates reorder alerts and Cloudinary-based image storage.

3.5.3 Shopping Cart and Checkout Component

The shopping-cart component manages customer-selected inventory items and quantities.

Cart information is persisted in MongoDB for authenticated customers.

The checkout component validates the customer's cart, verifies current inventory records, determines authoritative prices, checks product classifications, and creates orders.

Inventory reservation and order creation occur through backend-controlled transactional operations.

3.5.4 Order and Payment Component

The order component manages order records, authorized status transitions, fulfillment information, and customer order history.

The payment component communicates with PayHere, initiates payment attempts, validates payment notifications, and maintains payment status information.

The system separates successful order creation from successful payment confirmation.

Payment completion depends on verified provider notifications and approved reconciliation rules.

3.5.5 Appointment and Doctor Session Component

The appointment component manages customer bookings and relevant appointment lifecycle operations.

Doctor Sessions define available examination periods, assigned Doctors, branch locations, and patient capacity.

The application validates appointment eligibility, booking lead time, Doctor Session availability, and remaining capacity.

Confirmed bookings are maintained using transactional database operations.

3.5.6 Prescription Component

The prescription component manages the recording and retrieval of clinical optical information.

Authorized Optometrists can create relevant prescription records, while customers can access their own permitted prescription histories.

Prescription information is maintained separately from customer account records to preserve clinical history and enforce access restrictions.

3.5.7 Loyalty Component

The loyalty component manages approved loyalty-point calculations and transaction records.

Point calculations are performed through backend services using eligible order and payment information.

Loyalty transaction records provide traceability for point earnings and supported reversals.

The current implementation does not provide loyalty point redemption or tier-based reward functionality.

3.5.8 AI Chatbot Component

The chatbot component provides informational assistance to authenticated customers.

It uses internal knowledge retrieval and Google Gemini AI to generate responses for supported customer inquiries.

The backend enforces approved usage limits and restricts requests involving unsupported clinical interpretation or sensitive payment information.

Chatbot usage-window information is maintained in MongoDB, with the approved TTL mechanism supporting temporary record expiration.

3.5.9 Reporting and Audit Component

The reporting component provides sales and inventory information for authorized management users.

Reports apply the approved role and branch access restrictions.

The audit component maintains relevant critical business events and supports operational accountability through protected audit records.

3.6 Authentication and Authorization Design

3.6.1 Authentication Architecture

OSMS uses JWT-based authentication to identify customers and authorized staff members.

During login, the backend verifies the supplied credentials against the appropriate identity collection.

Passwords are validated against stored bcrypt hashes rather than plaintext passwords.

After successful authentication, a signed JWT is returned to the frontend and used to authenticate subsequent protected requests.

The current frontend stores JWT credentials in browser localStorage. This is an implementation characteristic documented in the final SDS and must be considered when evaluating residual security risks.

3.6.2 Role-Based Access Control

The application's authoritative RBAC model was established through ADR-002.

The five supported roles are:

Table 3.2: OSMS Application Roles and Authorized Functions

Role	Primary Authorized Functions
CUSTOMER	Product browsing, cart, checkout, orders, appointments, loyalty, chatbot, personal prescription history
BRANCH_ASSISTANT	Doctor Session coordination, appointment management, authorized order operations
BRANCH_MANAGER	Branch inventory, reorder alerts, authorized order operations, branch reporting
OPTOMETRIST	Clinical prescription recording and authorized prescription access
SYSTEM_ADMIN	Staff and Doctor administration, organization-wide reporting, authorized administrative operations

Authorization is enforced in the backend through middleware and service-level business rules.

Frontend route protection improves navigation and usability but does not replace backend access enforcement.

The authorization design additionally considers customer ownership and branch-level data restrictions.

The Doctor directory is maintained independently of authenticated staff accounts. Doctors do not have a separate authenticated OSMS role.

3.7 Major System Workflow Design

The final SDS contains sequence diagrams describing the interactions between the frontend, backend, database, and external services.

The following sections summarize the main workflows.

3.7.1 Customer Registration and Login

Customer registration collects the required identity and contact information.

The frontend performs initial validation before submitting the registration request to the backend.

The backend validates the data, checks identity uniqueness across relevant account collections, hashes the password, and creates the customer record.

During login, the backend verifies the credentials and account status before generating the JWT.

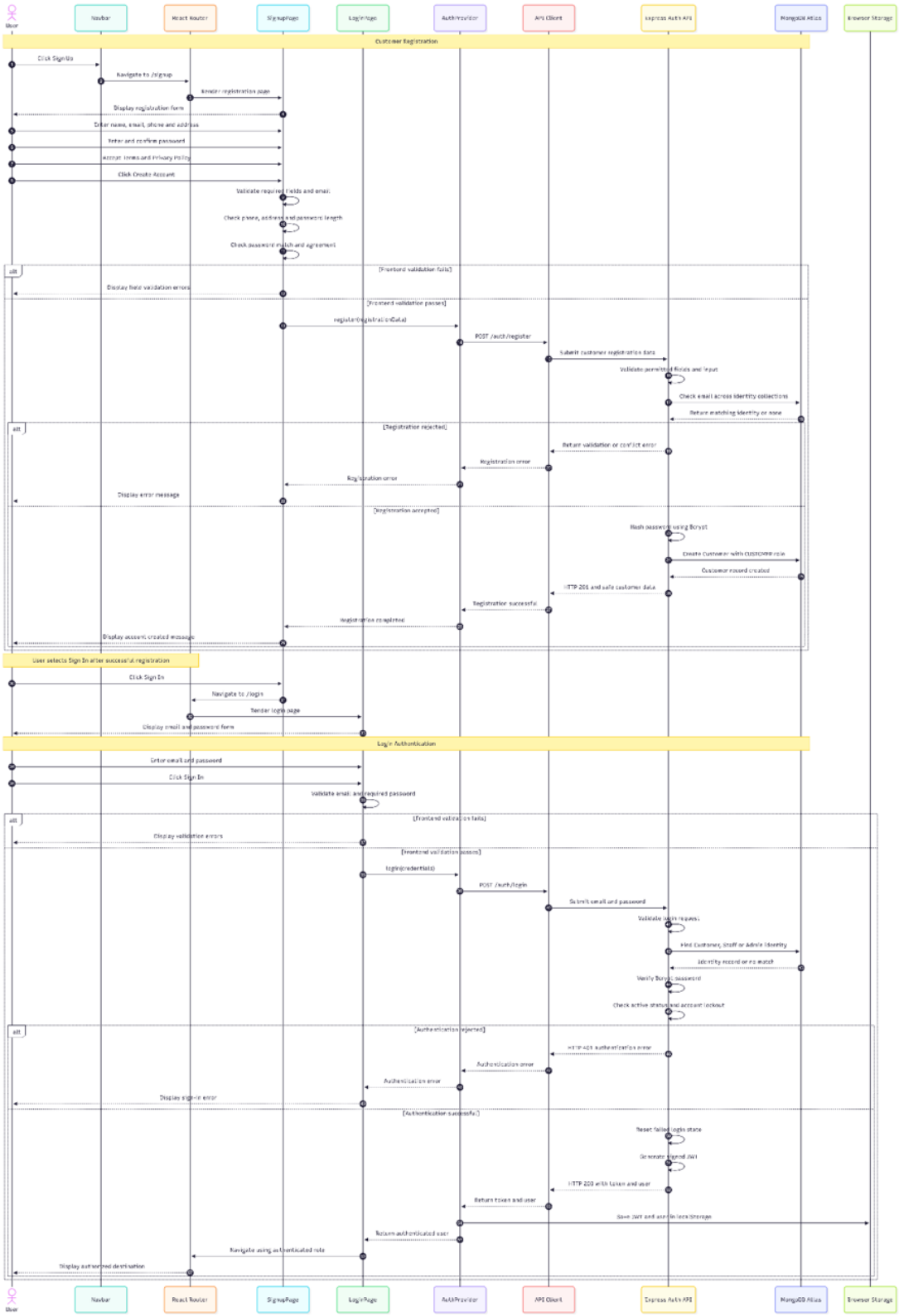


Figure 3.5: Authentication Sequence Diagram

3.7.2 Shopping Cart Management

Authenticated customers can add available products to their shopping cart, update quantities, remove items, and review the estimated total.

The frontend communicates with the cart API through the application's shared state and API integration components.

The Cart collection maintains the selected inventory identifiers and quantities.

Adding an item to the cart does not independently reserve or deduct inventory stock.

Stock and price validation take place during checkout.

3.7.3 Checkout and Payment Processing

The checkout process begins when an authenticated customer confirms the contents of the shopping cart.

The backend retrieves the persisted cart and current inventory records.

It then validates stock quantities, applicable item classifications, and authoritative prices.

If validation succeeds, a MongoDB transaction creates the Order and OrderItem records, deducts reserved stock, establishes the reservation period, and clears the persisted cart.

The approved workflow uses a 30-minute stock reservation period for unpaid orders.

Payment is initiated separately through PayHere.

The backend generates the required signed payment information and processes the provider's server-to-server payment notification.

Successful payment is established only after the required verification checks are completed.



Figure 3.6: Checkout and Order Confirmation Sequence Diagram

The browser payment return URL is not treated as authoritative proof of payment success.

Payment failures, cancellations, expired reservations, and reconciliation conditions are processed according to ADR-011.

3.7.4 Appointment Booking

The implemented appointment workflow uses capacity-based Doctor Sessions.

Customers can book appointments through a four-step process:

1. Review Customer Details.
2. Choose Doctor.
3. Select Date and Doctor Session.
4. Review and Confirm Booking.

The current appointment-booking scope is restricted to Gampaha Main Branch.

The selected Doctor Session must begin at least 24 hours after the booking request.

The backend validates session eligibility, Doctor activity status, booking lead time, remaining capacity, and duplicate booking conditions.

Successful bookings receive CONFIRMED status and an assigned queue position.

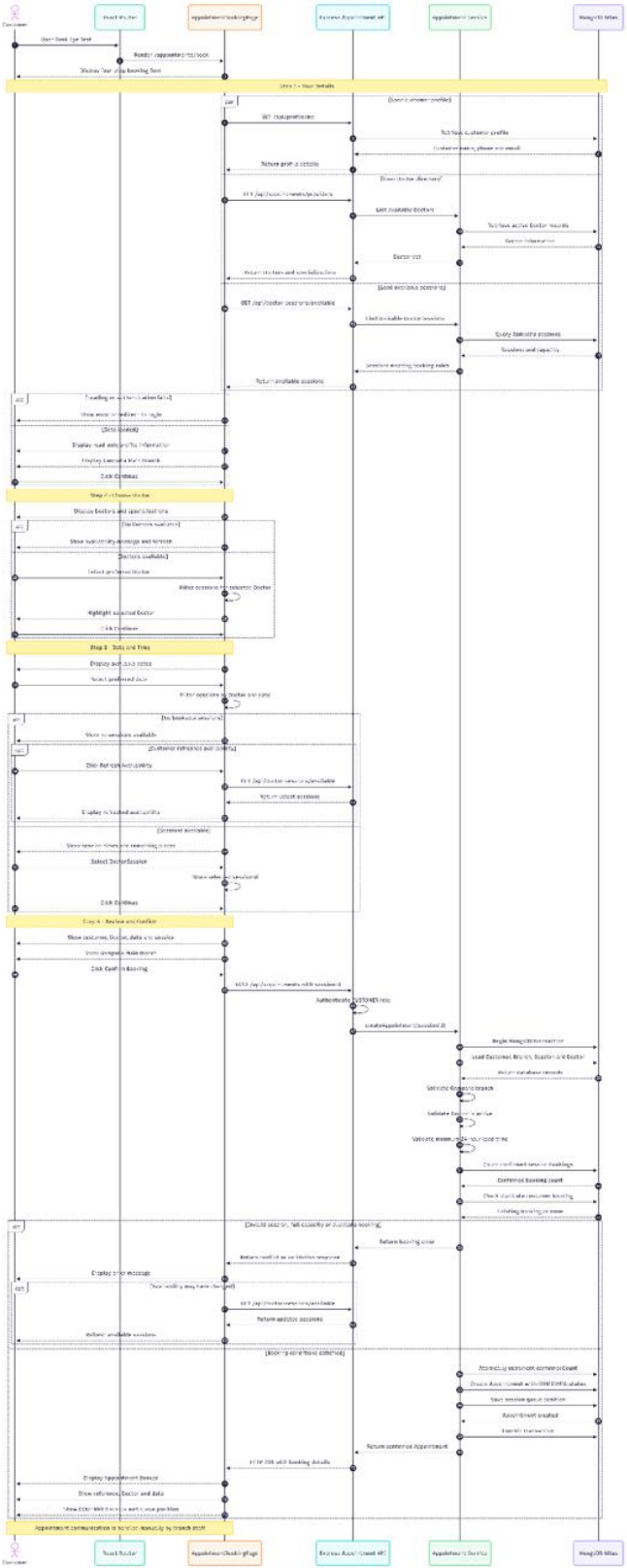


Figure 3.7: Appointment Booking Sequence Diagram

Appointment-related communication is handled manually by authorized branch staff in accordance with ADR-009.

Automated appointment reminder emails and SMS messages are excluded from the current implementation.

3.7.5 Order Lifecycle Management

The order lifecycle follows ADR-004.

The principal order statuses are:

Table 3.3: Order Lifecycle Status Definitions

Order Status	Description
ORDER_CREATED	Order created; payment or further processing may remain pending
PROCESSING	Order is being processed after the relevant payment confirmation
READY_FOR_PICKUP	Order is ready for collection
PICKED_UP	Customer pickup has been recorded
COMPLETED	Order has reached its completed state
CANCELLED	Order has been cancelled according to the approved rules

Authorized staff operations determine permitted order transitions.

Status histories preserve relevant transition information.

Supported order-status notifications are handled according to ADR-013.

3.7.6 Clinical Prescription Management

Clinical prescription recording is restricted to authorized Optometrists.

The system associates prescription records with the relevant customer and the responsible recording user.

Customers can retrieve their permitted prescription histories through the customer-facing application.

The design maintains the separation between appointment service-provider records and authenticated clinical staff responsibilities.

3.7.7 AI Chatbot Interaction

Customers access the chatbot through the frontend interface.

The backend validates authentication, request content, and the applicable chatbot usage allowance.

Supported requests are matched against approved internal knowledge records.

Relevant information is passed through the Gemini AI integration to generate grounded informational responses.

The backend applies the approved usage limit of ten question-and-response exchanges within a 30-minute usage window.

Unsupported or restricted requests receive controlled responses.

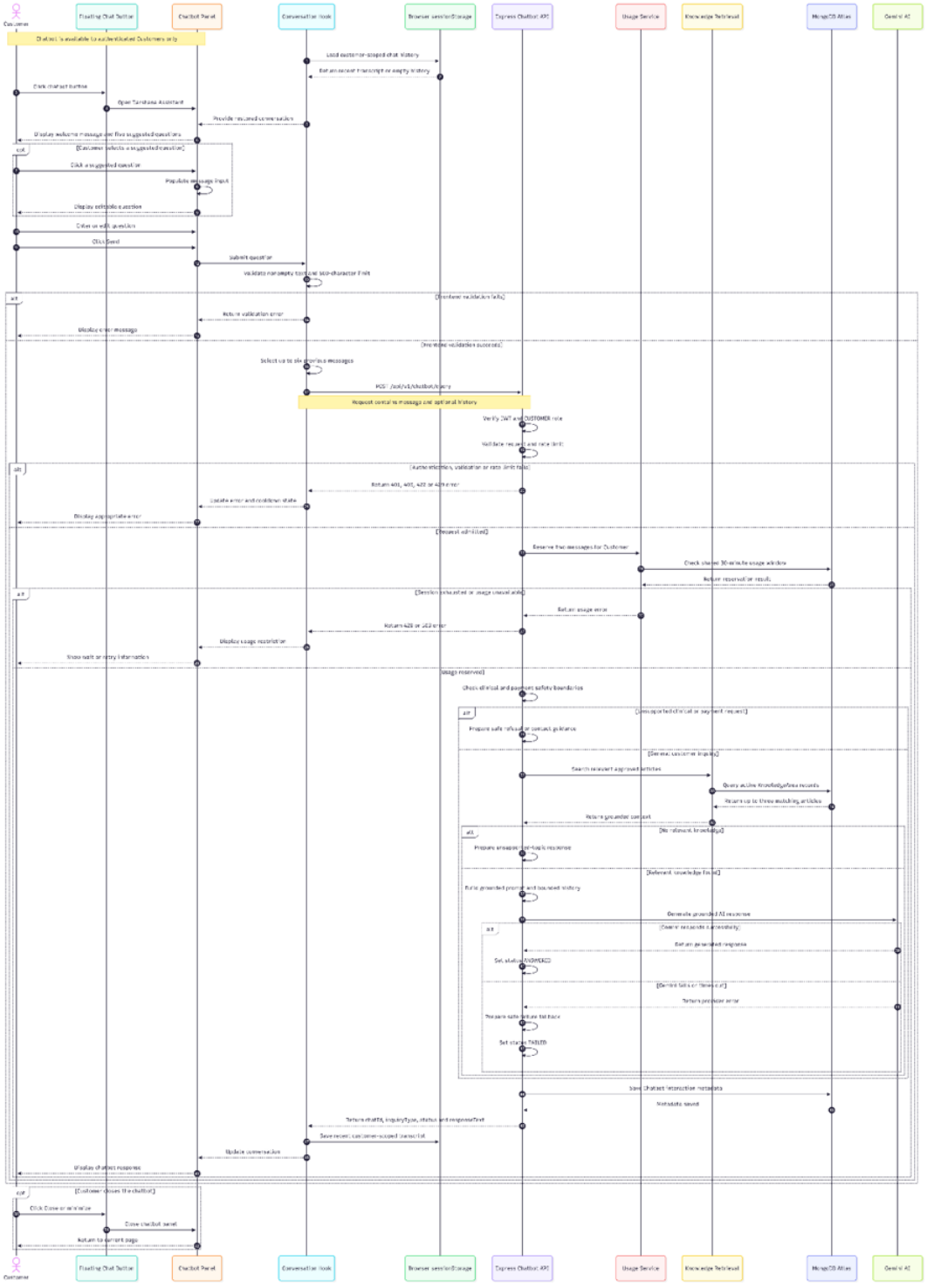


Figure 3.8: Chatbot Interaction Sequence Diagram

3.8 User Interface and User Experience Design

The user interface was designed to provide accessible and understandable navigation for the system's different user groups.

React-based components are used to present customer, staff, and administrative interfaces.

The final SDS documents screen mock-ups and navigation flows for major activities, including:

- Homepage and product discovery.
- Customer registration and login.
- Product information.
- Shopping-cart management.
- Checkout and payment.
- Appointment booking.
- Doctor Session administration.
- Order processing.
- Inventory management.
- Clinical prescription management.
- Sales and inventory reporting.

3.8.1 Customer Interface Design

Customer interfaces focus on product discovery, online purchasing, appointment booking, and permitted personal information.

The shopping and appointment workflows are divided into understandable steps to reduce interaction complexity.

3.8.2 Staff Interface Design

Staff interfaces expose features according to assigned roles.

Branch Assistants, Branch Managers, and Optometrists are provided with access to relevant operational or clinical functions.

3.8.3 Administrative Interface Design

The administrative interface supports approved system-management operations and access to authorized business information.

3.8.4 Accessibility Considerations

The final SDS includes accessibility-related considerations such as navigation consistency, interface clarity, appropriate feedback, and responsive design.

However, full accessibility compliance should not be assumed without appropriate evaluation evidence.

3.9 Security Design

Security is an important architectural concern because OSMS handles customer information, clinical prescription records, payment-related transactions, and administrative operations.

The security design establishes the controls and boundaries used to protect system resources.

3.9.1 Authentication Security

The system uses bcrypt password hashing and JWT-based authentication.

Authentication processes include server-side validation and applicable controls for unsuccessful login attempts.

Password recovery uses the approved reset-token workflow.

3.9.2 Authorization Security

Backend middleware and services enforce the authoritative five-role RBAC model.

Authorization restrictions include role checks, customer ownership validation, and branch-scoped permissions.

The system does not rely exclusively on frontend navigation controls to protect backend resources.

3.9.3 Data Protection

Sensitive customer and clinical information is protected through restricted backend access, controlled API responses, and appropriate authentication.

The application uses HTTPS for communication between deployed components.

The system does not directly collect or store customer payment-card credentials.

3.9.4 Input Validation and Error Handling

The backend applies input validation to protected operations and business-critical requests.

Validation is used to reject invalid identities, unauthorized operations, malformed requests, and violations of business rules.

Controlled error responses reduce the risk of exposing sensitive internal information.

3.9.5 Payment Security

PayHere payment processing follows the approved payment architecture in ADR-011.

Payment notifications undergo backend verification before relevant payment and order records are updated.

The application also applies controls for duplicate events, inconsistent payment information, and financial reconciliation conditions.

3.9.6 Audit Logging

The audit architecture defined by ADR-012 maintains protected, append-only records for approved critical operations.

These records support accountability and traceability for relevant administrative, financial, inventory, and clinical activities.

The approved minimum audit-retention requirement is seven years. Complete long-term retention and archival guarantees require separate operational verification.

3.9.7 Security Verification Boundaries

Security assessment activities under DDP-085 provide evidence of implemented controls and identified residual risks.

The final SDS records that no unresolved Critical or High dependency vulnerabilities were reported at the verified assessment baseline, while Moderate-level dependency findings remain documented.

The use of JWT credentials in browser localStorage is also recognized as a residual security consideration.

These findings do not independently establish complete regulatory or security compliance.

3.10 External Service Integration Design

OSMS integrates several external services through the backend.

Table 3.4: External Service Integration Responsibilities

Service	Integration Purpose
PayHere	Online payment processing and verified payment notifications
Cloudinary	Inventory image upload, storage, and management
Google Gemini AI	AI-assisted customer inquiry responses
SMTP Email Service	Password recovery and approved order-related email notifications

3.10.1 PayHere Integration

Payment operations are initiated through backend-controlled requests.

The backend validates provider notifications and maintains relevant payment and order records.

3.10.2 Cloudinary Integration

Inventory image uploads are handled through the authorized backend workflow.

Cloudinary stores and delivers hosted product images, while MongoDB stores relevant image references.

3.10.3 Gemini AI Integration

The chatbot service retrieves applicable knowledge content before communicating with the AI provider.

The integration includes approved request validation, usage controls, and fallback behavior.

3.10.4 Email Integration

SMTP-based email functionality supports approved customer communication events, including password recovery and relevant order-status notifications.

Automated appointment reminders are not part of the currently approved implementation.

3.11 Deployment Architecture

3.11.1 Cloud Deployment Overview

The original SDS proposed an AWS-based deployment architecture.

During development, the hosting design was revised through approved architectural decisions and deployment requirements.

The current implementation-aligned deployment architecture uses:

- Vercel for frontend hosting.
- Render.com for backend hosting.
- MongoDB Atlas for database services.
- Cloudinary for inventory image storage.

External services such as PayHere, Gemini AI, and SMTP email providers support the application through their respective interfaces.

The backend is deployed as a Docker-containerized Node.js/Express application.

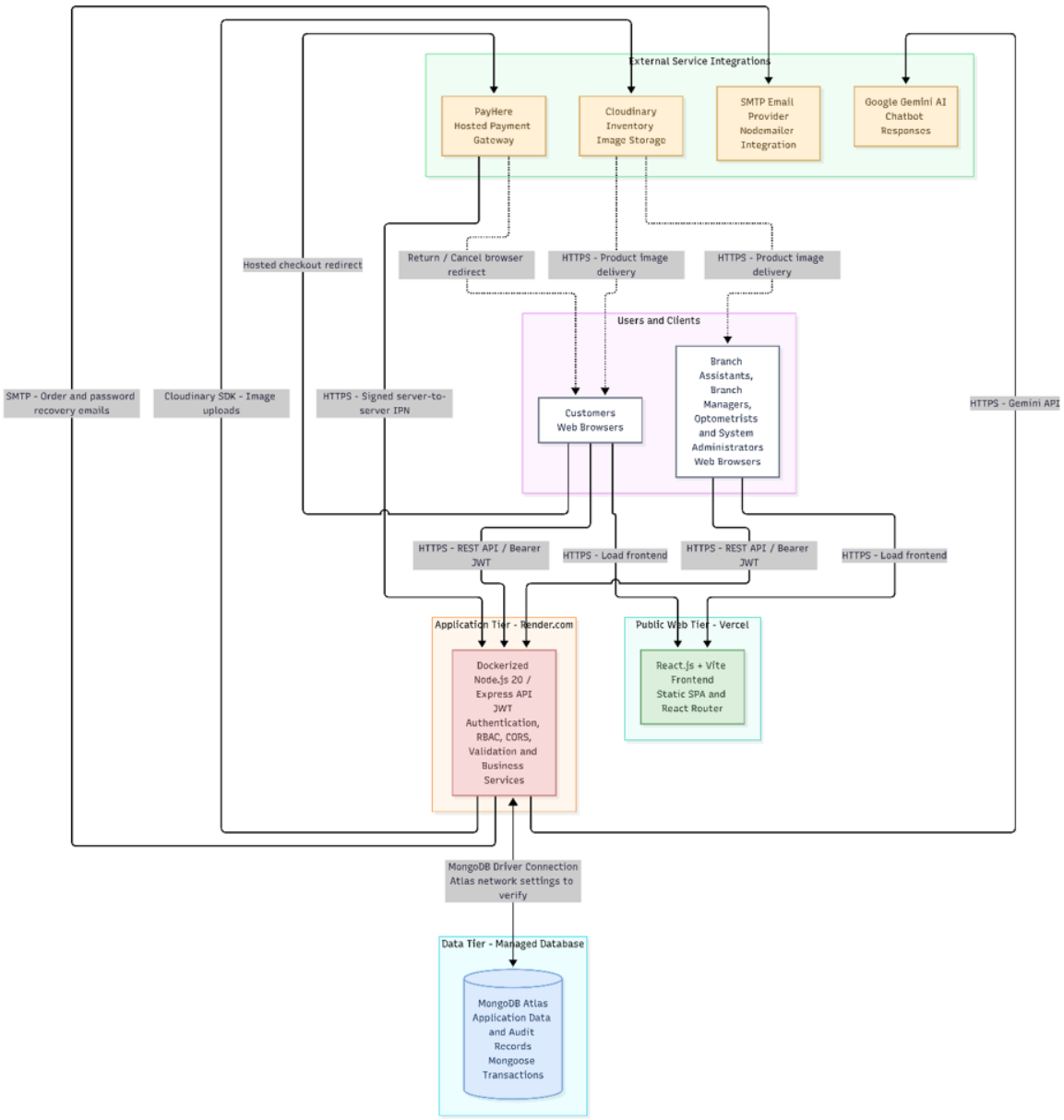


Figure 3.9: OSMS Cloud Deployment Architecture

3.11.2 Frontend Hosting

The React/Vite frontend is hosted on Vercel.

The deployment configuration supports React Router navigation through an SPA rewrite.

The approved deployment strategy uses the develop branch as the tracked source for the Vercel Production environment.

Frontend requests are sent to the configured Render backend API endpoint through HTTPS.

3.11.3 Backend Hosting

The Node.js/Express backend is hosted on Render.com using Docker.

The backend provides REST API endpoints, processes business operations, manages database communication, and integrates with external providers.

The backend exposes an API health-check endpoint for operational verification.

Sensitive configuration values are supplied through server-side environment variables.

3.11.4 Database Hosting

MongoDB Atlas provides managed cloud-based data storage.

Database access is performed through the backend rather than directly from the browser.

The hosting configuration must support the required data integrity and transaction capabilities.

3.11.5 Environment Configuration

The deployment design distinguishes local development from the cloud-hosted application environment.

The current strategy does not require three separately provisioned cloud environments corresponding to development, staging, and production.

Instead, the project uses local development environments and a configured cloud-hosted application environment for approved integration testing, user acceptance testing, and demonstration activities.

Actual testing must use authorized configurations and appropriate non-production data.

3.11.6 Continuous Integration and Deployment

GitHub Actions provides automated validation for frontend and backend changes.

Frontend checks include linting, formatting verification, tests, coverage generation, and production build validation.

Backend checks include linting, formatting verification, automated tests, coverage generation, and applicable dependency auditing.

Approved changes are integrated through Pull Requests into the develop branch.

Vercel and Render.com provide the corresponding application hosting and deployment capabilities.

A successful CI workflow confirms the configured checks passed for the tested code revision but does not independently establish successful deployment or full production readiness.

3.12 Architectural Decisions and Design Evolution

During implementation, approved Architectural Decision Records were used to document important design refinements.

The following table summarizes the principal decisions reflected in the final design.

Table 3.5: Architectural Decision Records Summary

ADR	Design Area
ADR-001	Clinical prescription storage
ADR-002	Authoritative RBAC role model
ADR-003	Shopping cart persistence and checkout boundaries
ADR-004	Order lifecycle and tracking
ADR-005	Loyalty calculation and persistence
ADR-006	Inventory reorder-alert design
ADR-007	Appointment scheduling and lifecycle
ADR-008	Inventory images and Cloudinary integration
ADR-009	Appointment communication policy
ADR-010	AI chatbot architecture
ADR-011	Secure payment integration
ADR-012	Immutable audit-trail architecture

ADR-013	Order-status notifications
ADR-014	Sales and inventory reporting
ADR-015	Render.com backend hosting fallback

The ADR process helped preserve traceability between the initial SDS and the implemented architecture.

Significant changes included the consolidation of operational roles, introduction of a dedicated prescription collection, definition of the cart persistence model, adoption of capacity-based Doctor Sessions, refinement of payment verification, and replacement of the originally proposed AWS hosting architecture.

These decisions were incorporated into the final SDS while preserving the original approved design documents as historical baselines.

3.13 Design Limitations

Although the implemented architecture addresses the main system requirements, several design limitations remain.

The current product catalogue does not provide every filtering capability originally described in the SRS.

Automated inter-branch stock transfers and historical inventory-movement reporting are not included in the current implementation.

Loyalty point redemption and tier-based loyalty rewards are not implemented.

Automated appointment reminders are excluded under the approved communication policy.

The application does not include third-party courier tracking or medical diagnostic hardware integration.

The chatbot is intended for supported informational assistance and does not provide unrestricted clinical advice or automatic human-agent escalation.

The current deployment architecture does not independently guarantee the original availability and scalability targets.

Operational backup and restoration guarantees, long-term audit retention, and complete regulatory compliance require additional verification.

These limitations and their implications for the achievement of project objectives are evaluated further in Chapter 6.

3.14 Chapter Summary

This chapter presented the final system design of the Optical Shop Management System.

The application uses a three-tier MERN architecture that separates frontend presentation, backend business processing, and MongoDB-based data persistence.

The database design supports the main retail, clinical, and administrative entities through document references, embedded structures, validation, indexes, and transactions.

The component architecture separates authentication, inventory, orders, payments, appointments, prescriptions, loyalty, chatbot, reporting, and audit responsibilities.

The security design applies backend-controlled authentication, authorization, input validation, payment verification, and sensitive-data protection.

The cloud deployment architecture uses Vercel, Render.com, MongoDB Atlas, and approved external service integrations.

The final design incorporates architectural refinements documented through ADR-001 to ADR-015, providing traceability between the original requirements and the implemented system.

The following chapter describes the implementation of these design components, including the selected technologies, source-code structure, database implementation, CI/CD practices, and engineering challenges encountered during development.

CHAPTER 4 : SYSTEM IMPLEMENTATION

4.1 Introduction

System implementation is the process of transforming an approved software design into a functioning software application through programming, database development, system integration, and software verification.

The Optical Shop Management System (OSMS) was implemented to support the retail, clinical, customer-service, and administrative requirements of Darshana Opticals (Pvt) Ltd. The implementation followed the architectural structure defined in the Software Design Specification (SDS), with subsequent refinements introduced through approved Architectural Decision Records (ADR-001 to ADR-015).

The system was developed using the MERN technology stack, consisting of MongoDB, Express.js, React.js, and Node.js. The application follows a three-tier architecture that separates frontend presentation, backend business logic, and database persistence.

Development was organized across four GitHub repositories: Frontend, Backend, Infrastructure, and Documents. This separation allowed different team members to work on application features, supporting infrastructure, and technical documentation while following a shared version-control and review process.

The implementation includes authentication and role-based authorization, product browsing, inventory management, persistent shopping carts, checkout, payments, order tracking, Doctor Session management, appointment booking, clinical prescriptions, loyalty points, AI-assisted customer support, reporting, and administrative operations.

This chapter describes the implemented technology stack, development environment, major application modules, database implementation, external service integrations, continuous integration and deployment processes, and significant engineering challenges encountered during development.

4.2 Technology Stack and Justification

The technologies were selected based on the approved requirements, architectural design, available development resources, maintainability objectives, and the need to support a centralized web-based application.

4.2.1 Frontend Technologies

Table 4.1: Frontend Technology Stack

Technology	Purpose	Justification
React.js	User interface development	Supports reusable, component-based interfaces and state-driven rendering
Vite	Development and build tooling	Provides efficient frontend development and production build generation
React Router	Client-side navigation	Supports navigation between public and role-protected application pages
JavaScript	Frontend programming language	Supports consistent development across the frontend and backend
HTML and CSS	Interface structure and styling	Supports responsive web interfaces
Vitest	Automated frontend testing	Supports testing of frontend application behavior
React Testing Library	Component testing	Supports testing frontend components through user-oriented interactions
ESLint	Static code analysis	Helps identify coding problems and enforce consistent conventions
Prettier	Code formatting	Maintains consistent source-code formatting

The frontend uses React 18, React Router 6, and Vite 6 according to the application's dependency configuration.

React was selected because its component-based structure supports the development of reusable interface elements such as navigation components, forms, product cards, data tables, and role-specific page layouts.

Vite supports the development workflow by providing a local development server and production build functionality.

React Router manages navigation within the Single Page Application while allowing the frontend to provide appropriate route restrictions for different user roles.

4.2.2 Backend Technologies

Table 4.2: Backend Technology Stack

Technology	Purpose	Justification
Node.js	Backend runtime	Supports server-side JavaScript and asynchronous request processing
Express.js	REST API framework	Provides routing and middleware support for modular backend services
Mongoose	MongoDB data modelling	Supports schemas, validation, indexes, and database operations
bcrypt	Password hashing	Supports secure storage and verification of password hashes
JSON Web Token	Authentication	Supports signed authentication tokens for protected API requests
Helmet	HTTP security headers	Provides configurable application-level HTTP security protections
CORS middleware	Cross-origin access control	Restricts permitted cross-origin browser requests
express-rate-limit	Request rate limiting	Supports request-abuse protection for applicable endpoints

Jest	Backend testing	Supports unit and integration testing
Supertest	HTTP API testing	Supports automated testing of Express endpoints
Swagger	API documentation	Provides documentation for supported REST API interfaces

The backend uses Express 4 and Mongoose 8 according to the repository dependency configuration.

Node.js and Express were selected to provide a JavaScript-based API application that coordinates business logic, authentication, database access, and external integrations.

Mongoose provides structured modelling for MongoDB documents, including validation rules, data references, and indexing.

4.2.3 Database Technologies

MongoDB Atlas was selected as the primary database service.

MongoDB uses a document-oriented storage model, which supports the persistence of business entities such as customers, inventory records, orders, appointments, and prescriptions.

The application uses Mongoose schemas to define the structure of stored documents and manage the relationships between collections.

MongoDB transactions are used in business workflows requiring coordinated updates across multiple documents, particularly checkout operations and appointment booking.

4.2.4 External Services

Table 4.3: External Service Integration

Service	Implemented Purpose
PayHere	Online payment initiation and payment-notification processing
Cloudinary	Inventory product-image upload, storage, and management

Google Gemini AI	AI-generated customer assistance based on approved knowledge
SMTP Email Service	Password recovery and supported order-related notifications

External services are integrated through backend-controlled operations to avoid exposing sensitive provider credentials within the frontend.

4.2.5 Deployment and Development Tools

Table 4.4: Development and Deployment Tools

Tool / Platform	Purpose
Git	Source-code version control
GitHub	Repository hosting, Issues, Pull Requests, and collaboration
GitHub Actions	Automated continuous integration
Docker	Container-based application configuration
Vercel	Frontend hosting

Render.com	Backend hosting
MongoDB Atlas	Cloud-hosted database
Visual Studio Code	Source-code development and project maintenance

These tools support the project's iterative development process and provide separate mechanisms for application development, code verification, deployment, and documentation.

4.2.6 — Technology Stack and Justification

Table 4.5: Technology Stack and Justification

Technology	Purpose	Justification
React + Vite	Frontend	Reusable components, efficient builds
Node.js + Express	Backend	Modular REST APIs
MongoDB + Mongoose	Database	Flexible documents and transactions
JWT + bcrypt	Authentication	Signed tokens and password hashing
Jest + Vitest	Testing	Automated verification
GitHub Actions	CI	Automated quality gates
Vercel + Render	Deployment	Managed cloud hosting

4.3 Development Environment and Repository Organization

4.3.1 Development Environment

The development environment consists of the frontend application, backend API server, MongoDB database, and the external services required by individual application modules.

During local development, frontend and backend services can be executed independently.

The React/Vite development server provides the user interface, while the Node.js/Express server handles REST API requests.

Environment-specific configuration is used to provide database connections, authentication settings, external service credentials, and other application options.

Sensitive configuration values are not intended to be stored directly in source-code repositories.

4.3.2 Repository Organization

OSMS uses four GitHub repositories.

Table 4.6: OSMS Repository Responsibilities

Repository	Main Responsibility
Frontend	React application, pages, components, navigation, API integration, and frontend tests
Backend	Express routes, controllers, services, models, middleware, and backend tests

Infrastructure	Container configuration, deployment documentation, environment contracts, and release procedures
Documents	SRS, SDS, ADRs, development standards, and project reports

The repositories separate concerns while supporting integration through shared API contracts and documented architectural decisions.

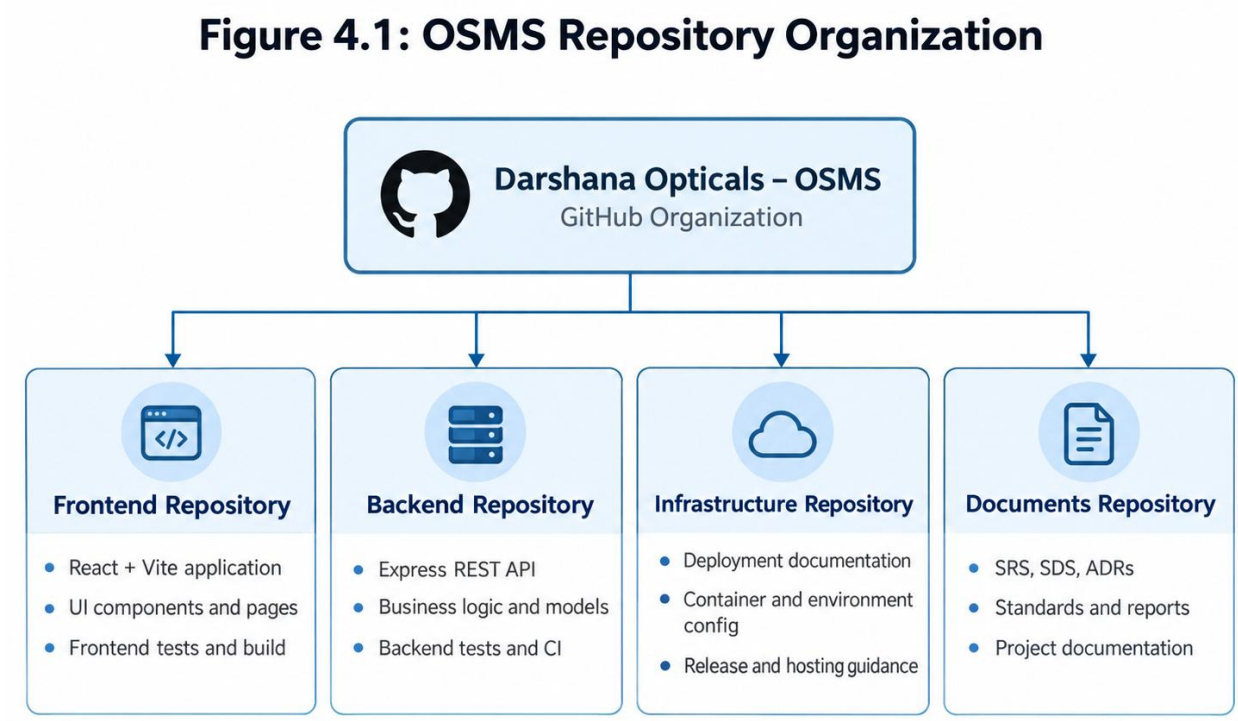


Figure 4.1: OSMS Repository Organization

4.3.3 Backend Code Organization

The backend follows a modular structure that separates request handling, business logic, persistence, and supporting functions.

A simplified representation is shown below.

```
backend/
├── src/
│   ├── config/
│   ├── constants/
│   ├── controllers/
│   ├── errors/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── services/
│   ├── utils/
│   ├── app.js
│   └── server.js
├── tests/
│   └── performance/
├── .github/
│   └── workflows/
├── package.json
└── package-lock.json
```

The main responsibilities are:

- Routes define REST API endpoints.
- Controllers coordinate HTTP requests and responses.
- Services implement domain-specific business logic.
- Models define MongoDB document structures.
- Middleware handles authentication, authorization, and relevant request processing.
- Configuration modules manage environment-dependent settings.
- Utility modules provide reusable supporting functions.

This separation allows business rules to remain within backend-controlled services rather than being duplicated across frontend interfaces.

4.4 Frontend Implementation

4.4.1 React-Based Application

The frontend was implemented as a React Single Page Application.

React components are used to construct public customer pages, protected customer interfaces, operational staff screens, and administrative views.

The application uses React Router for navigation between supported routes.

The interface communicates with the backend through REST APIs and processes returned information to update the displayed application state.

Reusable components and shared state-management mechanisms reduce duplication across related workflows.

4.4.2 Authentication and Protected Navigation

The frontend provides registration and login interfaces.

After successful authentication, the application maintains the relevant identity and role information used for interface navigation.

Protected routes restrict access to pages according to the authenticated user's role.

However, the frontend does not serve as the authoritative access-control boundary. Backend authorization independently verifies access permissions for protected API operations.

4.4.3 Product Discovery Implementation

The product catalogue retrieves product information from the backend.

Customers can browse available products, search by supported attributes, and filter results by category, brand, and price range.

The interface supports loading, empty-result, and error states.

Product information is retrieved from backend-managed inventory records rather than depending solely on static frontend data.

The current implementation does not provide every catalogue filtering capability originally described in the SRS. This limitation remains documented in the final requirements specification.

4.4.4 Shopping Cart Implementation

The shopping cart is available to authenticated Customers.

Customers can add available products, update item quantities, remove products, and review an estimated total.

The cart state is synchronized with the backend's persisted Cart collection.

The frontend presents the cart contents, while authoritative product prices and stock availability are validated by the backend during checkout.

This approach prevents the frontend's provisional calculations from becoming authoritative financial information.

4.4.5 Checkout and Payment Interfaces

The checkout interface allows Customers to review items before submitting an order-creation request.

Following successful order creation, the Customer can proceed to the payment process.

The payment interface communicates with the backend payment-initiation endpoint and redirects the Customer to the appropriate PayHere payment process.

Payment results are obtained from backend-controlled status information rather than relying only on browser redirects.

4.4.6 Appointment Booking Interface

The appointment-booking interface implements a four-step workflow.

1. Review Customer Details.
2. Choose Doctor.
3. Select Appointment Date and Doctor Session.
4. Review and Confirm Booking.

The interface retrieves available Doctors and Doctor Sessions through backend APIs.

Customers can select sessions with available capacity, review appointment information, and submit booking requests.

The backend performs final validation before an appointment is confirmed.

4.4.7 Role-Specific Operational Interfaces

The application includes interfaces for the five approved authenticated user roles.

Customers access retail and personal service functions.

Branch Assistants access supported order, Doctor Session, and appointment workflows.

Branch Managers access authorized inventory and reporting functionality.

Optometrists access clinical prescription functions.

System Administrators access supported administrative management operations and authorized reporting.

These interfaces follow the role boundaries defined by ADR-002.

4.4.8 Frontend Verification

The frontend project uses Vitest, React Testing Library, ESLint, and Prettier.

The repository provides scripts for executing tests, generating coverage reports, checking formatting, validating code quality, and creating production builds.

Detailed frontend test results and coverage evidence are discussed in Chapter 5.

4.5 Backend Implementation

4.5.1 REST API Architecture

The backend was implemented using Node.js and Express.js.

The system exposes REST API endpoints to support frontend operations.

Requests pass through the relevant middleware, route handlers, controllers, and business services.

The backend validates the request, determines the authorized operation, performs necessary database interactions, and returns a structured response.

This design maintains a clear separation between HTTP handling and domain-specific business logic.

4.5.2 Authentication Implementation

Authentication is implemented through the backend authentication service.

Customer, Staff, and Admin accounts are maintained in separate collections.

The login process searches the relevant collections and validates credentials using bcrypt.

The implementation also includes account-locking controls for repeated failed authentication attempts.

A successful login produces a signed JWT containing relevant authenticated identity information.

Representative Code Excerpt 4.1 — Password Hashing Configuration

```
const bcrypt = require('bcrypt');

const BCRYPT_SALT_ROUNDS = 12;

const passwordHash = await bcrypt.hash(
  password,
  BCRYPT_SALT_ROUNDS
);
```

Source: `backend/src/services/auth.service.js`

The source defines a bcrypt work factor of 12 rounds and uses the corresponding hashing utility for password storage.

This ensures that plaintext passwords are not stored directly in the account collections.

4.5.3 JWT Implementation

JWTs are created by the backend using the approved authentication configuration.

The token payload contains relevant identity and role information.

Where applicable, the user's branch identifier is also included.

Representative Code Excerpt 4.2 — JWT Creation

```
const createAccessToken = ({
  userId,
  role,
  branchId = null
}) => {
  const { jwtSecret, jwtExpiresIn } = getJwtConfig();
  const payload = { userId, role };

  if (branchId) {
    payload.branchId = branchId.toString();
  }

  return jwt.sign(payload, jwtSecret, {
```

```
    algorithm: JWT_ALGORITHM,
    expiresIn: jwtExpiresIn,
  });
};
```

Source: `backend/src/utils/jwt.js`

The backend uses the HS256 signing algorithm and obtains authentication configuration from the application's environment settings.

Token verification checks the token signature and validates the authenticated identity and role information.

Protected API endpoints also enforce the corresponding authorization requirements.

4.5.4 Role-Based Authorization

The backend enforces the five-role authorization model established through ADR-002.

Protected operations require a valid authenticated identity and the necessary permissions.

Certain operations include additional customer-ownership or branch-scope checks.

For example, customer-specific data must belong to the authenticated Customer, while Branch Managers are restricted to operations permitted within their assigned branch.

This prevents frontend route visibility from being treated as proof of backend authorization.

4.5.5 Product and Inventory Implementation

The product catalogue is supported by backend operations that retrieve applicable inventory information.

Inventory records maintain product attributes such as item name, category, brand, price, stock quantity, and image references.

Authorized inventory operations can update stock-related information according to applicable business rules.

The inventory module integrates with reorder-alert processing to identify low-stock conditions.

The approved reorder architecture uses an organization-wide threshold and role-specific management permissions.

Cloudinary integration supports hosted inventory product images.

4.5.6 Checkout and Order Creation

Checkout is one of the most important business-critical operations in OSMS because it coordinates the customer's shopping cart, product prices, inventory stock, order records, and subsequent payment workflow.

The backend checkout service:

1. Retrieves the authenticated Customer's persisted Cart.
2. Validates the selected inventory items.
3. Checks stock availability.
4. Validates product prices and classifications.
5. Calculates the authoritative order amount.
6. Creates Order and OrderItem records.
7. Reserves inventory quantities.
8. Establishes the payment reservation period.
9. Clears the persisted Cart after successful order creation.

MongoDB transactions are used to maintain consistency across these related operations.

Representative Code Excerpt 4.3 — Checkout Validation

```
if (inventory.quantity < cartItem.quantity) {
  throw new ConflictError(
    'Insufficient Inventory stock.',
    null,
    INSUFFICIENT_STOCK
  );
}

if (
  !inventory.itemClassification ||
  !VALID_CHECKOUT_CLASSIFICATIONS.includes(
    inventory.itemClassification
  )
) {
  throw new ValidationError(
    'One or more cart items are unavailable for checkout because their
classification is incomplete. Please contact the store.',
    null,
    CHECKOUT_ITEM_CLASSIFICATION_INVALID
  );
}
```

Source: `backend/src/services/checkout.service.js`

The checkout service validates both inventory availability and required item classification information before creating the relevant order records.

This validation prevents orders from proceeding with insufficient stock or incomplete classification data that could affect subsequent business operations.

The current implementation uses a 30-minute reservation window for unpaid orders.

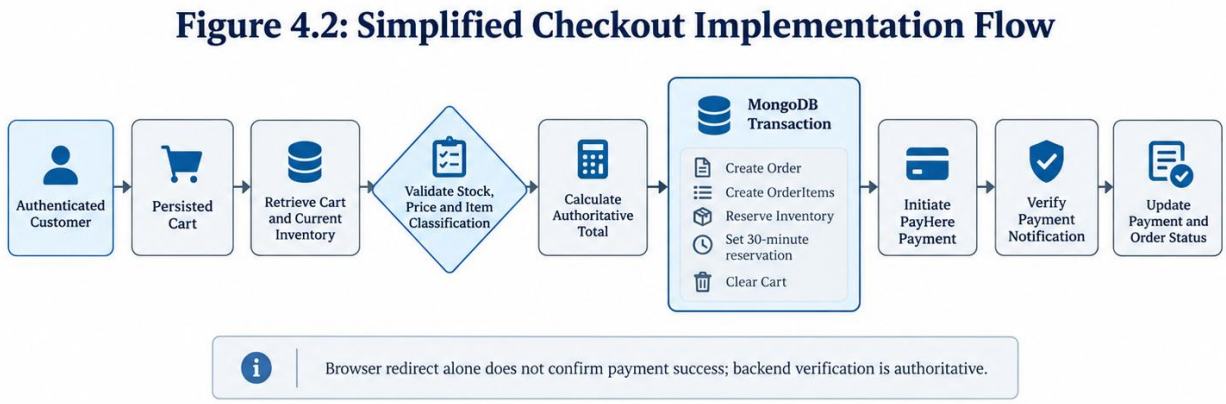


Figure 4.2: Simplified Checkout Implementation Flow

4.5.7 Order Lifecycle Implementation

The Order model stores customer ownership, branch information, order amount, delivery address, current status, status history, and reservation information.

Order status changes follow the lifecycle defined in ADR-004.

The supported order states include ORDER_CREATED, PROCESSING, READY_FOR_PICKUP, PICKED_UP, COMPLETED, and CANCELLED.

Representative Code Excerpt 4.4 — Order State and History

```
status: {
  type: String,
  enum: ORDER_STATUSES,
  default: ORDER_STATUS_VALUES.ORDER_CREATED,
},
statusUpdatedAt: {
  type: Date,
  default: Date.now,
},
statusHistory: {
  type: [orderStatusHistorySchema],
  default: [],
},
```

Source: `backend/src/models/order.model.js`

The Order schema maintains the current order status and a separate history of relevant status transitions.

This enables the system to provide order tracking while preserving information about previous lifecycle changes.

4.5.8 PayHere Payment Integration

Payment processing is implemented through the backend payment module and PayHere integration.

The backend initiates payment attempts using order-related information and generates the required signed payment parameters.

PayHere processes payment information through its hosted payment workflow.

The backend receives and verifies provider payment notifications before updating the relevant payment and order records.

Payment verification considers merchant identity, signature validity, amount, currency, and associated payment information.

The payment architecture also includes controls for duplicate notifications, failure conditions, reservation expiry, and reconciliation.

The application does not directly store customer payment-card credentials.

4.5.9 Doctor Session and Appointment Implementation

The appointment module uses a capacity-based Doctor Session architecture.

Doctors are maintained in a dedicated directory, independently of authenticated Optometrist staff accounts.

DoctorSession records contain the assigned Doctor, branch, start and end times, maximum capacity, and confirmed booking count.

Representative Code Excerpt 4.5 — Doctor Session Capacity Validation

```
capacity: {
  type: Number,
  required: true,
  min: [1, 'capacity must be at least 1.'],
  validate: {
    validator(value) {
      return Number.isInteger(value) && value > 0;
    },
    message: 'capacity must be a positive integer.',
  },
},
confirmedCount: {
  type: Number,
  default: 0,
  min: [0, 'confirmedCount cannot be negative.'],
},
```

Source: `backend/src/models/doctorSession.model.js`

This schema validates the structure of Doctor Session capacity information.

The appointment service is responsible for checking available capacity and enforcing the relevant booking rules during appointment creation.

The approved appointment implementation also enforces the minimum 24-hour booking lead time and the current Gampaha Main Branch restriction.

4.5.10 Clinical Prescription Implementation

Clinical prescription information is maintained in a dedicated Prescription collection.

The prescription module associates each clinical record with its relevant Customer and recording Optometrist.

Authorized clinical staff can record relevant prescription measurements, while Customers can retrieve their own permitted prescription history.

The implementation separates clinical records from general account information and enforces the applicable authorization requirements.

4.5.11 Loyalty Implementation

The loyalty module calculates eligible loyalty points using authoritative backend information.

Point calculations are based on the approved item-classification and eligible purchase rules.

LoyaltyTransaction records provide traceability for point earnings and supported reversals.

The current implementation supports automatic point accrual but does not include loyalty point redemption or tier-based reward functionality.

4.5.12 AI Chatbot Implementation

The AI chatbot is implemented using a frontend chat interface, backend chatbot services, MongoDB knowledge records, and Google Gemini AI.

The backend validates authenticated requests, applies usage restrictions, retrieves applicable internal knowledge, and communicates with the external AI service.

The approved usage allowance is ten question-and-response exchanges within a 30-minute window.

ChatbotUsage records maintain the applicable usage-window information.

The chatbot is restricted to approved informational assistance and does not provide unrestricted clinical diagnosis or payment-card handling.

4.5.13 Reporting and Audit Implementation

The reporting module retrieves relevant sales and inventory information for authorized management users.

Backend authorization controls the scope of information available to each role.

The application also supports approved reporting filters and exports.

The audit module stores relevant critical business and administrative events in an append-only AuditEvent collection.

These records support the traceability requirements established by ADR-012.

4.5.14 — Main Modules and Responsibilities

Table 4.7: Main Application Modules and Responsibilities

Module	Responsibility
Authentication	Login, registration, authorization
Inventory	Product records, stock, reorder alerts
Cart/Checkout	Cart persistence, order creation
Payments	PayHere initiation and verification
Orders	Lifecycle and tracking
Appointments	Doctor Sessions and bookings
Prescriptions	Clinical data recording and history
Loyalty	Eligible points and transactions
Chatbot	Knowledge retrieval and AI assistance
Reporting	Authorized sales and inventory reports

Audit	Critical business event records

4.6 Database Implementation

4.6.1 MongoDB and Mongoose Integration

The backend uses Mongoose to interact with MongoDB Atlas.

Mongoose schemas define the structure of application records and provide validation rules, indexes, document references, and other data-management functionality.

The database stores business data through separate collections rather than relational SQL tables.

4.6.2 Database Collections

The main implemented collections include Customer, Staff, Admin, Branch, Inventory, Cart, Order, OrderItem, Payment, Appointment, Doctor, DoctorSession, Prescription, LoyaltyTransaction, ReorderAlert, ChatbotUsage, KnowledgeArea, and AuditEvent.

Additional supporting collections maintain notification, payment reconciliation, and image-upload information.

The complete logical database design is presented in Chapter 3 and the final SDS.

4.6.3 Data Relationships

MongoDB ObjectId references associate related documents.

For example, Order records reference Customers, Inventory records reference Branches, and Doctor Sessions reference Doctors.

Embedded structures are used for information that belongs logically to a single record, such as order-status history and cart items.

The combination of references and embedded documents supports the application's domain-specific data requirements.

4.6.4 Database Indexing

Indexes are used to support identity uniqueness, query performance, operational lookups, and relevant business constraints.

The Order schema, for example, defines indexes associated with status and reservation expiry, as well as customer-order history retrieval.

Representative Code Excerpt 4.6 — Order Indexes

```
orderSchema.index({
  status: 1,
  reservationExpiresAt: 1,
  _id: 1,
});

orderSchema.index({
  customerId: 1,
  createdAt: -1,
});
```

Source: `backend/src/models/order.model.js`

The first index supports relevant queries involving order status and reservation expiry.

The second supports retrieving orders associated with a particular Customer, ordered by creation time.

4.6.5 Database Transactions

MongoDB transactions are applied to business operations that involve related records.

Checkout transactions coordinate order creation, inventory reservation, and cart updates.

Appointment booking uses transactional coordination to maintain consistency between appointments and Doctor Session capacity.

These mechanisms help reduce incomplete or conflicting states when multiple operations occur concurrently.

4.6.6 Data Validation and Security

Mongoose validation defines required fields, permitted values, and data types.

Backend service validation additionally applies business rules that cannot be enforced solely through database schemas.

Database access is performed through the backend application rather than directly from frontend components.

Sensitive credentials and connection details are supplied through environment configuration.

4.6.7 Data Retention and Temporary Records

The system maintains business records and historical information according to the approved domain requirements.

ChatbotUsage records support temporary expiration through a MongoDB TTL index.

AuditEvent records follow the append-only audit design, which specifies a minimum seven-year retention requirement.

Full operational retention, backup, archival, and recovery guarantees require separate deployment verification.

4.7 External Service Integration

4.7.1 Payment Gateway

PayHere integration is used for online payment operations.

The backend controls payment initiation, verifies payment notifications, and maintains payment-related records.

Provider credentials remain outside the frontend source code.

4.7.2 Cloudinary Image Management

Cloudinary is integrated to manage product images.

Authorized image-upload operations pass through backend-controlled interfaces.

MongoDB inventory records store the relevant image URLs and asset identifiers.

This separates image storage from application-managed business records.

4.7.3 Gemini AI Chatbot

The chatbot integration uses Google Gemini AI to generate responses based on approved knowledge information.

The backend restricts requests according to authentication, usage, and content rules.

4.7.4 Email Notifications

The application uses Nodemailer and a configured SMTP service for supported communication events.

These include password recovery and approved order-related email notifications.

Automated appointment email or SMS reminders are excluded from the current approved implementation.

4.8 Continuous Integration and Deployment

4.8.1 Version Control and Collaboration

Git and GitHub were used to manage source-code changes and coordinate development activities.

Feature branches were used to isolate development work.

GitHub Issues documented requirements, tasks, defects, and implementation decisions.

Pull Requests provided a mechanism for reviewing changes before integration into the shared development branch.

The project uses develop as its integration branch, while main is reserved for approved release management.

4.8.2 Backend Continuous Integration

The backend repository includes a GitHub Actions workflow that automatically validates code changes.

The workflow uses Node.js 20 and executes the project's defined validation scripts.

The principal validation activities include:

1. Repository checkout.
2. Node.js environment setup.
3. Dependency installation using npm ci.
4. ESLint validation.

5. Prettier formatting checks.
6. Jest tests with coverage collection.
7. Coverage report artifact upload.
8. Dependency security auditing for applicable branches and Pull Requests.

Representative Code Excerpt 4.7 — Backend CI Validation

```
- name: Install dependencies
  run: npm ci

- name: Run ESLint
  run: npm run lint

- name: Check formatting
  run: npm run format:check

- name: Run tests with coverage
  run: npm test -- --coverage
```

Source: `backend/.github/workflows/ci.yml`

These automated checks help identify quality problems before changes are accepted.

Test results and coverage reports provide evidence for the project's software testing activities.

Figure 4.3: Backend Continuous Integration Pipeline

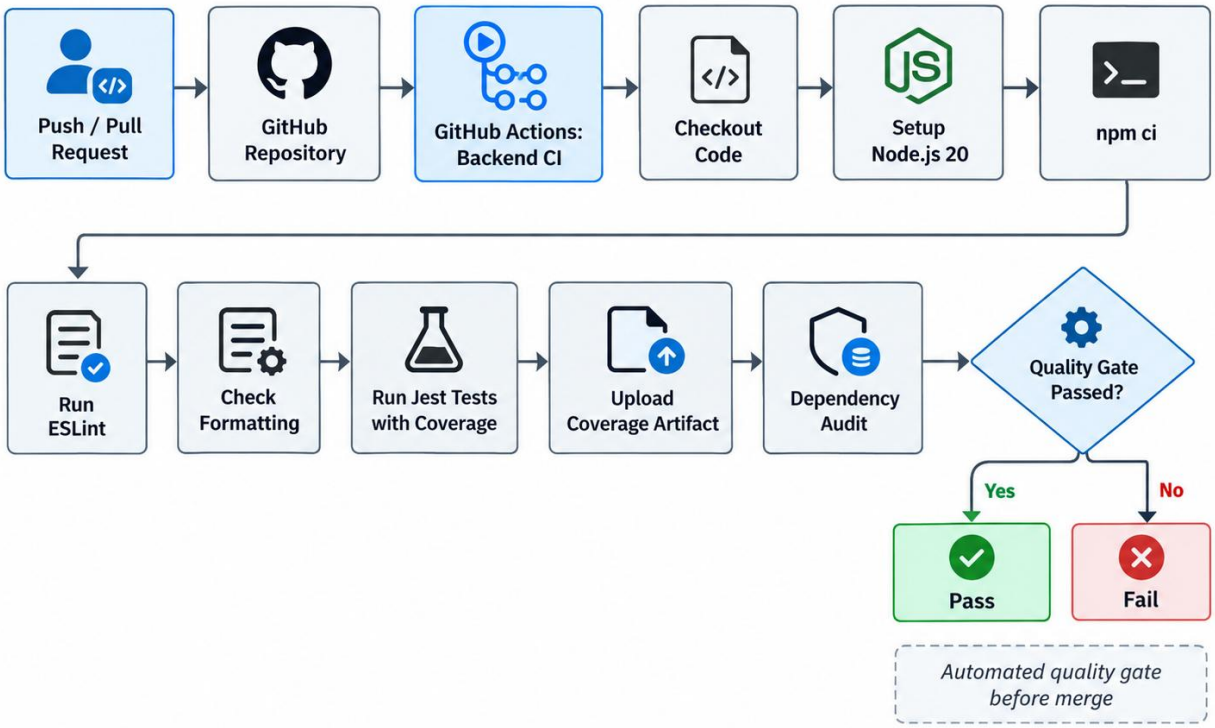


Figure 4.3: Backend Continuous Integration Pipeline

4.8.3 Frontend Continuous Integration

The frontend repository provides scripts for linting, formatting checks, component testing, coverage generation, and production builds.

The associated CI process supports automated verification of frontend changes.

The source-code checks help identify problems introduced during interface development and integration.

4.8.4 Frontend Deployment

The frontend is hosted on Vercel.

The deployed application uses the configured Vercel environment to serve the React/Vite frontend.

SPA routing configuration allows client-side routes to be loaded directly through the hosted application.

The approved deployment approach tracks the develop branch for the Vercel Production environment.

Figure 4.4: OSMS Deployment Workflow

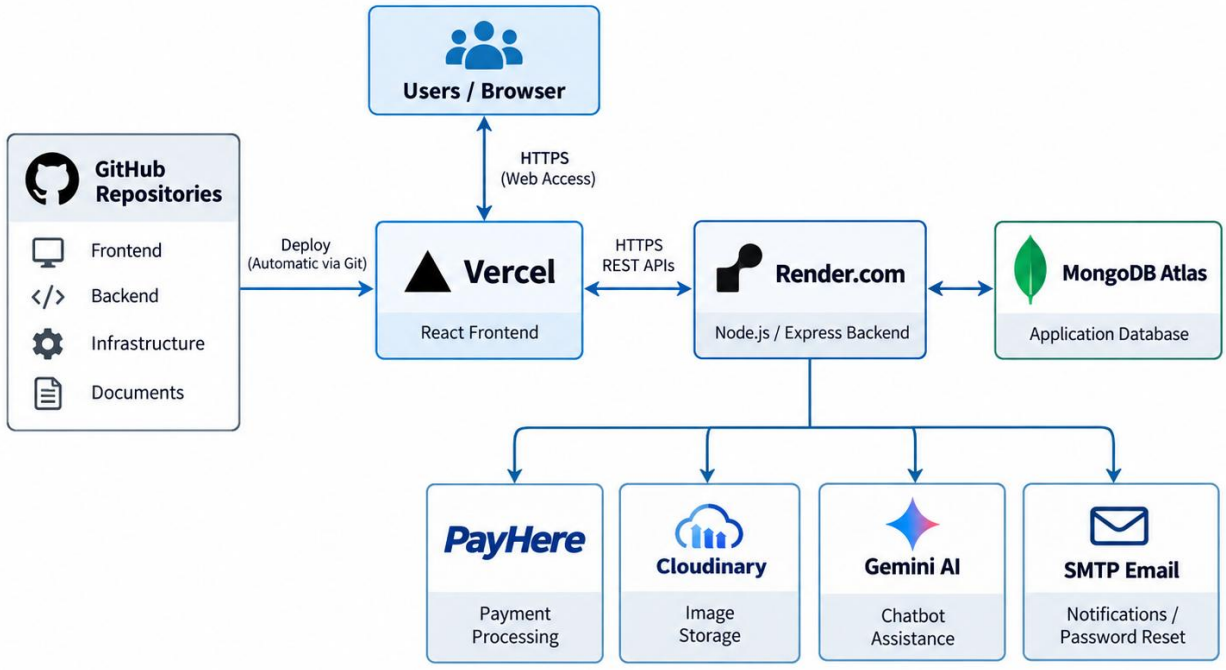


Figure 4.4: OSMS Deployment Workflow

4.8.5 Backend Deployment

The backend is hosted on Render.com using Docker-based application configuration.

Render provides the runtime environment for the Node.js/Express API.

The deployed backend communicates with MongoDB Atlas and the configured external services.

Environment variables provide configuration values required for application operation.

4.8.6 Deployment Architecture Evolution

The original SDS proposed an AWS-based hosting architecture.

During implementation, the backend hosting approach was changed to Render.com, with the decision documented in ADR-015.

This change allowed the project to proceed using an approved hosting alternative while retaining the existing backend application structure.

The frontend hosting approach uses Vercel, while MongoDB Atlas provides centralized database services.

4.8.7 Deployment Verification and Quality Gates

Application deployment and software quality are related but distinct activities.

Passing CI checks does not independently confirm that the deployed system is operational or that every integration works correctly.

Deployment verification includes checking service health, frontend availability, API connectivity, database connectivity, environment configuration, and relevant application workflows.

The evidence supporting these checks is maintained through the corresponding infrastructure, testing, and release tasks.

4.9 Implementation Challenges and Engineering Decisions

4.9.1 Maintaining a Consistent Authorization Model

Challenge: The original requirements included several overlapping staff responsibilities that could create inconsistencies between frontend navigation and backend permissions.

Decision: ADR-002 established an authoritative five-role model and defined clearer operational responsibilities.

Implementation: The backend applies role-based authorization and relevant branch or ownership restrictions. The frontend provides role-specific navigation and interfaces.

Outcome: The resulting architecture provides a consistent basis for assigning application access privileges.

4.9.2 Maintaining Checkout and Inventory Consistency

Challenge: Order creation involves multiple related records, including customer carts, inventory stock, and order details.

Decision: ADR-003 established the persistence boundary between cart contents and checkout operations.

Implementation: The backend validates product information and executes coordinated database updates using transactions.

Outcome: This approach reduces the risk of incomplete order creation and inconsistent inventory reservations.

4.9.3 Handling Payment Verification

Challenge: Payment processing involves an external service, and browser redirects alone cannot establish successful payment completion.

Decision: ADR-011 defined the secure payment initiation and verification lifecycle.

Implementation: The backend processes verified PayHere payment notifications, maintains payment records, and handles relevant failure and reconciliation conditions.

Outcome: The payment design separates customer-facing navigation from authoritative payment confirmation.

4.9.4 Supporting Appointment Capacity

Challenge: Appointment scheduling must prevent invalid bookings and coordinate patient capacity within available examination sessions.

Decision: ADR-007 established capacity-based Doctor Sessions and the approved appointment lifecycle.

Implementation: DoctorSession records maintain capacity and booking counts, while backend services validate booking eligibility and coordinate database updates.

Outcome: Appointment scheduling is governed by explicit availability and capacity constraints.

4.9.5 Separating Clinical and Operational Responsibilities

Challenge: The original architecture required clearer boundaries between appointment providers and authenticated staff members responsible for clinical records.

Decision: The approved implementation introduced a dedicated Doctor directory while retaining the OPTOMETRIST role for relevant clinical operations.

Implementation: Doctors are managed as separate records, while prescription operations remain subject to authenticated Optometrist permissions.

Outcome: The final architecture distinguishes appointment-provider information from application authorization roles.

4.9.6 Managing Inventory Images

Challenge: Inventory products require image-management functionality without unnecessarily storing image binaries in the application database.

Decision: ADR-008 established Cloudinary as the external image-storage provider.

Implementation: The backend manages authorized image operations and stores relevant Cloudinary references in MongoDB records.

Outcome: Product-image storage is separated from structured inventory information.

4.9.7 Controlling AI Chatbot Usage

Challenge: AI-assisted customer support requires appropriate knowledge boundaries, request limits, and external service handling.

Decision: ADR-010 defined the chatbot architecture and usage-management strategy.

Implementation: The chatbot combines authenticated request validation, MongoDB knowledge retrieval, usage tracking, and Gemini AI integration.

Outcome: The chatbot provides controlled informational assistance without unrestricted access to sensitive business or clinical information.

4.9.8 Adapting the Deployment Strategy

Challenge: The original AWS-based hosting design required revision due to practical deployment constraints.

Decision: ADR-015 documented the Render.com backend hosting alternative.

Implementation: The backend uses its existing Docker configuration for Render deployment, while the frontend is hosted using Vercel.

Outcome: The project adopted an implementation-aligned cloud hosting architecture based on Vercel, Render.com, and MongoDB Atlas.

4.9.9 Challenges and Engineering Decisions

Table 4.8: Implementation Challenges and Engineering Decisions

Challenge	Engineering Decision	ADR
Overlapping staff permissions	Five canonical roles	ADR-002
Cart/order consistency	Transactional checkout	ADR-003
Order-status control	Authoritative lifecycle	ADR-004
Appointment capacity	Doctor Sessions	ADR-007
Inventory image handling	Cloudinary storage	ADR-008
Payment verification	PayHere callback validation	ADR-011
Backend hosting constraints	Render.com fallback	ADR-015

4.10 Software Quality and Implementation Verification

The implementation process incorporated software quality activities throughout development.

Automated tests were used to verify backend services and frontend components.

Code quality was supported through ESLint and Prettier, while GitHub Actions executed automated checks during repository development.

Pull Request reviews provided a mechanism for identifying implementation issues before integration.

Dedicated testing activities were also conducted for system integration, security, performance, and reliability.

However, implementation completeness must be distinguished from full requirements compliance.

Some original requirements remain partially implemented or modified through approved architectural decisions.

Examples include product catalogue limitations, the absence of automatic inter-branch transfers, excluded appointment reminders, and unavailable loyalty redemption functionality.

Similarly, compliance with original non-functional targets, including availability, usability, scalability, and browser compatibility, requires appropriate verification evidence.

The detailed testing results, requirement verification statuses, and supporting evidence are presented in Chapter 5.

4.11 Chapter Summary

This chapter described the implementation of the Optical Shop Management System using the MERN technology stack.

The frontend was developed using React, Vite, and React Router, while the backend was implemented using Node.js, Express, and Mongoose.

MongoDB Atlas provides centralized data storage for customer, inventory, order, appointment, clinical, and supporting business information.

The implementation includes authentication, role-based authorization, product browsing, inventory management, shopping carts, checkout, payment processing, order tracking, Doctor Sessions, appointment booking, clinical prescriptions, loyalty, AI chatbot assistance, reporting, and audit logging.

Business-critical operations are controlled through backend validation, authorization, database schemas, indexes, and applicable transactions.

External services, including PayHere, Cloudinary, Gemini AI, and SMTP-based email services, support specialized application functions.

GitHub-based collaboration, automated testing, code-quality checks, and continuous integration were used throughout development.

The cloud-hosted deployment architecture uses Vercel, Render.com, and MongoDB Atlas, reflecting the implementation decisions documented in the approved ADRs.

The implementation process demonstrates how the initial system design was transformed into a functioning application while maintaining traceability between requirements, architectural decisions, and source-code modules.

The following chapter presents the software testing strategy and verification evidence used to evaluate the implemented system.

CHAPTER 5 : TESTING

5.1 Introduction

Software testing is an essential phase of the software development lifecycle that evaluates whether an implemented system behaves according to its specified requirements and supports the identification of defects before release.

The Optical Shop Management System (OSMS), developed for Darshana Opticals (Pvt) Ltd., consists of several interconnected modules involving customer authentication, inventory management, online purchasing, payment processing, appointments, clinical prescriptions, loyalty management, reporting, and administrative operations.

Because these modules process business-critical and potentially sensitive information, the project adopted a multi-level testing approach involving backend unit and integration testing, frontend component testing, system integration and end-to-end testing, security verification, and controlled performance and reliability assessment.

Testing activities were documented through DDP-084, DDP-085, DDP-086, and DDP-087. The final Test Case Register, Test Coverage Report, and Performance Test Results Report provide the main evidence used in this chapter.

The evaluation distinguishes successful local automated test execution from complete system acceptance, hosted integration verification, and production release readiness.

5.2 Testing Objectives

The main objective of testing was to assess the correctness, reliability, security, and performance of OSMS against the functional and non-functional requirements defined in the Software Requirements Specification (SRS).

The specific objectives were to:

1. Verify backend business logic and data validation.
2. Evaluate frontend component behavior and user interactions.
3. Validate interactions between related backend services and MongoDB.
4. Assess critical end-to-end business workflows.
5. Evaluate authentication, authorization, and security-sensitive operations.
6. Measure selected API response times and concurrent workload behavior.
7. Verify transactional consistency during competing checkout and appointment requests.
8. Document test execution results, code coverage, remaining gaps, and requirement traceability.

The testing strategy was designed to provide evidence across multiple levels rather than relying exclusively on a single test framework or test execution method.

5.3 Testing Strategy and Methodology

5.3.1 Testing Levels

The project adopted the following testing levels.

Table 5.1: Software Testing Strategy

Testing Level	Purpose	Tool or Evidence
Backend Unit Testing	Verify individual business services, validation rules, and supporting functions	Jest
Backend Integration Testing	Verify API behavior and service/database interactions	Jest, Supertest, MongoDB Memory Server
Frontend Component Testing	Verify React components, navigation, forms, and interface state	Vitest, React Testing Library
System Integration Testing	Assess cross-module functionality	DDP-084
End-to-End Testing	Evaluate selected real user workflows	DDP-084 Playwright evidence
Security Testing	Evaluate OWASP-related controls, authentication, and authorization	DDP-085
Performance Testing	Measure latency, throughput, and resource usage	DDP-086
Concurrency and Reliability Testing	Assess transaction integrity and fault handling	DDP-086
Regression Testing	Identify unintended changes to existing functionality	Jest, Vitest, DDP-092

Unit and component tests provide relatively fast feedback during development. Integration tests examine interactions between application components, while end-to-end tests focus on complete workflows.

Security, performance, and reliability testing provide additional evidence relating to selected non-functional requirements.

5.3.2 Requirement-Based Testing

The approved SRS identifies fourteen functional requirements, FR-001 to FR-014, and seven non-functional requirements, NFR-001 to NFR-007.

The DDP-087 Test Case Register links representative automated test scenarios to these identifiers where the tested behavior directly supports a requirement.

Supporting technical tests, such as database initialization and credential-safe error handling, are distinguished from direct business-requirement acceptance tests.

A passing automated assertion is considered evidence of the specific tested behavior and is not treated as proof that every acceptance criterion of the corresponding requirement has been satisfied.

5.3.3 Risk-Based Testing

Testing priorities considered the consequences of defects in critical business operations.

Important risk-sensitive areas included authentication, role-based authorization, stock reservation, payment processing, appointment capacity, clinical data access, and administrative actions.

These areas were prioritized because failures could affect business integrity, customer information, or the consistency of operational records.

5.4 Testing Environment

5.4.1 Backend Environment

The backend automated testing environment used Node.js, Jest, Supertest, and MongoDB-based test infrastructure.

The coverage report documents local Jest execution with MongoDB Memory Server.

The performance test environment used a single-node in-memory MongoDB replica set to support multi-document transactions while avoiding destructive operations on shared databases.

The performance assessment environment included:

Table 5.2: Performance Testing Environment

Component	Documented Configuration
Operating System	Windows 11, x64
Processor	8-core / 16-thread virtualized processor
RAM	16 GB
Node.js	v22.14.0
Express.js	v4.19.2
Mongoose	v8.8.4
Performance Database	MongoMemoryServer v11.3.0, replica set `rs0`

These specifications refer to the performance testing environment, not necessarily the hosted production infrastructure.

5.4.2 Frontend Environment

The frontend testing environment used Vitest, React Testing Library, and a simulated browser environment.

The final coverage report identifies Vitest v5.0.5 with V8 coverage instrumentation.

Frontend automated tests used fixtures and mocked API behavior for relevant component-level scenarios.

Therefore, passing component tests are not automatically equivalent to successful real-browser, full-stack execution.

5.4.3 Test Data and Isolation

The performance test harness used representative synthetic records.

The documented dataset included three branches, fourteen system users, one Doctor with five sessions, fifty-two inventory items, fifteen appointments, fifteen prescriptions, and twenty-five orders and payment records.

Safeguards were implemented to prevent destructive performance-test seeding against production or external databases.

External payment processing, Cloudinary uploads, and email dispatch were simulated or mocked during the isolated performance assessment.

This approach protected shared data and reduced external-service dependencies, but also limited what could be concluded about the real hosted integrations.

5.5 Backend Automated Testing

5.5.1 Backend Testing Scope

Backend tests covered service behavior, database operations, validation, authorization, and selected API interactions.

Representative passing suites included authentication-related security, staff and Doctor management, inventory image handling, loyalty reads, chatbot session APIs, password recovery, payments, orders, checkout, reporting, auditing, and authorization middleware.

The complete automated suite contains more scenarios than those individually reproduced in the final Test Case Register.

5.5.2 Backend Test Execution Results

The DDP-087 coverage evidence records a successful local backend Jest execution on 26 October 2026.

Table 5.3: Backend Automated Testing Results

Metric	Result
Total test suites	118
Passed test suites	118
Total tests	2,075
Passed tests	2,075

Failed tests	0
Fresh recorded execution duration	346.634 seconds

The documented execution completed with all reported backend tests passing.

The associated pre-run source revision was:

`47eb042a7627d4ae1f1f67b9c48c370379faa7c5`

This revision is supported by the coverage report's execution-provenance addendum.

The successful local result does not independently establish that the exact same revision was deployed as the final release.

5.6 Frontend Automated Testing

5.6.1 Frontend Testing Scope

Frontend testing evaluated React components, navigation behavior, input validation, protected routes, and relevant interface states.

Representative tested areas included registration, login, password recovery, inventory quantity validation, payment initiation, appointment booking, prescription entry, staff administration, order-status controls, chatbot interactions, and reporting.

Many tests used mocked backend responses to isolate user-interface behavior.

5.6.2 Frontend Test Execution Results

The DDP-087 evidence records a successful local Vitest execution on 26 October 2026.

Table 5.4: Frontend Automated Testing Results

Metric	Result
Total test files	110

Passed test files	110
Total tests	1,341
Passed tests	1,341
Failed tests	0
Fresh recorded execution duration	77.81 seconds

The documented frontend pre-run revision was:

`3e6fa9b5b277d9663982cb6e36a5af8c8187c97c`

All reported frontend tests passed.

The testing output also contained non-failing React Router and React `act(...)` warnings. These were documented as testing and maintenance observations rather than failed assertions.

5.7 Consolidated Automated Testing Results

The combined local automated testing evidence is summarized in Table 5.4.

Table 5.5: Consolidated Automated Test Results

Metric	Backend	Frontend	Total
Tests Executed	2,075	1,341	3,416
Tests Passed	2,075	1,341	3,416
Tests Failed	0	0	0
Suites / Test Files	118	110	228

The reported local automated test pass rate was 100% for the documented executions.

This result indicates that all assertions included in those executions passed. It does not mean that the entire application has been verified with 100% functional, integration, or acceptance coverage.

5.8 Test Case Register and Requirement Traceability

5.8.1 Final Test Case Register

The Final Test Case Register was prepared under DDP-087.

Document: `testing/test-register.pdf`

The register contains forty representative, evidence-backed automated test scenarios.

The cases use two identifier groups:

- `QA-BE-###` for backend cases.
- `QA-FE-###` for frontend cases.

Each registered scenario records its test identifier, requirement mapping where applicable, test type, priority, source file, preconditions, steps or inputs, expected result, actual result, and execution status.

The register deliberately does not claim to enumerate all 3,416 automated test assertions.

5.8.2 Selected Representative Test Cases

Table 5.6: Representative Test Cases from DDP-087

Test Case ID	Test Scenario	Requirement	Recorded Result
QA-BE-004	Staff and Doctor lifecycle security	FR-001 / ADR-002	Pass — Local
QA-BE-009	Password reset service	FR-001	Pass — Local
QA-BE-010	Payment validation	FR-009	Pass — Local
QA-BE-012	Checkout service	FR-004	Pass — Local
QA-BE-013	Order service	FR-014	Pass — Local

QA-BE-015	Authorization middleware	FR-001 / NFR-004	Pass — Local
QA-FE-016	Login routes according to backend role	FR-001	Pass — Local
QA-FE-021	Inventory quantity validation	FR-005	Pass — Local
QA-FE-024	Payment success requires backend confirmation	FR-009	Pass — Local
QA-FE-028	Handling stale appointment availability	FR-007	Pass — Local
QA-FE-030	Prescription eye-axis validation	FR-013	Pass — Local
QA-FE-035	Safe rendering of hostile chatbot output	FR-011 / NFR-004	Pass — Local
QA-FE-036	Staff order-status progression	FR-014	Pass — Local
QA-FE-037	Reporting masks sensitive fields	FR-012 / NFR-004	Pass — Local

These cases provide concrete examples of verified local automated behavior.

They do not independently establish complete acceptance of their associated functional requirements.

5.8.3 Requirement Traceability Limitations

The register maps its representative assertions to the original FR and NFR identifiers.

However, the selected forty scenarios do not directly demonstrate every functional requirement.

For example, the representative register does not contain a direct mapped case for FR-002, FR-003, FR-006, or FR-008.

This does not prove that those requirements lack tests elsewhere in the larger test suites. It means they are not directly demonstrated by those particular register entries.

Complete requirement acceptance requires reviewing the wider test inventory, relevant E2E evidence, and individual acceptance criteria.

5.9 Code Coverage Analysis

5.9.1 Coverage Metrics

The DDP-087 Test Coverage Report provides statement, branch, function, and line coverage measurements for both repositories.

Table 5.7: Automated Code Coverage Results

Coverage Metric	Backend	Frontend
Statements	92.34%	88.05%
Branches	81.68%	81.80%
Functions	97.86%	90.95%
Lines	93.00%	88.86%

Both applications exceeded 80% across the four aggregate coverage metrics.

However, the guideline's new-code coverage requirement was not separately measured using only the new or changed code.

Therefore, aggregate coverage above 80% cannot independently establish the specific new-code target.

5.9.2 Critical Business Logic Branch Coverage

The project guideline also identifies a 90% branch coverage target for critical business logic.

The overall backend branch coverage was 81.68%, while the frontend achieved 81.80%.

These aggregate figures do not prove that every designated critical module satisfies the required threshold.

The coverage report records the following selected frontend file-level branch measurements.

Table 5.8: Selected Frontend Branch Coverage Gaps

Module	Branch Coverage
Payment Entry Page	68.70%
Checkout Entry Page	68.05%
Appointment Booking Page	71.55%
Admin Staff Editor	66.37%
Cart Provider	68.08%
Staff Move Appointment Page	57.64%
Inventory Editor	92.40%
Order Service	100.00%
Payment Service	86.20%

Several risk-sensitive modules are below 90% branch coverage.

The backend critical-module file-level measurements were not available in the reviewed coverage summary.

Consequently, compliance with the 90% critical-business-logic branch coverage target remains unverified.

5.9.3 Interpretation of Code Coverage

High code coverage indicates that a substantial proportion of instrumented source code was executed by automated tests.

However, coverage measurements alone do not establish the correctness of every behavior, branch, API contract, or business requirement.

The final QA assessment therefore distinguishes measured coverage from verified functional completeness.

5.10 API Integration Coverage

5.10.1 API Route Inventory

The final Test Case Register includes an inventory of seventy-three method-and-path route declarations across seventeen backend route groups.

The route inventory was derived from the backend application and route source files.

The existence of a route declaration or a similarly named test suite does not establish that a corresponding HTTP integration assertion was successfully executed.

5.10.2 Confirmed Integration Test Source References

The supplementary source audit identified direct Supertest HTTP request references for eight of the seventy-three route declarations.

Examples include authentication login, staff-directory listing, inventory image upload, Doctor Session operations, appointment listing, and loyalty retrieval.

The remaining sixty-five route declarations were not independently confirmed by that targeted audit.

This does not mean that sixty-five endpoints were untested or failed. Their endpoint-specific integration evidence remains unverified by the sampled audit.

The complete requirement of at least one verified integration test per API endpoint is therefore not established by the available register.

5.11 System Integration and End-to-End Testing

5.11.1 Integration Testing Scope

System integration and E2E testing were coordinated under DDP-084.

The planned critical workflows included customer authentication, purchasing and payment, inventory consistency, Doctor Session booking, prescription recording and retrieval, and role-based access restrictions.

The objective was to evaluate the integration between the frontend, backend, database, and applicable external service boundaries.

5.11.2 Recorded E2E Evidence

The DDP-087 Test Case Register references infrastructure Issue #11 and Pull Request #19 as sources of E2E evidence.

The supporting QA documentation refers to ten previously reported Playwright scenarios, including seven repeated authorization checks reported as passing.

However, the final scenario-level browser logs, execution dates, environments, and release revisions require independent reconciliation.

These historical references are not treated as proof that every critical hosted workflow passed.

5.11.3 Hosted PayHere Verification Limitation

The QA documents identify an important unresolved integration limitation.

Hosted PayHere payment confirmation was constrained by a merchant-configuration error.

Accordingly, the complete hosted purchase and payment-confirmation workflow remains classified as **Blocked / Not Verified** in the DDP-087 evidence.

Passing mocked payment tests or local payment-service tests cannot replace a successfully executed hosted payment workflow.

This limitation must remain visible in the final project evaluation and release-readiness assessment.

5.12 Security Testing

5.12.1 Security Assessment

Security verification was carried out under DDP-085.

The testing scope addressed OWASP-related risks and important OSMS security boundaries, including authentication, authorization, customer and branch isolation, input validation, clinical information protection, and payment integrity.

The detailed security assessment is maintained separately in the project's OWASP checklist.

5.12.2 Supporting Adversarial Test Results

The DDP-086 performance report also includes selected adversarial tests related to fault handling and error sanitization.

Table 5.9: Selected Adversarial Test Results

Test Scenario	Observed HTTP Result	Reported Outcome
NoSQL injection payload	422	Pass
Expired JWT	401	Pass
Tampered JWT signature	401	Pass
Oversized request body	413	Pass
Invalid API route	404	Pass

The report records no stack-trace or credential leakage in these evaluated scenarios.

These results support the effectiveness of selected input-validation and error-handling controls within the tested environment.

They do not independently establish complete OWASP compliance or full application security certification.

5.12.3 Residual Security Risks

The final SDS and security documentation identify residual considerations, including JWT handling in browser localStorage and remaining dependency findings.

The DDP-085 checklist should be used as the authoritative source for the detailed security findings and their individual resolution statuses.

5.13 Performance Testing

5.13.1 Performance Evaluation Method

Performance and reliability testing were conducted under DDP-086.

The assessment included baseline latency measurements for thirteen critical endpoints, controlled concurrent workloads, transaction race-condition tests, and MongoDB query profiling.

All measurements in the Performance Test Results Report were obtained from an isolated local environment.

5.13.2 Baseline API Performance

The baseline assessment recorded response-time distributions including p50, p90, p95, and p99.

All thirteen evaluated endpoints achieved P95 response times below 130 milliseconds.

Table 5.10: Selected Baseline API Results

Endpoint / Operation	P95 Response Time
Health Endpoint	52.3 ms
Customer Login	128.5 ms
Customer Profile	41.3 ms
Default Product Listing	69.4 ms
Product Text Search	58.7 ms

Customer Order History	33.4 ms
Available Doctor Sessions	49.3 ms
Customer Prescription History	57.2 ms
Sales Summary Reporting	82.1 ms

These results demonstrate low observed backend response times for the selected operations in the local test environment.

They do not measure complete browser rendering or real cloud network latency.

5.13.3 Controlled Load Testing

The performance assessment used three controlled workload stages.

Table 5.11: Controlled Load Test Results

Stage	Virtual Users	Requests	Throughput	P95 Latency	Result
Warmup / Light Load	5	25	86.6 req/s	102.7 ms	Pass
Target Peak	15	60	146.8 req/s	130.9 ms	Pass
High Stress	30	120	152.8 req/s	241.8 ms	Pass

The report recorded a 0.00% HTTP 5xx error rate across the evaluated workloads.

The application remained responsive under the bounded load scenarios.

However, the results should not be interpreted as proof of unlimited scalability or long-term cloud availability.

5.13.4 Memory Usage

The performance report records heap usage increasing from 57.5 MB during the light workload to 75.3 MB during the high-stress workload.

Resident Set Size increased from 140.9 MB to 150.9 MB.

The recorded workload did not produce observed server crashes or identified memory leaks.

These observations apply to the bounded test execution and do not substitute for long-duration monitoring.

5.14 Database Optimization and Concurrency Testing

5.14.1 Database Query Optimization

MongoDB execution-plan profiling was performed using `explain('executionStats')`.

The assessment identified inefficient collection scans and sorting operations in selected order, prescription, appointment, and inventory queries.

Targeted compound indexes were applied to improve query efficiency.

Table 5.12: Database Query Optimization Results

Query Area	Documents Examined Before	Documents Examined After	Reduction
Customer Order History	25	2	92.0%
Prescription History	15	2	86.7%
Appointment History	15	2	86.7%
Inventory Category Filtering	52	3	94.2%

The measured reductions demonstrate improved query efficiency for the selected synthetic dataset.

The effects may differ with larger production datasets and real traffic distributions.

5.14.2 Concurrent Checkout Verification

The performance suite evaluated simultaneous checkout requests competing for limited inventory.

Table 5.13: Checkout Concurrency Results

Scenario	Concurrent Requests	Available Stock	Successful Orders	Rejected Requests	Outcome
Single-item contention	5	1	1	4	Pass
Two-item contention	4	2	2	2	Pass

Both scenarios finished with zero recorded inventory drift.

These results support the effectiveness of the implemented transactional stock-reservation safeguards within the tested conditions.

5.14.3 Concurrent Appointment Booking Verification

The suite also evaluated simultaneous requests competing for Doctor Session capacity.

Table 5.14: Appointment Concurrency Results

Scenario	Concurrent Requests	Available Capacity	Confirmed Bookings	Rejected Requests	Outcome
Single-slot contention	5	1	1	4	Pass

Two-slot contention	4	2	2	2	Pass

The assigned queue positions were consistent with the available capacity.

No overbooking was recorded in these contention scenarios.

5.15 Non-Functional Requirements Evaluation

The original SRS specifies measurable quality requirements that must be assessed independently of functional implementation.

Table 5.15: Non-Functional Requirement Verification

Requirement	Quality Attribute	Evaluation
NFR-001	Performance	Partially Verified
NFR-002	Reliability / Availability	Not Fully Verified
NFR-003	Usability	Not Verified
NFR-004	Security	Partially Verified
NFR-005	Scalability	Not Fully Verified
NFR-006	Maintainability	Not Fully Verified
NFR-007	Browser Portability	Not Verified

The local performance assessment demonstrated successful results against its selected backend latency and bounded reliability thresholds.

However, these results do not establish complete satisfaction of the original browser-level response-time requirement.

Likewise, zero server errors during controlled tests do not establish 99.9% long-term operational availability.

The original scalability, maintainability, usability, and browser compatibility criteria require additional criterion-specific evidence.

Therefore, the successful local performance tests are treated as supporting verification evidence rather than unrestricted satisfaction of every original NFR.

5.16 Defect Management and Regression Testing

OSMS uses GitHub Issues, implementation branches, Pull Requests, and automated testing to manage identified defects and reduce regressions.

DDP-092 coordinates final defect resolution and cross-module regression activities.

The recorded Jest and Vitest executions passed all included tests, indicating no failing assertions within those specific local runs.

However, this does not prove that the final deployed application has zero unresolved defects.

The final defect register, associated fixes, regression evidence, and release decision must be assessed separately.

5.17 Testing Limitations and Outstanding Verification

The final QA evidence identifies several limitations that remain relevant to project evaluation.

First, the Test Case Register documents forty representative automated test scenarios rather than independently enumerating all 3,416 tests.

Second, the API route inventory contains seventy-three declarations, but the available source audit confirms direct HTTP integration-test references for only eight routes. Complete endpoint-level integration verification remains unestablished.

Third, the 90% branch coverage criterion for every designated critical business module has not been demonstrated. Several risk-sensitive frontend components remain below that threshold.

Fourth, the complete hosted PayHere purchase and payment-confirmation workflow remains blocked or unverified in the available documentation.

Fifth, the local automated execution evidence has pre-run source revision information, but exact CI artifact identities and final deployed release parity have not been established.

Finally, the performance results were obtained using local, isolated workloads with simulated external service boundaries. Sustained cloud reliability, complete browser performance, and final formal user acceptance require separate evidence.

These limitations are retained to ensure that the report accurately distinguishes demonstrated test outcomes from outstanding acceptance requirements.

5.18 Chapter Summary

This chapter presented the testing strategy, test execution evidence, coverage analysis, security verification, and performance assessment of the Optical Shop Management System.

The project used Jest and Supertest for backend testing, Vitest and React Testing Library for frontend testing, and additional targeted testing activities for integration, security, performance, and reliability.

The documented local automated executions completed successfully with 2,075 backend tests and 1,341 frontend tests passing, producing a combined total of 3,416 passing tests.

Backend code coverage reached 92.34% statements, 81.68% branches, 97.86% functions, and 93.00% lines.

Frontend code coverage reached 88.05% statements, 81.80% branches, 90.95% functions, and 88.86% lines.

The final Test Case Register provides forty representative test scenarios and maintains traceability to relevant approved requirements.

Performance testing demonstrated successful local baseline and controlled load results, while concurrency tests recorded consistent inventory reservations and appointment-capacity enforcement.

Database profiling identified measurable improvements following targeted indexing.

Nevertheless, the evidence does not establish complete API endpoint integration coverage, full critical-module branch coverage, successful hosted PayHere payment verification, or unrestricted production release readiness.

The recorded findings provide an evidence-based foundation for evaluating project achievements, remaining limitations, and future improvements in Chapter 6.

CHAPTER 6 : EVALUATION

6.1 Introduction

The evaluation phase assesses the extent to which the **Optical Shop Management System (OSMS)** developed for Darshana Opticals (Pvt) Ltd. satisfies its original objectives, functional and non-functional requirements, and intended business needs.

The evaluation is based on the Software Requirements Specification (SRS v1.1), Software Design Specification (SDS v1.1), approved Architectural Decision Records (ADR-001 to ADR-015), implemented functionality, and available software verification evidence.

The project aims to digitalize the principal retail, clinical, customer-service, inventory, and administrative processes of Darshana Opticals through an integrated web-based system.

This chapter evaluates the achievement of these objectives, distinguishes implemented functionality from verified quality requirements, identifies remaining limitations, considers available stakeholder feedback, and proposes realistic areas for future improvement.

The evaluation maintains a distinction between functionality that has been implemented, requirements that have been modified through approved decisions, and acceptance criteria that remain partially verified or unverified.

6.2 Evaluation of Project Objectives

The overall project objective was to develop an integrated Optical Shop Management System that improves the management of retail operations, inventory information, appointments, clinical prescription records, and customer-facing services.

The achievement of individual objectives is assessed using the functional requirements defined in the approved SRS.

6.2.1 Digitalization of Retail and E-Commerce Operations

The project aimed to establish a digital storefront that enables customers to browse products and complete supported online purchasing activities.

The implemented system includes a product catalogue, search and filtering functionality, persistent shopping carts, checkout processing, payment integration, and order tracking.

The backend manages authoritative pricing, inventory validation, order creation, and stock reservations.

These capabilities demonstrate substantial achievement of the e-commerce objective.

However, the product catalogue does not include every filtering option originally specified in the requirements. In particular, a separate style-based filtering capability has not been verified.

Therefore, the objective is assessed as **Partially Achieved** against the full original requirement scope.

Related SRS Requirements: FR-003, FR-004, FR-009, FR-014.

6.2.2 Improvement of Inventory Management

The project aimed to provide centralized inventory management across branches and improve the visibility of available stock.

The implemented system maintains branch-specific inventory records, supports authorized stock updates, and provides low-stock alerts based on an approved organization-wide reorder threshold.

These capabilities support more structured inventory management compared with disconnected manual processes.

Nevertheless, the original real-time stock synchronization requirement has not been fully verified, and automated inter-branch stock transfers are not available.

The inventory-management objective is therefore assessed as **Partially Achieved**.

Related SRS Requirements: FR-005, FR-006.

6.2.3 Online Appointment Scheduling

The project aimed to provide customers with an online mechanism for booking eye examinations.

The implemented system supports appointment booking through capacity-based Doctor Sessions at the approved Gampaha Main Branch.

Customers can select an available Doctor and session, while the backend enforces relevant scheduling constraints, including the minimum 24-hour booking lead time and session-capacity limits.

This approach replaces the original individual optometrist-slot design with a model better aligned with the approved operational requirements.

The original automated appointment-reminder requirement was formally modified through ADR-009, which established manual appointment communication for the current scope.

The appointment-booking objective is assessed as **Achieved Within the Approved Revised Scope**.

Related SRS Requirements: FR-007, FR-008.

6.2.4 Customer Information and Clinical Prescription Management

The project aimed to maintain customer accounts and provide controlled access to relevant customer information.

The implemented system supports customer registration, authentication, profile management, and access to authorized customer records.

Clinical prescription information is maintained separately and is associated with the relevant customer and recording Optometrist.

The application provides restricted prescription recording and customer prescription-history access.

These capabilities satisfy the principal implementation requirements associated with customer profile management and clinical prescription records.

The objective is assessed as **Achieved at the Functional Implementation Level**, while complete security and privacy compliance remains subject to separate verification.

Related SRS Requirements: FR-001, FR-002, FR-013.

6.2.5 Customer Assistance and Engagement

The project aimed to improve customer engagement through additional digital services.

The implemented system includes an AI-driven informational chatbot and a loyalty points mechanism.

The chatbot retrieves information from approved knowledge sources and applies authenticated usage restrictions.

The loyalty system calculates and records points for eligible purchases and maintains relevant transaction history.

However, loyalty point redemption and tier-based reward functionality are not available in the current implementation.

Therefore, the broader customer-engagement objective is assessed as **Partially Achieved**, although the approved loyalty accrual and informational chatbot requirements are implemented.

Related SRS Requirements: FR-010, FR-011.

6.2.6 Business Administration and Reporting

The project aimed to provide authorized staff and management with tools for managing business operations and accessing relevant reports.

The implemented system includes role-based staff and Doctor management, branch-specific inventory functions, and sales and inventory reporting.

Reporting access is controlled according to the approved application roles and organizational boundaries.

The system also maintains audit records for relevant business and administrative operations.

The principal reporting requirement is implemented. However, historical inventory turnover analysis and dedicated appointment-frequency analytics are outside the current reporting implementation.

The objective is assessed as **Achieved Within the Approved Reporting Scope**.

Related SRS Requirements: FR-006, FR-012 and the approved administrative requirements.

6.2.7 Software Quality, Security, and Maintainability

The project aimed to develop OSMS using established software engineering practices and appropriate mechanisms for ensuring functional correctness, security, reliability, and maintainability.

The final automated testing evidence recorded under DDP-087 demonstrates successful local execution of 2,075 backend Jest tests across 118 test suites and 1,341 frontend Vitest tests across 110 test files. All 3,416 reported automated tests passed, with no failed assertions during the documented executions.

The backend achieved 92.34% statement coverage, 81.68% branch coverage, 97.86% function coverage, and 93.00% line coverage. The frontend achieved 88.05% statement coverage, 81.80% branch coverage, 90.95% function coverage, and 88.86% line coverage.

Although these results demonstrate substantial automated testing activity, full verification of the required 90% branch coverage for critical business logic remains outstanding. The final QA documentation also identifies incomplete API endpoint-level integration verification, limitations in hosted payment verification, and the need to establish equivalence between tested and deployed release revisions.

Controlled performance testing under DDP-086 demonstrated satisfactory local API response times, zero reported HTTP 5xx errors under evaluated workloads, and consistent inventory and appointment transactions during selected concurrency tests.

Nevertheless, sustained availability, complete browser-level performance, formal usability criteria, scalability targets, and other original non-functional requirements have not all been fully verified.

The software quality objective is therefore assessed as **Partially Verified**, with significant positive automated testing results and clearly documented remaining acceptance boundaries.

6.3 Functional Requirements Achievement Assessment

The functional requirements register in SRS v1.1 identifies the implementation status of FR-001 through FR-014.

Table 6.1 summarizes the reported status of each requirement.

Requirement	Functional Area	Final SRS Status	Evaluation
FR-001	Registration and Authentication	Implemented	Principal functionality implemented
FR-002	Profile and Prescription History	Implemented	Principal functionality implemented
FR-003	Product Catalogue	Partially Implemented	Style filtering remains unverified
FR-004	Shopping Cart and Checkout	Implemented	Persistent cart and checkout implemented
FR-005	Inventory Tracking	Partially Implemented	Full real-time synchronization not verified

FR-006	Reorder Alerts	Implemented	Approved low-stock alert functionality implemented
FR-007	Appointment Booking	Modified by Approved Decision	Capacity-based Doctor Sessions implemented
FR-008	Appointment Reminders	Modified by Approved Decision	Manual communication approved under ADR-009
FR-009	Payment Gateway	Implemented	PayHere integration implemented
FR-010	Loyalty Points	Implemented	Automatic accrual implemented; redemption unavailable
FR-011	AI Chatbot	Implemented	Informational chatbot implemented
FR-012	Sales and Inventory Reporting	Implemented	Approved reports and exports implemented
FR-013	Prescription Recording	Implemented	Authorized clinical recording implemented
FR-014	Order Tracking	Modified by Approved Decision	Approved order lifecycle implemented

Functional Requirement Verification Evidence

The DDP-087 Final Test Case Register documents forty representative automated test scenarios linked to the applicable approved requirement identifiers. These scenarios provide evidence of selected authentication, authorization, inventory, checkout, payment, appointment, prescription, chatbot, reporting, and administrative behaviors.

However, the register is a representative inventory rather than an exhaustive enumeration of all 3,416 automated tests. Passing individual assertions does not establish complete acceptance of the corresponding functional requirement.

The companion API route inventory identifies seventy-three implemented method-and-path route declarations. A supplementary source audit identified direct HTTP integration-test references for eight routes. The remaining routes were not independently confirmed by that targeted audit, which does not necessarily mean they are untested.

Additionally, the hosted PayHere payment-confirmation workflow remains blocked or unverified in the available QA documentation.

Accordingly, the functional requirement implementation classifications in Table 6.1 should be interpreted separately from complete requirement acceptance and system-level verification.

According to the final SRS, nine requirements are classified as Implemented, two as Partially Implemented, and three as Modified by Approved Decision.

These categories describe implementation and scope status, not independent proof that all corresponding acceptance tests have passed.

The results demonstrate that the system addresses a substantial proportion of its functional requirements while maintaining traceability for approved deviations and remaining gaps.

6.4 Non-Functional Requirements Evaluation

The original SRS identifies seven measurable non-functional requirements relating to performance, reliability, usability, security, scalability, maintainability, and portability.

The final SRS preserves these original acceptance criteria and records the extent to which they have been verified.

Table 6.2: Non-Functional Requirements Verification Assessment

Requirement	Original Acceptance Criterion	Verification Status
NFR-001 Performance	95% of database search queries processed and displayed within 2 seconds	Partially Verified
NFR-002 Reliability	99.9% uptime excluding scheduled maintenance	Not Verified
NFR-003 Usability	New staff complete a sales transaction within 10 minutes of initial training	Not Verified

NFR-004 Security	SSL/TLS for sensitive data transmission and bcrypt password hashing	Partially Verified
NFR-005 Scalability	Support 50% more concurrent users and transactions without response-time degradation	Not Fully Verified
NFR-006 Maintainability	90% of minor bug fixes implemented and deployed within 24 hours	Not Fully Verified
NFR-007 Portability	Compatible with the latest Chrome, Firefox, and Safari versions	Not Verified

6.4.1 Performance

Performance testing under DDP-086 evaluated thirteen critical backend endpoints in an isolated local test environment.

All thirteen benchmarks achieved P95 response times below 130 milliseconds. During controlled load testing, the application recorded P95 response times of 102.7 milliseconds at five virtual users, 130.9 milliseconds at fifteen virtual users, and 241.8 milliseconds at thirty virtual users.

The highest recorded workload achieved approximately 152.8 requests per second without reported HTTP 5xx failures.

Database query profiling also identified improvements following targeted indexing. For example, document examination was reduced from twenty-five to two documents for customer order history retrieval and from fifty-two to three for inventory category filtering.

These results provide positive evidence of backend responsiveness and database query efficiency under the evaluated conditions.

However, the tests were conducted using a local environment with synthetic data and simulated external service boundaries. They do not independently demonstrate that 95% of complete database-search-to-browser-display operations meet the original two-second threshold in the hosted environment.

NFR-001 therefore remains **Partially Verified** against its full original acceptance criterion.

6.4.2 Reliability

The DDP-086 performance assessment recorded a 0.00% HTTP 5xx error rate across the evaluated baseline, load, and concurrency scenarios.

No server crashes were reported during the bounded testing activities.

The concurrency tests also demonstrated consistent transactional behavior. Simultaneous checkout requests did not cause stock overselling, and competing appointment requests did not exceed the available Doctor Session capacity.

Although these findings provide evidence of application stability and data integrity under the tested conditions, they do not establish the original 99.9% long-term availability requirement.

NFR-002 therefore remains **Not Verified** against the complete uptime criterion.

6.4.3 Usability

The application provides role-specific interfaces and supports common customer and staff workflows.

However, the original usability requirement specifies that newly trained staff should complete a sales transaction within ten minutes.

The available documentation does not demonstrate that this criterion has been measured through the required recorded usability assessment.

NFR-003 therefore remains Not Verified.

6.4.4 Security

OSMS implements several important security mechanisms, including password hashing, JWT-based authentication, backend authorization, and input validation.

The project's security assessment under DDP-085 evaluated relevant OWASP-related risks and documented residual findings.

The DDP-086 report also provides supporting evidence from selected adversarial tests involving malformed NoSQL inputs, expired and tampered JWTs, oversized request bodies, and invalid API routes.

These scenarios returned the expected HTTP error responses without reported stack-trace or credential exposure.

However, successful targeted security tests do not independently establish unrestricted OWASP compliance, complete privacy compliance, or verification of every hosted and external integration boundary.

The final QA documentation also identifies unresolved acceptance boundaries involving hosted payment processing and deployment-specific evidence.

NFR-004 therefore remains **Partially Verified**.

6.4.5 Scalability

Controlled workload testing under DDP-086 evaluated five, fifteen, and thirty concurrent virtual users.

The recorded throughput increased from approximately 86.6 requests per second at five virtual users to 152.8 requests per second at thirty virtual users.

The P95 response time increased from 102.7 milliseconds to 241.8 milliseconds across these workloads.

These results demonstrate the application's behavior under bounded local concurrency.

However, the original NFR-005 criterion requires evidence of supporting a 50% increase in concurrent users and transactions without response-time degradation.

The recorded workloads do not establish this exact requirement, particularly because response time increased as workload increased.

NFR-005 therefore remains **Not Fully Verified**.

6.4.6 Maintainability

The project uses modular source-code organization, automated testing, GitHub-based collaboration, and continuous integration.

These practices support maintainability and the ability to correct defects.

However, the original measurable target requires evidence that 90% of minor bug fixes were implemented and deployed within 24 hours.

The available records have not been consolidated to demonstrate this criterion.

NFR-006 therefore remains Not Fully Verified.

6.4.7 Portability

The system is developed as a browser-based application using web technologies.

Its architecture is intended to support modern browsers and common operating systems.

However, documented compatibility testing across the latest versions of Chrome, Firefox, and Safari has not been established.

NFR-007 therefore remains Not Verified.

6.5 Evaluation of Architectural and Engineering Decisions

During development, several architectural decisions were introduced to clarify the original system design and support implementation.

These decisions were documented through ADR-001 to ADR-015.

6.5.1 Role-Based Access Control

ADR-002 consolidated overlapping operational responsibilities into five authoritative authenticated application roles.

This provided a clearer basis for implementing frontend navigation and backend authorization.

6.5.2 Shopping Cart and Checkout

ADR-003 clarified the persistence boundary between temporary cart information and order creation.

The implemented checkout service validates current stock, prices, and item classification before creating orders through controlled database operations.

6.5.3 Appointment Management

ADR-007 refined the original appointment architecture into a capacity-based Doctor Session model.

This supported the separation of Doctor records, session management, and authenticated Optometrist responsibilities.

6.5.4 Payment Processing

ADR-011 established the payment verification and reconciliation design.

The implementation distinguishes order creation from confirmed payment processing and applies server-side checks to payment notifications.

6.5.5 Cloud Hosting

The initial AWS-based deployment design was replaced by a Vercel frontend and Render.com backend hosting approach, documented through the approved architecture decisions, including ADR-015.

This change maintained the application's layered structure while adapting the hosting plan to the project's deployment requirements.

These decisions demonstrate that the original SDS was refined in a controlled and traceable manner rather than changed without documentation.

6.6 Client Feedback and User Acceptance

Client feedback is important for evaluating whether the implemented system addresses actual business needs and operational expectations.

The OSMS project identifies Darshana Opticals management and relevant operational users as important stakeholders.

User Acceptance Testing activities are documented under DDP-091, while system integration and end-to-end testing evidence is maintained under DDP-084.

However, automated test execution and development-team demonstrations are not equivalent to formal client acceptance.

At the time of this draft, independently verified final client feedback and formal client sign-off information have not been provided for inclusion in this report.

Accordingly, no client satisfaction scores, quotations, or acceptance statements are asserted.

Client Feedback Status: Pending Verified Evidence

If formal client feedback becomes available, this section should be updated to include the date of the evaluation, stakeholder role, evaluated workflows, key findings, requested improvements, and any verified acceptance decision.

6.7 Limitations of the Implemented System

Although OSMS implements its principal functional modules, several limitations remain in the current version.

6.7.1 Product Catalogue Limitations

The product catalogue supports search and filtering by relevant attributes.

However, a separate style-based filtering function originally identified in the SRS has not been verified.

Additional product-discovery features are therefore potential areas for improvement.

6.7.2 Inventory Management Limitations

The system maintains centralized branch-specific inventory records.

However, automatic inter-branch stock transfers and historical inventory movement reporting are not implemented.

The original broader real-time synchronization requirement has not been fully verified.

6.7.3 Appointment Communication Limitations

The implemented appointment system uses capacity-based Doctor Sessions and supports confirmed bookings.

Automated appointment email and SMS reminders are excluded from the current approved scope.

Appointment-related communication is handled manually by authorized branch staff.

6.7.4 Loyalty Functionality Limitations

The current system supports eligible loyalty point accrual and transaction history.

However, customers cannot redeem accumulated loyalty points through the implemented application.

Tier-based rewards and exclusive loyalty-tier discounts are also unavailable.

6.7.5 Reporting Limitations

The system provides current sales and inventory reporting.

However, historical inventory turnover analysis and dedicated appointment-frequency analytics are not implemented in the current version.

6.7.6 External Integration Limitations

The system depends on external services for payment processing, image storage, AI assistance, and supported email notifications.

Interruptions or configuration failures involving these providers may affect the corresponding functionality.

The system does not include direct medical diagnostic hardware integration or automated third-party courier tracking.

6.7.7 Deployment and Operational Limitations

The implementation uses managed cloud services for frontend, backend, and database hosting.

However, the original 99.9% availability target has not been demonstrated through long-term monitoring evidence.

Comprehensive database backup and restoration verification, long-term audit-retention guarantees, and automatic horizontal scaling have not been independently established.

6.7.8 Security and Compliance Limitations

Although security controls and targeted assessments have been implemented, full regulatory compliance has not been independently established.

Residual security considerations include browser localStorage-based JWT handling and documented Moderate-level dependency risks.

The complete security assessment and remaining risks are discussed through the supporting DDP-085 evidence.

6.7.9 Testing and Release Verification Limitations

Although all 3,416 automated tests passed during the documented local executions, several important verification limitations remain.

The Final Test Case Register includes forty representative test scenarios and does not enumerate every individual automated assertion.

The backend API inventory contains seventy-three route declarations, but complete route-by-route integration test verification has not been established.

The project has also not demonstrated 90% branch coverage for every designated critical business logic module. Several risk-sensitive frontend components, including payment initiation, checkout, appointment booking, and administrative staff editing, have recorded branch coverage below this threshold.

The hosted PayHere payment-confirmation workflow remains blocked or unverified in the available documentation.

The DDP-087 execution-provenance addendum records local test revisions and timestamps for both repositories. However, correspondence between these tested revisions and the final deployed release remains unverified.

Consequently, successful local regression testing must not be treated as complete release or user-acceptance sign-off.

6.8 Outstanding Requirements and Known Defects

The final SRS distinguishes original requirements that remain incomplete or unverified from approved changes to the project scope.

Table 6.3: Outstanding Requirements and Verification Gaps

Area	Outstanding Item	Classification
Product Catalogue	Separate style-based filtering	Functional requirement gap
Inventory	Full real-time synchronization evidence	Verification gap
Inventory	Automated inter-branch stock transfers	Implementation limitation
Loyalty	Point redemption and tier-based rewards	Implementation limitation
Reporting	Historical inventory turnover analysis	Implementation limitation
Sales Rules	Automated promotional discount restriction	Unverified business rule

Prescription Orders	Automated 24-month prescription-age validation	Unverified business rule
Returns	Automated seven-day frame return policy	Implementation gap
Session Security	Separate 30-minute inactivity termination	Unverified business rule
Reliability	Sustained 99.9% uptime evidence	Verification gap
Usability	Staff 10-minute transaction assessment	Verification gap
Portability	Chrome, Firefox, and Safari compatibility evidence	Verification gap
Operations	Backup restoration and long-term retention verification	Operational verification gap
API Testing	Complete verified integration assertions for inventoried routes	Verification gap
Test Coverage	90% branch coverage for all designated critical business logic	Coverage gap
Payments	Hosted PayHere payment-confirmation E2E test	Blocked / Not verified
E2E Testing	Final scenario-level evidence reconciliation	Verification gap
Release Evidence	Confirm tested-to-deployed commit parity	Traceability gap
CI Evidence	Retain corresponding workflow runs and coverage artifacts	Evidence gap

These items represent documented limitations or outstanding requirements, not necessarily confirmed software defects.

A separate defect register should identify actual reproducible faults, their severity, affected versions, and resolution status.

At the time of this draft, a complete final known-defect register has not been independently verified. Any remaining defects confirmed during the final QA and acceptance activities should be added here.

6.9 Future Work

The identified limitations provide opportunities to extend OSMS beyond its current approved implementation.

6.9.1 Enhanced Product Discovery

Future development could introduce additional product filtering capabilities, including style-based filtering and other relevant product attributes.

6.9.2 Advanced Inventory Operations

Automated stock transfers between branches and historical stock movement analysis could improve inventory coordination and business visibility.

6.9.3 Extended Loyalty Management

Implementing customer loyalty point redemption and tier-based rewards would extend the current loyalty accrual functionality.

6.9.4 Automated Appointment Communication

Future versions could introduce scheduled appointment reminders through approved email or SMS providers, subject to updated requirements and communication policies.

6.9.5 Advanced Business Analytics

Additional reporting could include historical inventory turnover, appointment trends, and other operational metrics required by business management.

6.9.6 Improved Operational Reliability

Future work could strengthen operational monitoring, automated backups, tested restoration procedures, long-term audit archival, and service availability verification.

6.9.7 Extended Security Verification

Further security improvements could include evaluating alternatives to browser localStorage for authentication tokens, resolving remaining dependency findings, and completing broader privacy and compliance assessments.

6.9.8 Enhanced Cross-Browser and Usability Evaluation

Additional testing should evaluate the application across supported browsers and measure the original staff usability criterion.

These improvements should be prioritized based on business value, technical risk, stakeholder feedback, and the project's future maintenance resources.

6.10 Overall Project Evaluation

The OSMS project has achieved substantial implementation of the principal functional requirements identified during the system analysis and design phases.

According to the final SRS, nine functional requirements are classified as Implemented, two as Partially Implemented, and three as Modified by Approved Decision.

The system provides integrated support for customer accounts, product browsing, shopping carts, checkout, order management, inventory, Doctor Sessions, appointments, clinical prescriptions, loyalty points, chatbot assistance, reporting, and administrative operations.

The final QA evidence demonstrates significant automated verification activity. A total of 3,416 automated tests passed across the backend and frontend applications during the documented local executions.

The recorded coverage results exceeded 80% for statement, branch, function, and line coverage in both repositories. Controlled performance testing also demonstrated satisfactory local latency, throughput, transaction consistency, and selected error-handling behavior.

However, the results do not establish complete satisfaction of every functional and non-functional acceptance criterion.

Remaining verification gaps include critical business logic branch coverage, complete API endpoint integration evidence, successful hosted PayHere payment confirmation, formal client acceptance, and tested-to-deployed revision traceability.
Several original non-functional requirements, including long-term availability, usability, scalability, maintainability, and browser compatibility, remain partially verified or unverified.

Overall, OSMS represents a substantial implementation of its approved functional scope, supported by significant positive automated testing evidence. Nevertheless, final release

readiness and complete requirements compliance require the outstanding verification activities to be resolved or formally accepted with their documented limitations.

6.11 Chapter Summary

This chapter evaluated the Optical Shop Management System against the approved project objectives, functional requirements, non-functional requirements, and available verification evidence.

The evaluation identified substantial implementation of the core retail, clinical, inventory, customer-service, and administrative modules.

Approved architectural decisions were used to address significant design and operational challenges during development.

Performance and security assessments provide evidence of selected system quality characteristics, while several original non-functional acceptance criteria remain incompletely verified.

The evaluation also identified limitations concerning product discovery, inventory operations, loyalty redemption, reporting, external integrations, operational reliability, and formal acceptance evidence.

Future improvements were proposed to address these limitations and support further development of the system.

The overall findings provide a basis for the final project conclusion presented in Chapter 7.

CHAPTER 7 : CONCLUSION

7.1 Introduction

The **Optical Shop Management System (OSMS)** was developed for **Darshana Opticals (Pvt) Ltd**. to address the limitations associated with manual and disconnected retail, clinical, customer-service, and administrative operations.

The primary objective of the project was to develop an integrated, web-based platform capable of supporting core optical retail activities while improving customer accessibility, centralized information management, and operational efficiency.

The project progressed from requirements analysis and system design to implementation, integration, and software verification. The original Software Requirements Specification (SRS v1.0) and Software Design Specification (SDS v1.0) established the initial requirements and architectural foundation. Subsequent development decisions were documented through Architectural Decision Records (ADR-001 to ADR-015) and incorporated into the implementation-aligned SRS v1.1 and SDS v1.1.

This chapter summarizes the principal achievements, software engineering contributions, remaining limitations, and overall outcome of the project.

7.2 Summary of Project Achievements

The project resulted in the development of an integrated Optical Shop Management System using the MERN technology stack, consisting of MongoDB, Express.js, React.js, and Node.js.

The implemented application supports several major functional areas required for the operations of Darshana Opticals.

The customer-facing functionality includes account registration and authentication, product browsing, persistent shopping-cart management, checkout, order tracking, appointment booking, prescription-history access, loyalty information, and AI-assisted informational support.

Operational functionality includes branch-specific inventory management, automated reorder alerts, order processing, Doctor Session administration, appointment management, and clinical prescription recording.

Administrative functionality includes authorized staff and Doctor management, sales and inventory reporting, and the recording of relevant audit events.

The system also integrates external services, including PayHere for payment processing, Cloudinary for inventory image management, Google Gemini AI for chatbot assistance, and SMTP-based services for supported email communication.

These capabilities demonstrate substantial progress towards the project's objective of providing a centralized digital platform for optical retail and clinical operations.

According to the final SRS, nine of the fourteen functional requirements are classified as Implemented, two as Partially Implemented, and three as Modified by Approved Decision.

This classification indicates that the principal functional modules have been developed while preserving traceability for incomplete requirements and formally approved scope changes.

In addition to the implemented application functionality, the project produced substantial automated testing evidence. The final DDP-087 documentation records 2,075 passing backend tests and 1,341 passing frontend tests, with zero reported failures across the documented local executions.

Both repositories achieved more than 80% overall statement, branch, function, and line coverage. These results demonstrate the project's investment in automated regression testing and source-code verification.

However, the coverage measurements and test execution totals do not independently establish complete functional acceptance, integration coverage, or production readiness.

7.3 Achievement of Project Objectives

The project addressed several important objectives established during the requirements analysis phase.

The implementation of a digital product catalogue, shopping cart, checkout process, and payment integration provides customers with an alternative to relying exclusively on physical-store interactions for supported purchasing activities.

Centralized branch-specific inventory management improves the organization and availability of stock information, while reorder alerts support the identification of products requiring replenishment.

The implementation of online appointment booking through capacity-based Doctor Sessions provides a structured mechanism for managing eye examination appointments at the approved Gampaha Main Branch.

Customer profiles and clinical prescription records provide a centralized approach to maintaining relevant personal and clinical information, subject to the application's authorization controls.

Loyalty point accrual and AI-powered informational assistance extend the customer-service capabilities of the platform.

Furthermore, reporting and administrative modules provide authorized business users with tools for managing selected operations and reviewing sales and inventory information.

These outcomes demonstrate that the project has achieved a substantial portion of its approved functional objectives.

However, full achievement of every original requirement cannot be asserted because certain functions remain partially implemented, some requirements were modified through approved architectural decisions, and several quality-related acceptance criteria have not been fully verified.

7.4 Software Engineering Contributions

In addition to delivering application functionality, the project demonstrates the application of software engineering principles throughout the development lifecycle.

A three-tier layered architecture was used to separate frontend presentation, backend business processing, and persistent data storage.

The application was structured into reusable frontend components and modular backend services, supporting maintainability and the separation of concerns.

MongoDB schemas, validation rules, document relationships, indexes, and database transactions were used to support consistent management of operational information.

Authentication and authorization mechanisms were implemented to manage access to customer, staff, administrative, and clinical functionality.

Business-critical operations, including checkout and appointment booking, use backend-controlled validation and relevant transactional safeguards.

The project also employed GitHub repositories, feature branches, Pull Requests, coding standards, automated tests, and continuous integration to support collaborative development and source-code quality.

One of the important engineering contributions was the use of Architectural Decision Records to document approved changes and maintain traceability between the original design and final implementation.

These practices provided a structured approach to managing technical complexity and evolving implementation requirements.

7.5 Implementation Challenges and Lessons Learned

The development of OSMS involved several challenges relating to business-rule interpretation, integration, data consistency, security, and deployment.

The original authorization structure required refinement to establish consistent responsibilities across the frontend and backend. ADR-002 addressed this challenge by establishing five authoritative application roles.

Checkout and payment processing required particular attention because order creation, inventory reservation, external payment confirmation, and payment failures involve multiple related states. Separating these responsibilities and applying backend-controlled validation helped establish a more consistent transaction workflow.

Appointment scheduling also required a revised design to support capacity-based Doctor Sessions and maintain separation between Doctor records and authenticated Optometrist responsibilities.

The integration of external services introduced additional configuration, security, and operational considerations, particularly for payment processing, hosted images, AI assistance, and notifications.

The original AWS-based deployment architecture was subsequently revised to use Vercel, Render.com, and MongoDB Atlas as the implementation-aligned hosting solution.

These challenges highlighted the importance of defining clear system boundaries, maintaining reliable business-rule enforcement, verifying integrations independently, and documenting significant design decisions.

The project also demonstrated that implementing a software feature does not automatically establish compliance with every associated non-functional requirement.

Measurable acceptance criteria require appropriate testing conditions, recorded evidence, and, where relevant, sustained operational observation.

7.6 Testing and Quality Evaluation

The project adopted a multi-level software testing approach involving backend unit and integration testing, frontend component testing, system integration and end-to-end testing, security assessment, and controlled performance and reliability testing.

The final automated testing evidence recorded under DDP-087 demonstrates successful local execution of 2,075 backend Jest tests across 118 test suites and 1,341 frontend Vitest tests across 110 test files.

All 3,416 documented automated tests passed, with zero failed assertions during the recorded executions.

Backend code coverage reached 92.34% for statements, 81.68% for branches, 97.86% for functions, and 93.00% for lines.

Frontend coverage reached 88.05% for statements, 81.80% for branches, 90.95% for functions, and 88.86% for lines.

The final Test Case Register documents forty representative automated test scenarios with supporting requirement mappings.

Performance testing under DDP-086 demonstrated P95 response times below 130 milliseconds across thirteen selected backend endpoints. At thirty concurrent virtual users, the recorded P95 response time was approximately 241.8 milliseconds, with no reported HTTP 5xx failures in the controlled workload.

Concurrency testing also recorded consistent inventory reservations and Doctor Session capacity enforcement, while database profiling demonstrated improved query efficiency following targeted indexing.

Security testing provided evidence of selected authentication, authorization, input-validation, and error-handling controls.

Nevertheless, the evaluation identified important verification limitations. The required 90% critical-business-logic branch coverage has not been demonstrated across every designated module, complete API endpoint-level integration coverage remains unverified, and hosted PayHere payment completion remains blocked or unverified.

The DDP-087 documentation records the local tested revisions, but their equivalence to the final deployed release has not been independently established.

Accordingly, the available testing evidence demonstrates substantial automated quality assurance activity and positive results under the recorded conditions. It does not establish complete compliance with every original non-functional requirement or unrestricted production readiness.

7.7 Project Limitations

Although the system provides its principal operational modules, several limitations remain within the current implementation.

The product catalogue does not include every filtering capability originally proposed.

Automatic inter-branch inventory transfers and historical stock movement reporting are unavailable.

The loyalty system supports eligible point accrual but does not provide point redemption or tier-based reward functionality.

Automated appointment email and SMS reminders were excluded from the approved current scope, with appointment communication handled manually by authorized branch staff.

The current reporting implementation does not include historical inventory turnover analysis or dedicated appointment-frequency analytics.

The system also does not integrate directly with medical diagnostic hardware or third-party courier tracking services.

From an operational perspective, additional verification is required to establish sustained system availability, complete data backup and restoration readiness, long-term audit retention, and full regulatory compliance.

These limitations do not invalidate the implemented functionality but establish boundaries that must be considered when evaluating overall project completion and operational readiness.

The final quality assurance assessment also identifies verification limitations that extend beyond the implementation of individual features.

Although all documented local automated tests passed, complete API endpoint integration verification, critical-module branch coverage compliance, and final hosted payment confirmation remain outstanding.

The tested source revisions have been documented, but their correspondence to the final deployed application requires additional confirmation.

These limitations must be considered when interpreting the project's verification results and determining final operational acceptance.

7.8 Future Development Opportunities

The existing architecture provides a foundation for extending the system to address additional business requirements.

Potential improvements include enhanced product filtering, automated inter-branch stock transfers, loyalty point redemption, tier-based rewards, and additional operational analytics.

Future versions could also introduce automated appointment reminders where formally approved by the business.

Operational improvements may include expanded monitoring, verified backup and restoration procedures, stronger long-term data retention mechanisms, and additional deployment reliability controls.

Further security, usability, accessibility, and cross-browser testing would provide stronger evidence of compliance with the original quality requirements.

These enhancements should be prioritized according to stakeholder needs, business value, available development resources, and the outcomes of future system evaluation.

7.9 Overall Conclusion

The Optical Shop Management System represents a substantial implementation of the digitalization objectives established for Darshana Opticals (Pvt) Ltd.

The project transformed its requirements and architectural design into an integrated web-based application supporting retail, inventory, appointment, clinical, customer-service, and administrative workflows.

The implementation demonstrates the practical application of modular software architecture, database modelling, transactional consistency, role-based authorization, third-party service integration, version control, peer review, and automated software testing.

The final documented local quality assurance executions completed successfully with 3,416 passing automated tests. Both frontend and backend applications exceeded 80% aggregate code coverage across the four reported metrics.

Controlled performance and concurrency testing also provided positive evidence of selected backend response times, transaction integrity, and database query efficiency.

However, not all original functional and non-functional acceptance criteria have been fully demonstrated. Important remaining verification activities include complete endpoint-level integration evidence, critical business logic branch coverage, hosted payment confirmation, and final deployed-release traceability.

Overall, the project establishes a functional, maintainable, and extensible foundation for the digitalization of Darshana Opticals' principal business operations.

Further verification, stakeholder acceptance, and targeted enhancements will be necessary to address the remaining requirements and establish full operational readiness.

CHAPTER 8 — REFERENCES

This chapter presents the academic, technical, and project-specific sources supporting the development and evaluation of the Optical Shop Management System (OSMS) for Darshana Opticals (Pvt) Ltd.

The references are organized into two categories: academic and technical references, which provide the theoretical and technological foundation, and project evidence references, which provide traceability to the approved requirements, architectural decisions, implementation artifacts, testing results, and deployment documentation.

The IEEE numerical referencing style is used throughout the report.

8.1 Academic and Technical References

[1] Software Engineering Teaching Unit, *SENG 34213 – System Development Project*, course guideline, Faculty of Science, University of Kelaniya, Sri Lanka, 2026.

[2] React Team, “Describing the UI,” *React Documentation*. [Online]. Available: https://react.dev/learn/describing-the-ui

[3] OpenJS Foundation, “Introduction to Node.js,” *Node.js Documentation*. [Online]. Available: https://nodejs.org/learn/getting-started/introduction-to-nodejs

[4] Express.js, “Routing,” *Express.js Guide*. [Online]. Available: https://expressjs.com/en/guide/routing/

[5] MongoDB, Inc., “Transactions,” *MongoDB Database Manual*. [Online]. Available: https://www.mongodb.com/docs/manual/core/transactions/

[6] Mongoose, “Schemas,” *Mongoose Documentation*. [Online]. Available: https://mongoosejs.com/docs/guide.html

[7] PayHere, “Checkout API,” *PayHere Knowledge Base*. [Online]. Available: https://support.payhere.lk/api-%26-mobile-sdk/checkout-api

[8] Cloudinary, “Node.js SDK,” *Cloudinary Documentation*. [Online]. Available: https://cloudinary.com/documentation/node_integration

[9] Google, “Gemini API,” *Google AI for Developers*. [Online]. Available: https://ai.google.dev/gemini-api/docs

[10] GitHub, “Workflow Syntax for GitHub Actions,” *GitHub Documentation*. [Online]. Available: https://docs.github.com/en/actions/reference/workflows-and-actions/workflow-syntax

[11] Vercel, “Deploying Git Repositories with Vercel,” *Vercel Documentation*. [Online]. Available: https://vercel.com/docs/git

[12] Render, “Deploy a Node Express App on Render,” *Render Documentation*. [Online]. Available: https://render.com/docs/deploy-node-express-app

[13] OWASP Foundation, “OWASP Top 10:2021,” *OWASP Web Application Security Risks*. [Online]. Available: https://top10.owasp.org/2021/

8.2 Project Evidence References

Original Design and Requirements Documentation

[14] Darshana Opticals OSMS Project Team, *Optical Shop Management System – Final Design Report*, SENG 31242 System Design Project, University of Kelaniya, Sri Lanka, Apr. 2026.

[15] Darshana Opticals OSMS Project Team, *Software Requirements Specification (SRS)*, approved baseline, version 1.0, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/srs/srs-v1.0.pdf

[16] Darshana Opticals OSMS Project Team, *Software Requirements Specification (SRS)*, implementation-aligned version 1.1, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/srs/srs-final.pdf

[17] Darshana Opticals OSMS Project Team, *Software Design Specification (SDS)*, approved baseline, version 1.0, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/SDS/sds-v1.0.pdf

[18] Darshana Opticals OSMS Project Team, *Software Design Specification (SDS)*, implementation-aligned version 1.1, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/SDS/sds-final.pdf

Architectural Decision Records

[19] Darshana Opticals OSMS Project Team, “ADR-001: Clinical Prescription Data Storage Design,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-001-clinical-prescription-storage.md

[20] Darshana Opticals OSMS Project Team, “ADR-002: Authoritative RBAC Role Model,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-002-rbac-role-model.md

[21] Darshana Opticals OSMS Project Team, “ADR-003: Shopping Cart Persistence and Checkout Boundary,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-003-shopping-cart-checkout-boundary.md

[22] Darshana Opticals OSMS Project Team, “ADR-004: Order Lifecycle and Tracking Status Model,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-004-order-lifecycle-tracking.md

[23] Darshana Opticals OSMS Project Team, “ADR-005: Loyalty Points Calculation and Persistence Model,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-005-loyalty-points-model.md

[24] Darshana Opticals OSMS Project Team, “ADR-006: Inventory Reorder-Alert Rules and Persistence Model,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-006-inventory-reorder-alert-model.md

[25] Darshana Opticals OSMS Project Team, “ADR-007: Appointment Scheduling, Availability, and Lifecycle Rules,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-007-appointment-scheduling-lifecycle.md

[26] Darshana Opticals OSMS Project Team, “ADR-008: Inventory Image Upload and Cloudinary Storage Architecture,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-008-inventory-image-upload-cloudinary.md

[27] Darshana Opticals OSMS Project Team, “ADR-009: Appointment Communication and Automated Reminder Scope Decision,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-009-appointment-communication-policy.md

[28] Darshana Opticals OSMS Project Team, “ADR-010: AI Chatbot Architecture and Knowledge Strategy,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-010-ai-chatbot-architecture-and-knowledge-strategy.md

[29] Darshana Opticals OSMS Project Team, “ADR-011: Secure Payment Gateway Integration and Payment Lifecycle Architecture,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-011-secure-payment-gateway-integration.md

[30] Darshana Opticals OSMS Project Team, “ADR-012: Authoritative Immutable Audit-Trail Architecture,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-012-immutable-audit-trail-architecture.md

[31] Darshana Opticals OSMS Project Team, “ADR-013: Authoritative Order-Status Notification Policy,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-013-order-status-notification-policy.md

[32] Darshana Opticals OSMS Project Team, “ADR-014: Authoritative Sales and Inventory Reporting Definitions, Metrics, and Access Policy,” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-014-sales-inventory-reporting-definitions.md

[33] Darshana Opticals OSMS Project Team, “ADR-015: Staging Backend Container Hosting Operational Fallback (Render.com),” 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/adr/ADR-015-staging-backend-container-hosting-fallback.md

Testing and Quality Assurance Evidence

[34] Darshana Opticals OSMS Project Team, “DDP-084: End-to-End Execution Report,” OSMS Infrastructure Repository, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra/blob/develop/docs/e2e-execution-report.md

[35] Darshana Opticals OSMS Project Team, *OWASP Security Verification Checklist*, DDP-085, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/security/owasp-checklist.pdf

[36] Darshana Opticals OSMS Project Team, “DDP-085: Backend Security Assessment and Hardening Evidence,” Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/backend/blob/develop/docs/security/owasp-assessment.md

[37] Darshana Opticals OSMS Project Team, *Performance Test Results Report*, DDP-086, Version 1.0 (Official Release Gate), Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/testing/performance-report.pdf

[38] Darshana Opticals OSMS Project Team, *Final Test Case Register*, DDP-087, evidence baseline Oct. 25, 2026, revised Oct. 26, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/testing/test-register.pdf

[39] Darshana Opticals OSMS Project Team, *Final Test Coverage Report*, DDP-087, historical evidence Oct. 25, 2026, updated Oct. 26, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents/blob/develop/testing/coverage-sprint8.pdf

[40] Darshana Opticals OSMS Project Team, “DDP-091: Final User Acceptance Testing Report,” OSMS Infrastructure Repository, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra/blob/develop/docs/demo/uat-report.md

[41] Darshana Opticals OSMS Project Team, “DDP-092: Final Defect and Regression Summary,” OSMS Infrastructure Repository, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra/blob/develop/docs/quality/defect-regression-summary.md

Deployment and Continuous Integration Evidence

[42] Darshana Opticals OSMS Project Team, “DDP-083: Frontend Vercel and Render Deployment Runbook,” OSMS Infrastructure Repository, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra/blob/develop/docs/frontend-vercel-deployment.md

[43] Darshana Opticals OSMS Project Team, “Final Software Release Manifest,” OSMS Infrastructure Repository, Oct. 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra/blob/develop/docs/release/release-manifest.md

[44] Darshana Opticals OSMS Project Team, “Backend Continuous Integration Workflow,” GitHub Actions YAML configuration, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/backend/blob/develop/.github/workflows/ci.yml

[45] Darshana Opticals OSMS Project Team, “Frontend Continuous Integration Workflow,” GitHub Actions YAML configuration, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/frontend/blob/develop/.github/workflows/frontend-ci.yml

Source Code Repositories

[46] Darshana Opticals OSMS Project Team, *OSMS Backend Repository*, GitHub, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/backend

[47] Darshana Opticals OSMS Project Team, *OSMS Frontend Repository*, GitHub, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/frontend

[48] Darshana Opticals OSMS Project Team, *OSMS Infrastructure Repository*, GitHub, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/infra

[49] Darshana Opticals OSMS Project Team, *OSMS Project Documentation Repository*, GitHub, 2026. [Online]. Available: https://github.com/darshana-opticals-osms/documents

Online sources accessed: 26 October 2026.

CHAPTER 9 : APPENDICES

9.1 Introduction

The appendices provide supplementary documents, diagrams, implementation references, and verification evidence supporting the development and evaluation of the **Optical Shop Management System (OSMS)** for **Darshana Opticals (Pvt) Ltd**.

These materials complement the main chapters of the Final Development Report by providing additional technical details and traceability without unnecessarily repeating information in the main report.

The appendices include references to the approved requirements and design documents, architectural diagrams, database models, application screenshots, selected source-code modules, Architectural Decision Records (ADRs), testing documentation, deployment evidence, and project management records.

The original SRS and SDS are retained as historical baselines, while their implementation-aligned versions provide the latest documented system specifications.

Evidence that has not yet been independently verified is identified as pending rather than presented as completed.

Appendix A — Requirements and Design Documentation

A.1 Original Software Requirements Specification

Document: Software Requirements Specification (SRS), Version 1.0

Approval baseline date: 21 March 2026

Repository: Documents

File: `srs/srs-v1.0.pdf`

The original SRS establishes the approved project scope, functional requirements, non-functional requirements, business rules, constraints, use cases, feasibility analysis, and project risks.

The document defines functional requirements FR-001 to FR-014 and non-functional requirements NFR-001 to NFR-007.

Document reference: [1]

A.2 Original Software Design Specification

Document: Software Design Specification (SDS), Version 1.0

Baseline date: 19 April 2026

Repository: Documents

File: `SDS/sds-v1.0.pdf`

The original SDS describes the initial architectural design, database structure, class diagrams, sequence diagrams, user interface designs, security architecture, and deployment plan.

It serves as the historical design baseline against which implementation-related changes were documented.

Document reference: [2]

A.3 Final Software Requirements Specification

Document: Software Requirements Specification (SRS), Version 1.1

Revision date: October 2026

File: `srs/srs-final.pdf`

The implementation-aligned SRS records the current functional requirements, approved modifications, implementation statuses, known limitations, and non-functional verification status.

The final document preserves the original requirement identifiers and provides traceability to relevant architectural decisions.

Document reference: [3]

A.4 Final Software Design Specification

Document: Software Design Specification (SDS), Version 1.1

Revision date: October 2026

File: `SDS/sds-final.pdf`

The implementation-aligned SDS describes the developed architecture, database collections, component responsibilities, major workflows, security boundaries, and updated deployment design.

The document reflects the approved Architectural Decision Records and the current Vercel, Render.com, MongoDB Atlas, and Cloudinary architecture.

Document reference: [4]

Appendix B — System Architecture and Design Diagrams

This appendix identifies the principal diagrams used to document and explain the system design.

The diagrams are maintained in the final SDS and should be reproduced at a readable resolution when included in the report.

B.1 System Context Diagram

Source: Final SDS v1.1, Figure 1, printed page 1.

The diagram identifies the OSMS system boundary, principal user roles, and external service providers.

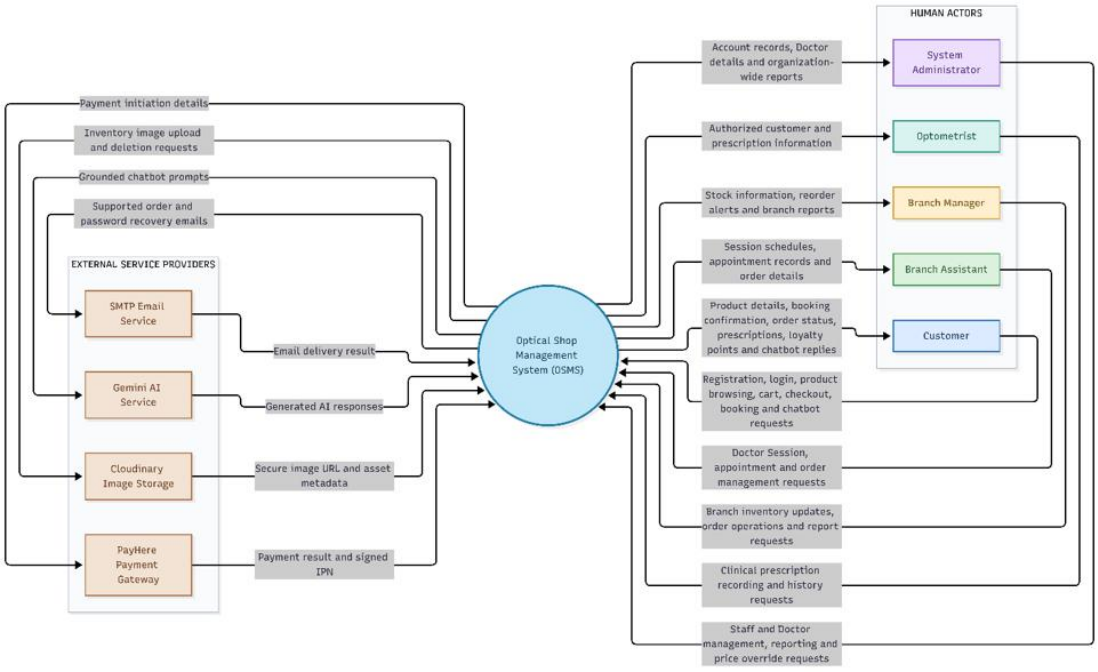


Figure B.1: OSMS System Context Diagram

B.2 High-Level Three-Tier Architecture

Source: Final SDS v1.1, Figure 2, printed page 3.

The diagram illustrates the separation between the presentation layer, application layer, and database layer.

File: SDS/sds-final.pdf

B.3 Logical Entity Relationship Diagram

Source: Final SDS v1.1, Figure 3, printed page 14.

The diagram illustrates the relationships among the principal MongoDB entities used by the system.

File: SDS/sds-final.pdf

B.4 Core Domain Model Class Diagram

Source: Final SDS v1.1, Figure 4(A), printed page 26.

The diagram represents the main application domain models, attributes, and relationships.

File: SDS/sds-final.pdf

B.5 Application Service Operations Diagram

Source: Final SDS v1.1, Figure 4(B), printed page 27.

The diagram summarizes the service-level operations responsible for important backend business processes.

File: SDS/sds-final.pdf

B.6 Major Sequence Diagrams

The final SDS includes sequence diagrams for the following activities:

Table B.1: Sequence Diagrams Referenced from the SDS

Figure	Sequence Diagram	SDS Printed Page
Figure 5	User Visit and Homepage	29
Figure 6	Authentication	31
Figure 7	Product Discovery	34

Figure 8	Cart Management	37
Figure 9	Checkout and Order Confirmation	40
Figure 10	Appointment Booking	44
Figure 11	Chatbot Interaction	48
Figure 12	Staff Order Processing and Tracking	53
Figure 13	Inventory Management and Reorder Alerts	56
Figure 14	Doctor Session Management	59
Figure 15	Prescription Recording and History	62

The original diagrams should be referenced from the final SDS. Detailed diagrams may be reproduced in this appendix if they are not already presented in the main report.

B.7 Deployment Architecture

Source: Final SDS v1.1, Figure 52, printed page 155.

The deployment diagram represents the application hosting arrangement and interactions among the frontend, backend, database, and external services.

File: `SDS/sds-final.pdf`

Appendix C — User Interface Screenshots

This appendix provides supporting screenshots of the implemented OSMS application.

Screenshots should be captured from a verified application build and should not contain real customer credentials, private patient information, or sensitive payment information.

The following screens are recommended for inclusion.

Table C.1: Recommended User Interface Screenshots

Screenshot ID	Application Screen	Related Requirement
C.1	Customer Homepage	FR-003
C.2	Customer Registration and Login	FR-001
C.3	Product Catalogue	FR-003
C.4	Product Details	FR-003
C.5	Shopping Cart	FR-004
C.6	Checkout and Order Confirmation	FR-004
C.7	PayHere Payment Interface (Sandbox)	FR-009
C.8	Customer Order History and Tracking	FR-014
C.9	Appointment Booking	FR-007
C.10	Doctor Session Management	FR-007
C.11	Prescription Recording	FR-013

C.12	Customer Prescription History	FR-002, FR-013
C.13	Inventory Management	FR-005
C.14	Reorder Alerts	FR-006
C.15	Loyalty Points and History	FR-010
C.16	AI Chatbot	FR-011
C.17	Sales Reporting	FR-012
C.18	Inventory Reporting	FR-012
C.19	System Administration	Administrative Requirements

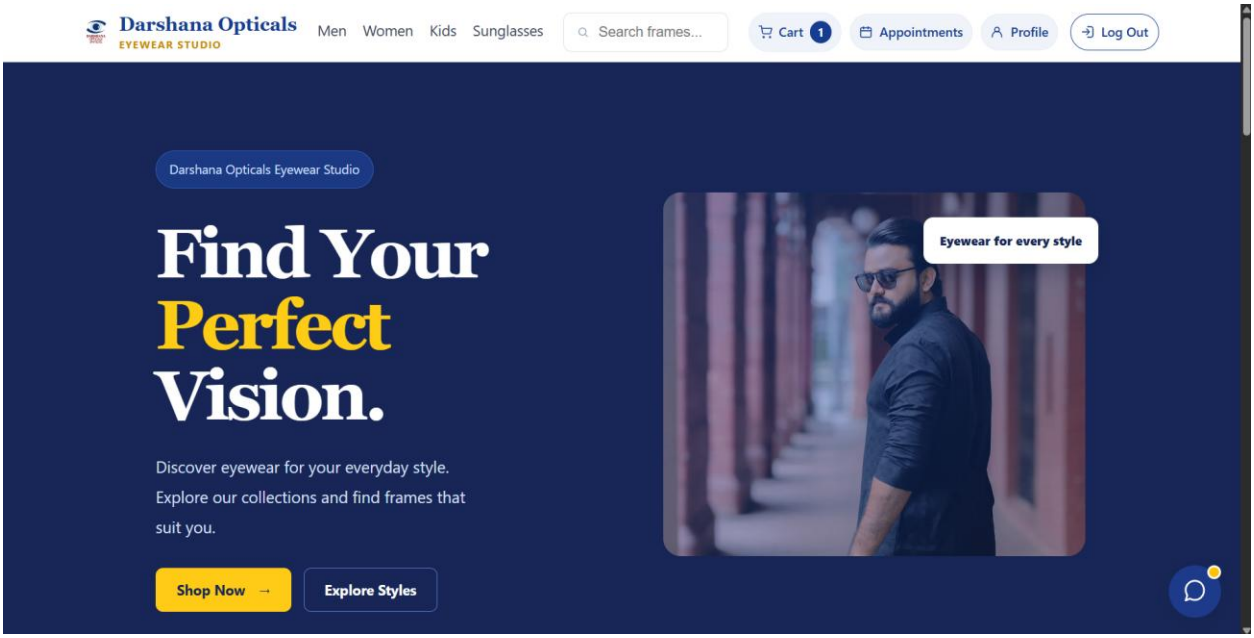


Figure C.1: Customer Homepage of the OSMS

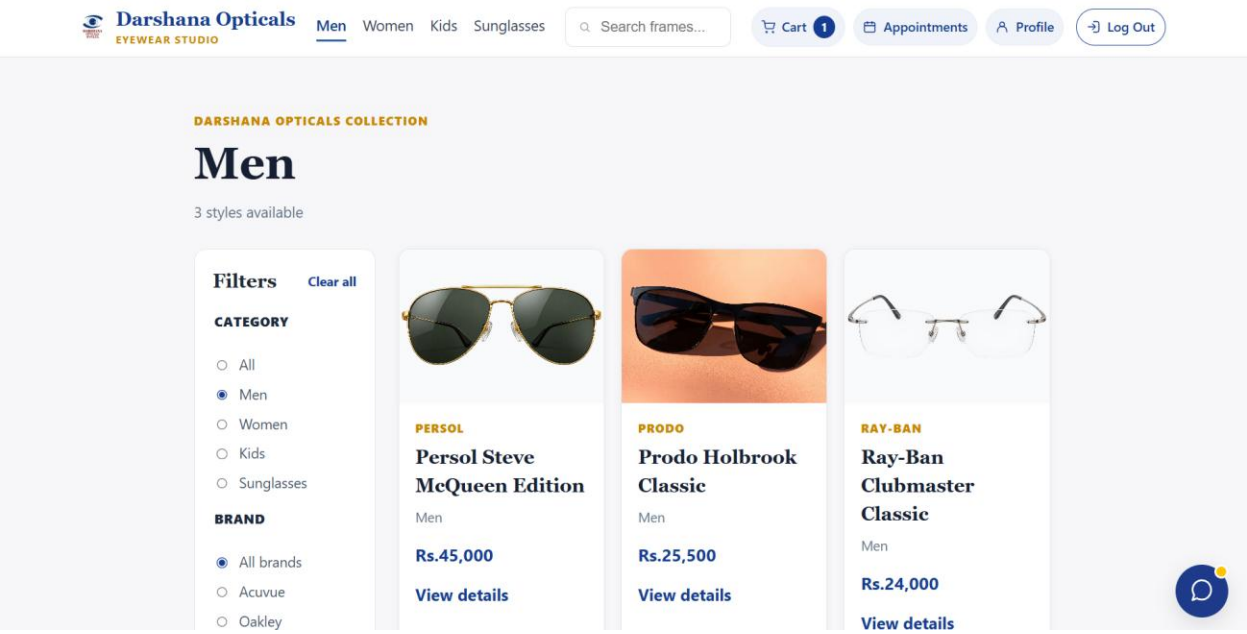


Figure C.2: Product Catalogue Interface

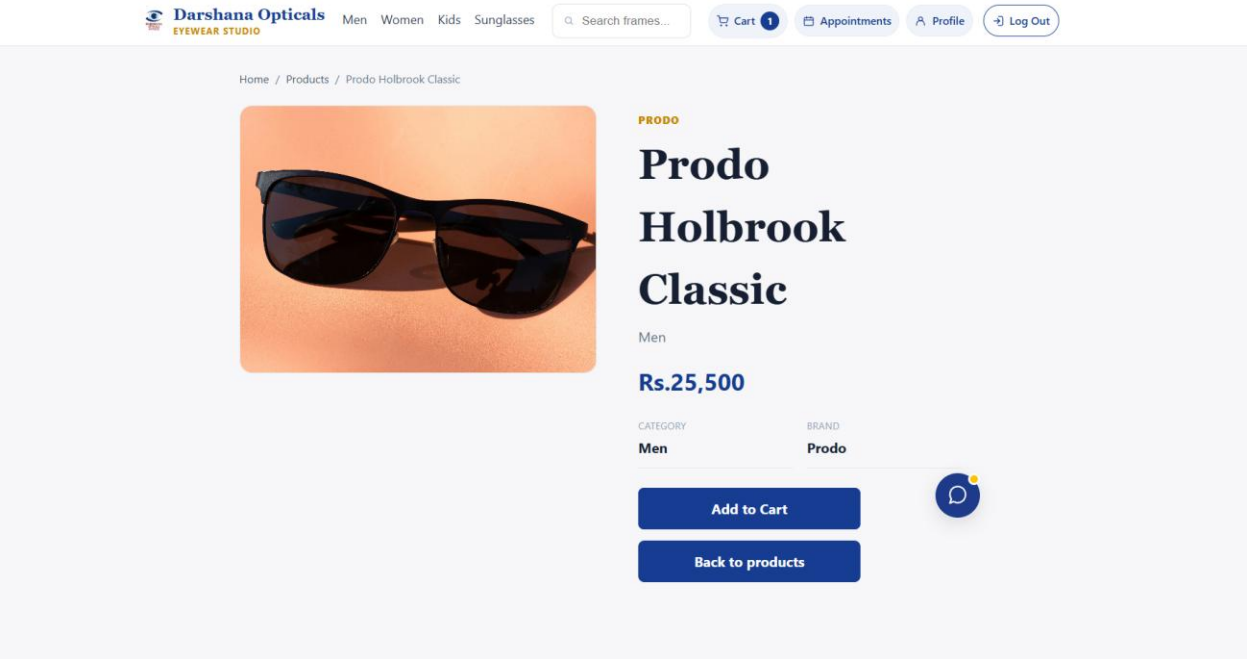


Figure C.3: Product Details Interface

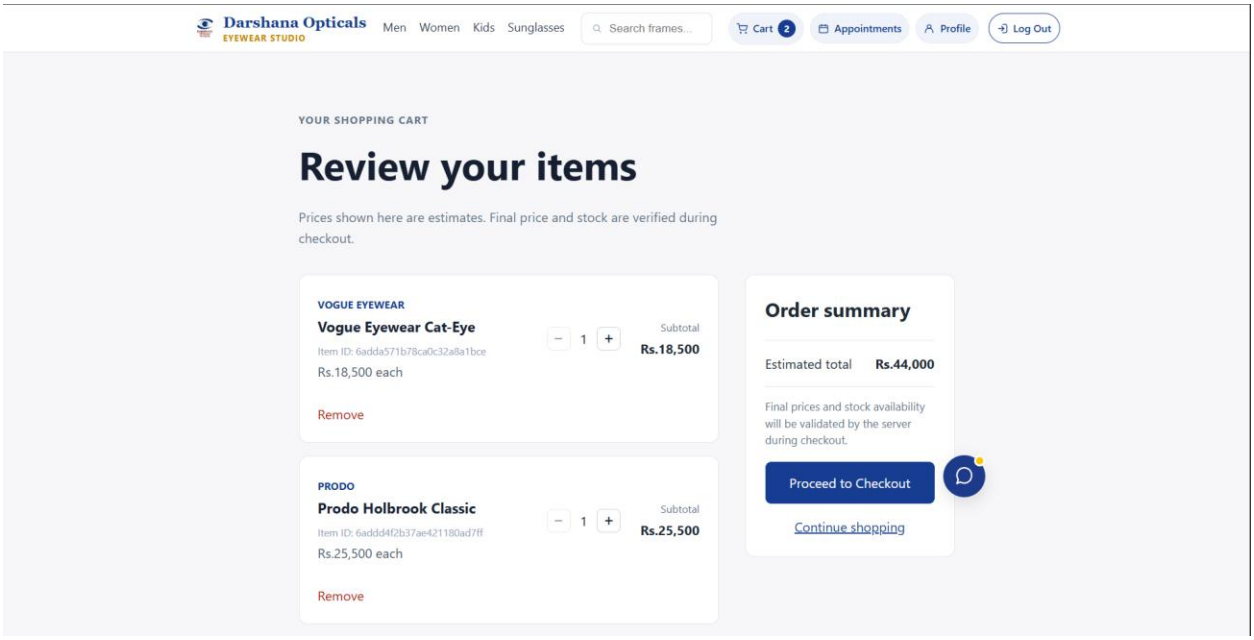


Figure C.4: Customer Shopping Cart Interface

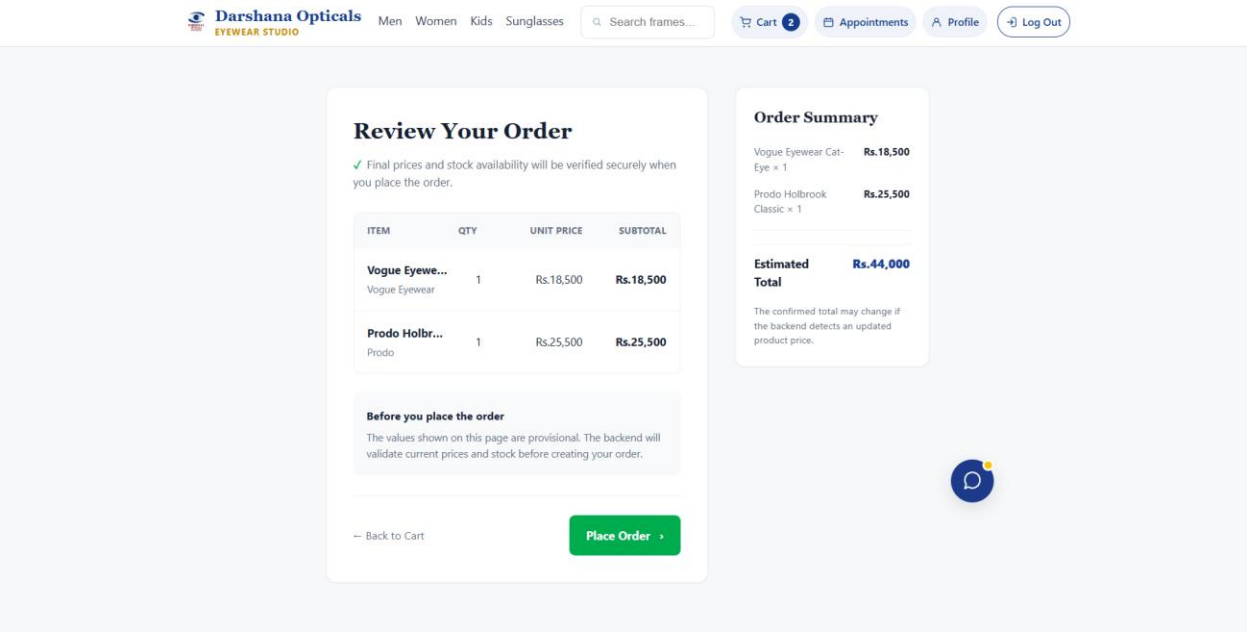


Figure C.5: Checkout and Order Confirmation Interfaces

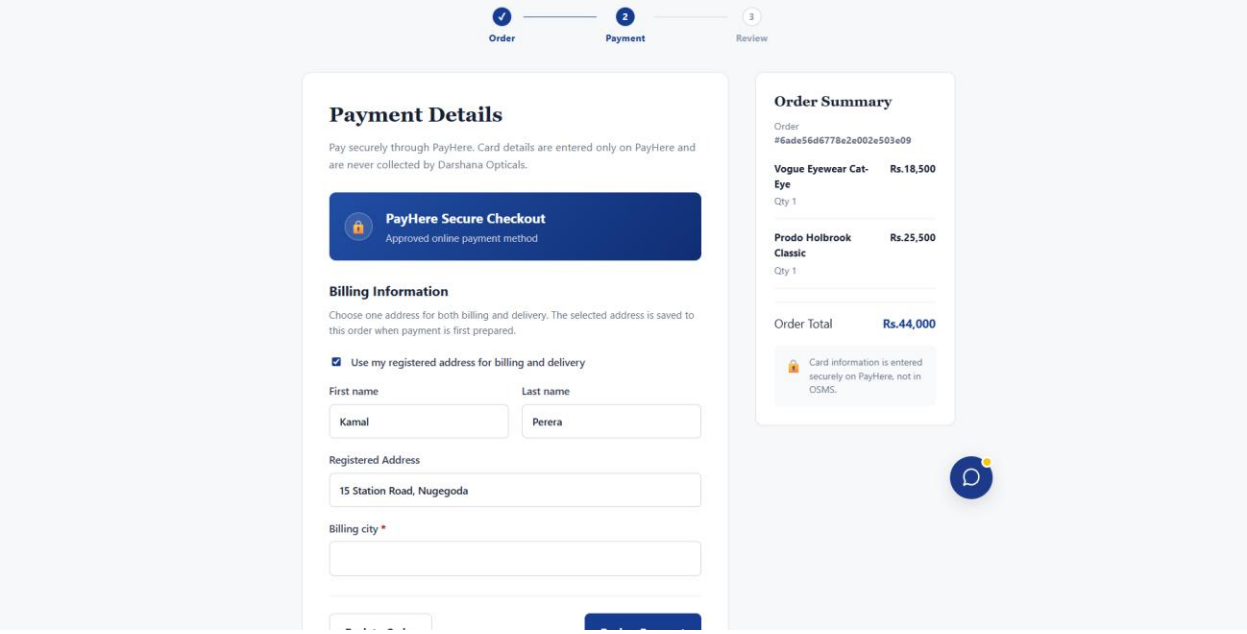


Figure C.6: PayHere Sandbox Payment Interface

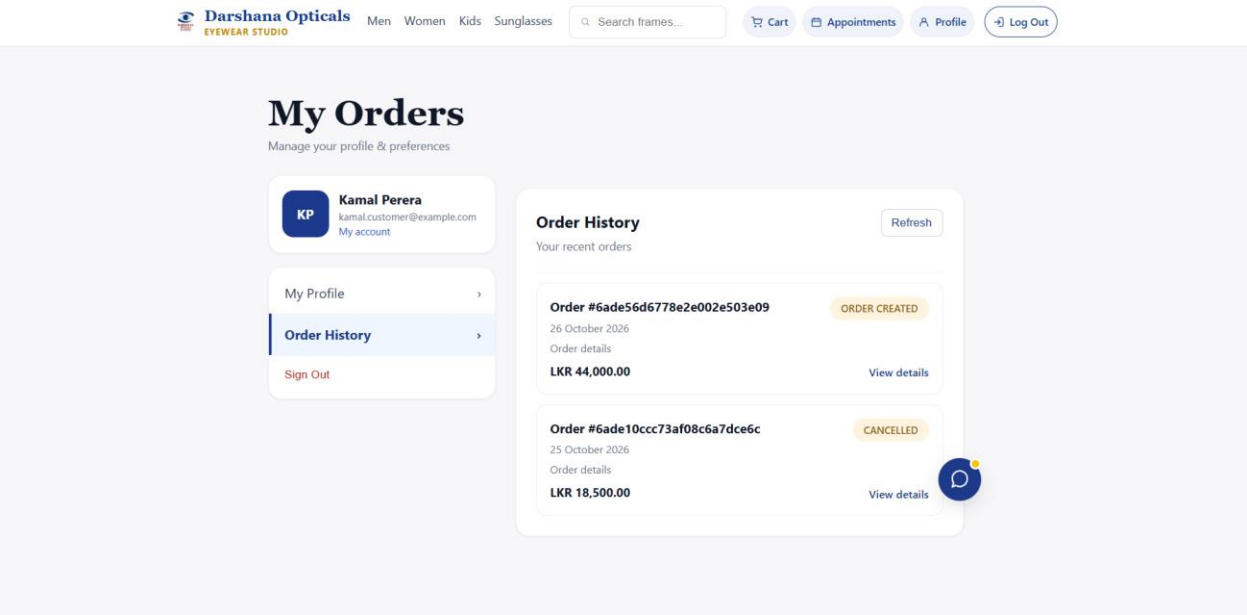


Figure C.7: Customer Order History and Order Tracking Interfaces

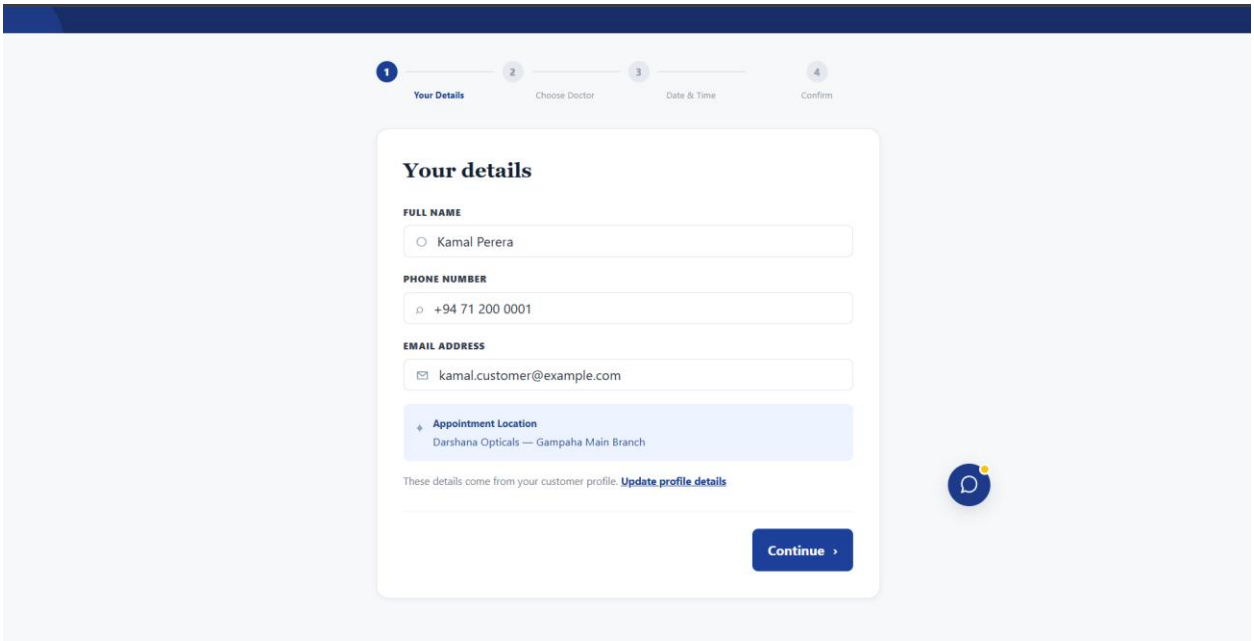


Figure C.8: Customer Appointment Booking Interface

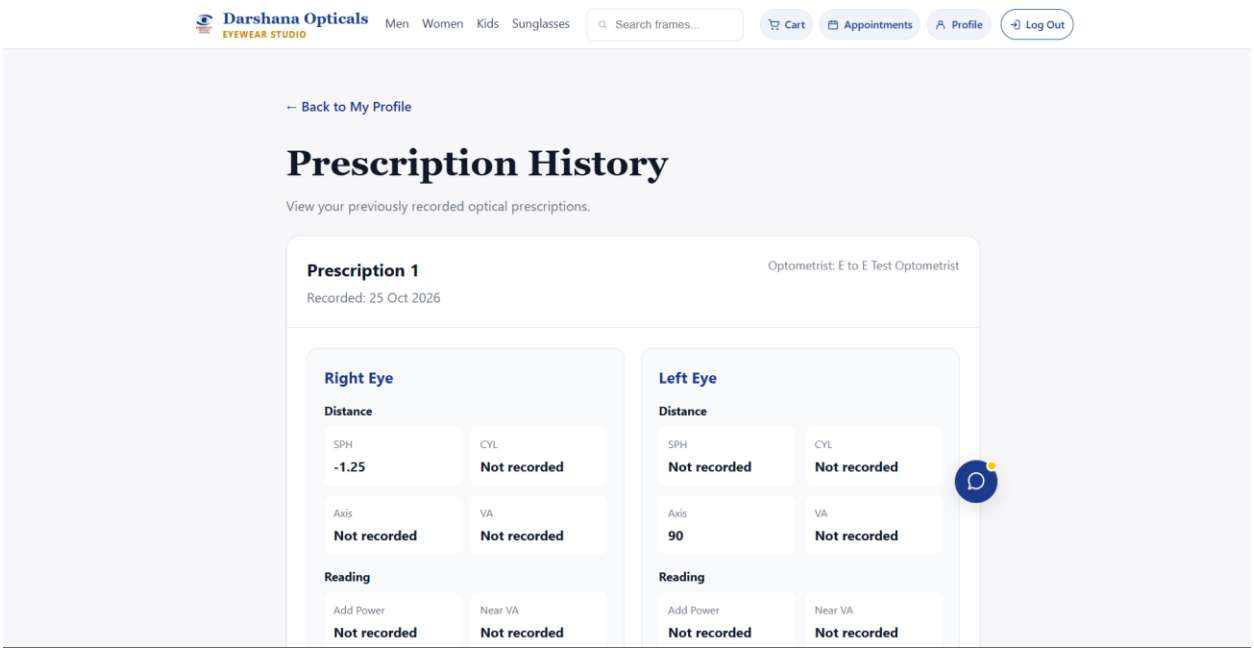


Figure C.9: Customer Prescription History Interface

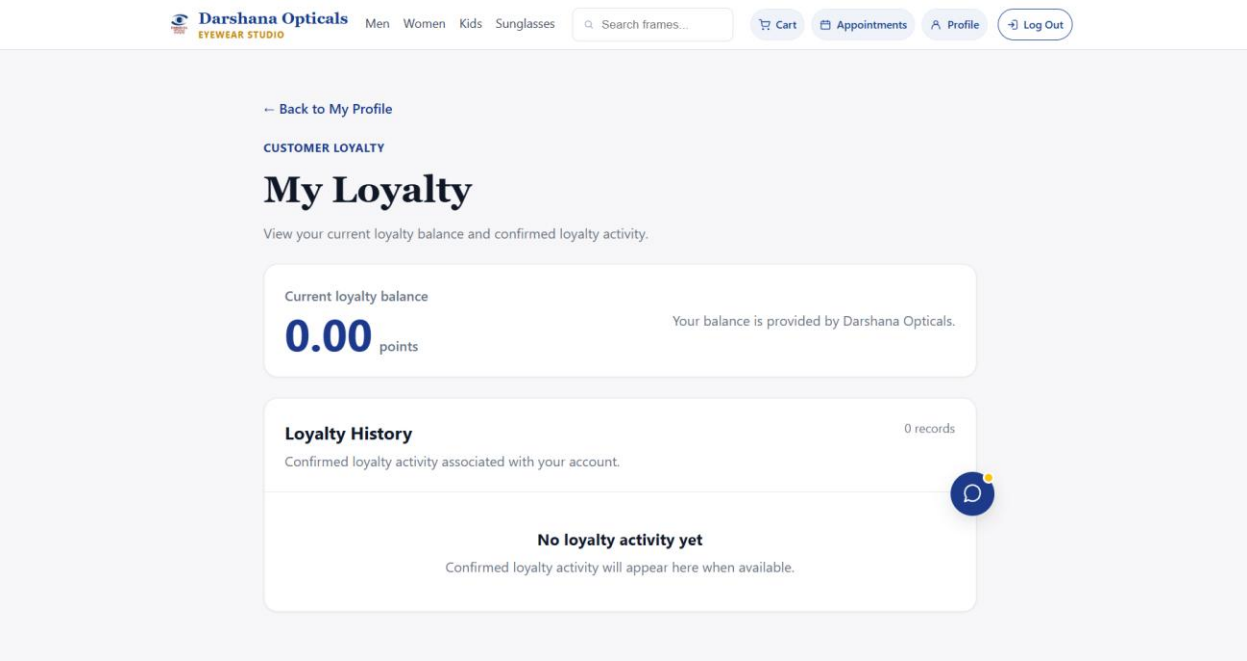


Figure C.10: Customer Prescription History Interface

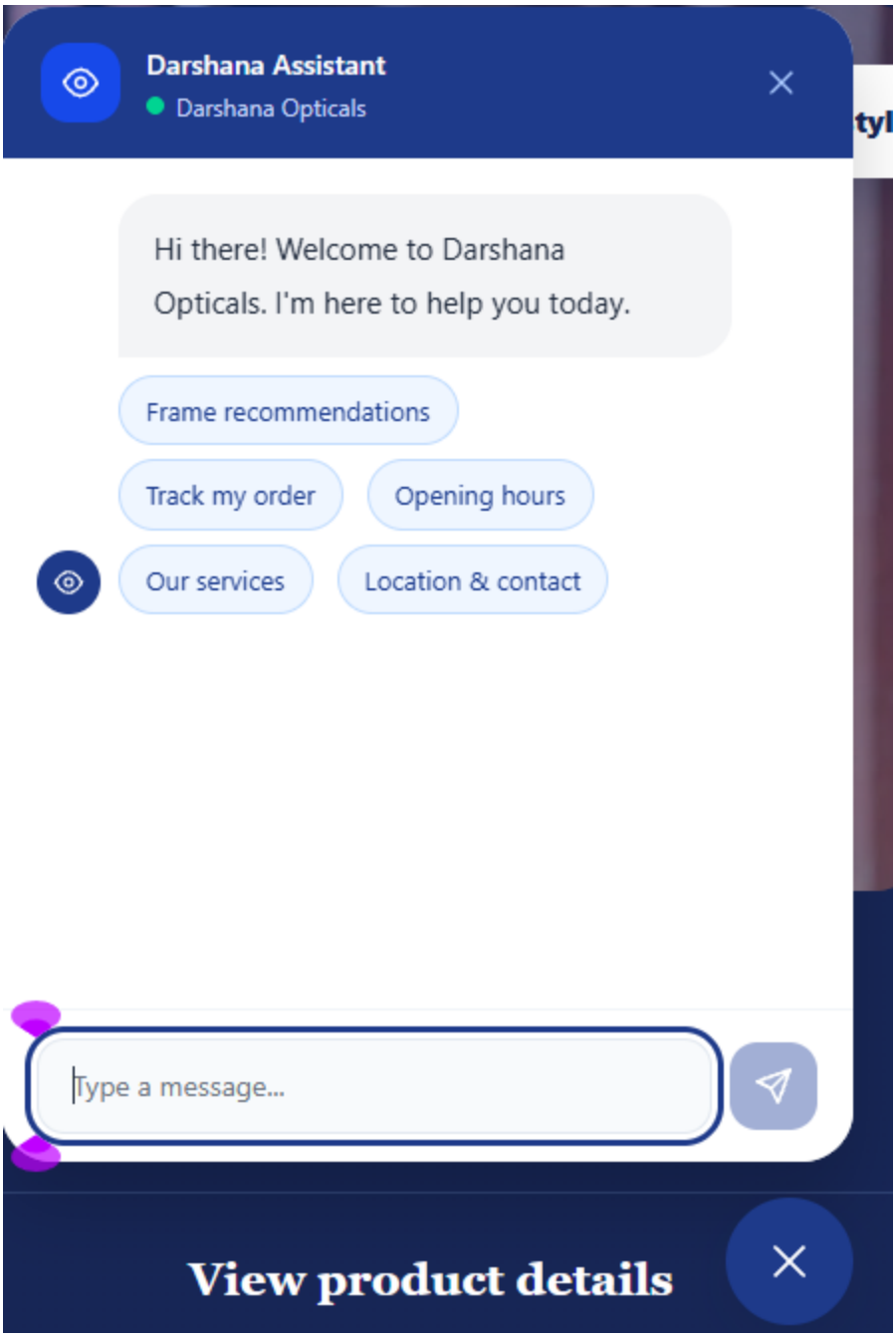


Figure C.11: AI Chatbot Interface

Screenshots should be selected to demonstrate the primary system functionality rather than include every interface variation.

The final SDS contains approved user-interface mock-ups that can be used as design references. However, mock-ups should not be presented as screenshots of the implemented application.

Appendix D — Source Code and Repository References

D.1 GitHub Repository Organization

The project source code and supporting documents are managed through four separate repositories.

Table D.1: Project Source Code Repositories

Repository	URL
Frontend	https://github.com/darshana-opticals-osms/frontend
Backend	https://github.com/darshana-opticals-osms/backend
Infrastructure	https://github.com/darshana-opticals-osms/infra
Documents	https://github.com/darshana-opticals-osms/documents

D.2 Selected Backend Implementation Files

Table D.2: Selected Backend Implementation Files

File	Implementation Responsibility
`src/services/auth.service.js`	Authentication and credential verification
`src/utils/jwt.js`	JWT creation and verification
`src/services/checkout.service.js`	Cart validation and checkout processing

`src/models/order.model.js`	Order schema and order lifecycle data
`src/models/doctorSession.model.js`	Doctor Session schema and capacity constraints
`src/models/inventory.model.js`	Inventory document structure
`src/models/customer.model.js`	Customer account data
`.github/workflows/ci.yml`	Backend continuous integration

D.3 Selected Frontend Implementation References

The frontend repository contains the React application and its associated components, routing, state management, and API integration.

The following implementation areas are relevant:

- Authentication interfaces.
- Product catalogue and product details.
- Shopping-cart state management.
- Checkout and payment pages.
- Appointment booking.
- Customer order and prescription history.
- Staff inventory and order interfaces.
- Administrative management pages.
- Reporting interfaces.

Exact frontend file paths and the corresponding commit identifiers should be recorded from the final released source revision.

D.4 Source Revision Information

Frontend release commit: 127afc2ed6af3d99b4b079dfa70babc6d7345d28

Backend release commit: c00d99c30af94121eff5185442686f88c9746df3

Infrastructure release commit: 684ae8551d27cd3cfe85174a90737a6c29a58fe1

Documentation release commit: 09e390e3861cbec21bc93e15dfb52be12493c445

Recording the final commit identifiers establishes which application version was evaluated in the report.

Appendix E — Architectural Decision Records

The project maintains Architectural Decision Records to document important design refinements and implementation decisions.

Table E.1: Architectural Decision Records

ADR	Title
ADR-001	Clinical Prescription Data Storage Design
ADR-002	Authoritative RBAC Role Model
ADR-003	Shopping Cart Persistence and Checkout Boundary
ADR-004	Order Lifecycle and Tracking Status Model
ADR-005	Loyalty Points Calculation and Persistence Model
ADR-006	Inventory Reorder-Alert Rules and Persistence Model
ADR-007	Appointment Scheduling, Availability, and Lifecycle Rules
ADR-008	Inventory Image Upload and Cloudinary Storage Architecture
ADR-009	Appointment Communication and Automated Reminder Scope Decision
ADR-010	AI Chatbot Architecture and Knowledge Strategy
ADR-011	Secure Payment Gateway Integration and Payment Lifecycle Architecture

ADR-012	Authoritative Immutable Audit-Trail Architecture
ADR-013	Order-Status Notification Policy
ADR-014	Sales and Inventory Reporting Definitions, Metrics, and Access Policy
ADR-015	Render.com Backend Hosting Fallback

Repository location: `documents/adr/`

These records provide traceability between the original design and implementation-related architectural changes.

The complete ADR documents remain available in the Documents repository and are referenced in Chapter 8.

Appendix F — Final Test Case Register and Quality Assurance Evidence

This appendix provides the principal quality assurance documents and selected supporting evidence used in Chapter 5.

F.1 Final Test Case Register

Document: Final Test Case Register

Related Issue: DDP-087

Evidence Baseline: 25 October 2026

Revision: 26 October 2026

Repository File: `testing/test-register.pdf`

The register documents forty representative automated test scenarios supported by local Jest and Vitest execution evidence.

Each scenario includes a test identifier, requirement mapping where applicable, test type, priority, source reference, preconditions, steps or inputs, expected outcome, actual result, and execution status.

The forty scenarios represent selected verified tests and are not an exhaustive enumeration of all 3,416 automated assertions.

The complete thirty-page register is maintained as an accompanying evidence document.

F.2 Consolidated Automated Testing Results

Table F.1: Local Automated Test Execution

Metric	Backend	Frontend
Framework	Jest	Vitest
Test Suites / Files	118	110
Executed Tests	2,075	1,341
Passed Tests	2,075	1,341
Failed Tests	0	0
Fresh Recorded Duration	346.634 seconds	77.81 seconds

Combined Result: 3,416 of 3,416 automated tests passed during the documented local executions.

F.3 Automated Test Coverage Report

Document: Final Test Coverage Report

Related Issue: DDP-087

Repository File: `testing/coverage-sprint8.pdf`

Table F.2: Reported Code Coverage

Metric	Backend	Frontend
Statement Coverage	92.34%	88.05%

Branch Coverage	81.68%	81.80%
Function Coverage	97.86%	90.95%
Line Coverage	93.00%	88.86%

Both repositories exceeded 80% overall coverage across the reported metrics.

However, the 90% branch coverage target for every designated critical business module remains unverified.

F.4 Selected Critical-Module Coverage

Table F.3: Selected Frontend Branch Coverage

Component	Branch Coverage
Payment Entry Page	68.70%
Checkout Entry Page	68.05%
Appointment Booking Page	71.55%
Admin Staff Editor	66.37%
Cart Provider	68.08%
Staff Move Appointment Page	57.64%
Inventory Editor	92.40%
Order Service	100.00%
Payment Service	86.20%

These figures identify areas requiring additional branch-level verification or targeted testing, depending on the final critical-module classification.

F.5 API Integration Test Evidence

The final register identifies seventy-three method-and-path route declarations in the backend source code.

A supplementary audit identified matching direct HTTP integration-test source references for eight of these routes.

This source-level evidence does not establish that all eight routes individually passed on the final deployed revision.

The remaining sixty-five routes were not independently confirmed through the targeted source audit and must not automatically be classified as untested or failed.

Complete endpoint-level integration verification remains an outstanding acceptance activity.

F.6 End-to-End Test Evidence

Related Issue: DDP-084

Primary References: Infrastructure Issue #11 and Pull Request #19.

The DDP-087 documentation refers to ten previously reported Playwright scenarios and seven repeated authorization checks reported as passing.

The precise scenario-level outcomes, execution environment, supporting logs, and tested source revisions require verification against the original DDP-084 evidence.

The hosted PayHere payment-confirmation workflow remains blocked or unverified in the available QA documentation.

F.7 Performance Test Report

Document: Performance Test Results Report

Related Issue: DDP-086

Repository File: `testing/performance-report.pdf`

The report documents thirteen baseline API benchmarks, controlled workloads of five, fifteen, and thirty virtual users, database query profiling, and transaction-concurrency tests.

Table F.4: Controlled Load Testing Results

Virtual Users	Requests	P95 Latency	Throughput
5	25	102.7 ms	86.6 req/s
15	60	130.9 ms	146.8 req/s
30	120	241.8 ms	152.8 req/s

All thirteen baseline endpoints recorded P95 response times below 130 milliseconds.

The tested workloads recorded no HTTP 5xx failures.

These results were obtained in an isolated local environment.

F.8 Transaction Concurrency Evidence

Table F.5: Transactional Consistency

Scenario	Concurrent Requests	Available Units / Slots	Successes	Rejections	Result
Checkout — Single Item	5	1	1	4	Pass
Checkout — Two Items	4	2	2	2	Pass
Appointment — Single Slot	5	1	1	4	Pass
Appointment — Two Slots	4	2	2	2	Pass

No inventory overselling or appointment overbooking was recorded during these controlled scenarios.

F.9 Security Evidence

Related Issue: DDP-085

Repository File: `security/owasp-checklist.pdf`

The OWASP checklist provides the relevant security assessment details and identified residual risks.

Additional selected error-handling and adversarial test results are documented in the Performance Test Results Report.

These supporting materials should be read together when evaluating security-related requirements.

F.10 Final QA Evidence Limitations

The final automated testing results are supported by documented local execution evidence.

However, complete release acceptance remains subject to additional verification.

Important outstanding items include:

- Complete endpoint-level integration test confirmation.
- Critical business logic branch coverage evaluation.
- Hosted PayHere end-to-end payment confirmation.
- Reconciliation of individual DDP-084 execution results.
- CI workflow and coverage artifact traceability.
- Tested-to-deployed source revision equivalence.
- Formal final QA approval and applicable user acceptance evidence.

These limitations are documented to maintain transparency regarding the evidence supporting final system acceptance.

Appendix G — CI/CD and Deployment Evidence

G.1 Backend Continuous Integration

Source: Backend repository.

Workflow: `.github/workflows/ci.yml`

The backend continuous integration workflow executes configured code-quality checks, formatting checks, automated tests, and dependency auditing.

[Insert a screenshot of a verified GitHub Actions workflow run.]

G.2 Frontend Continuous Integration

The frontend repository provides automated validation for frontend code quality, tests, and production build generation.

[Insert a screenshot or artifact reference from the relevant successful frontend workflow.]

G.3 Frontend Deployment

Hosting Platform: Vercel.

Application: React/Vite frontend.

https://darshana-opticals.vercel.app

G.4 Backend Deployment

Hosting Platform: Render.com.

Application: Node.js/Express API.

https://osms-backend-staging.onrender.com

G.5 Database Configuration

Database Configuration

Database Platform: MongoDB Atlas

MongoDB Atlas serves as the database for the Darshana Opticals OSMS. Database access is managed securely through the Node.js/Express backend deployed on Render.com. The

frontend communicates with the backend through HTTPS APIs without directly accessing the database.

Verification Evidence:

- **Backend Health:** https://osms-backend-staging.onrender.com/api/health — HTTP 200 OK recorded.
- **Database Operations:** Deployed application tests verified appointment booking, order creation, prescription recording, and data persistence.
- **Configuration:** MongoDB Atlas connectivity was confirmed at team level; independent verification of database isolation remains outstanding.
- **Supporting Evidence:** DDP-084 E2E Execution Report.

Database credentials, connection strings, secrets, and real customer information are excluded from the documentation.

G.6 Deployment Verification

Deployment verification evidence should include relevant frontend availability, backend health, API connectivity, database connectivity, and selected application workflow checks.

Frontend deployment: https://darshana-opticals.vercel.app
Backend deployment: https://osms-backend-staging.onrender.com

G.7 Automated Test Execution Provenance

The DDP-087 Test Coverage Report includes an execution-provenance addendum dated 26 October 2026.

The addendum records the pre-run Git revisions and timestamps associated with the successful local backend and frontend test executions.

Table G.1: Local Test Revision Evidence

Item	Backend	Frontend
Repository	OSMS Backend	OSMS Frontend

Recorded Test Result	2,075 / 2,075 passed	1,341 / 1,341 passed
Revision	`47eb042a7627d4ae1f1f67b9c48c370379faa7c5`	`3e6fa9b5b277d9663982cb6e36a5af8c8187c97c`
Execution Type	Local Jest	Local Vitest
Execution Date	26 October 2026	26 October 2026

The recorded evidence supports the documented local test baseline.

However, it does not establish that these revisions correspond exactly to the final deployed application.

The final release documentation should record the deployed frontend and backend commit identifiers, relevant GitHub Actions workflow runs, and associated artifacts.

Differences between local test revisions and release revisions must be evaluated before asserting final release parity.

Appendix H — Project Management and Development Evidence

H.1 GitHub Issue Management

The project uses GitHub Issues to manage development tasks, defects, documentation updates, and verification activities.

Issues are linked to corresponding implementation branches and Pull Requests where applicable.

H.2 Pull Request Reviews

Pull Requests provide a review mechanism for changes before integration.

Relevant review evidence may include:

- Pull Request identifier.
- Related issue.
- Source and target branches.
- Summary of changes.
- Review outcome.
- Successful validation checks.
- Merge commit identifier.

Pull Request Evidence Summary

Table H.1: Pull Request Evidence Summary

Repository	Pull Request / Related Issue	Summary of Changes	Review Outcome and Validation	Merge Commit
Backend	PR #70 — DDP-041, Issue #47	Secure checkout APIs, MongoDB transactions, stock validation and Order creation	Approved; 586 tests passed across 48 suites, ESLint and Prettier passed	61aa490
Backend	PR #80 — Payment implementation, Issue #53	PayHere payment integration, callback verification, refunds and reconciliation	Changes requested → Approved; 1,541 tests passed, 90 suites, ESLint and Prettier passed	663db53
Backend	PR #107 — DDP-086, Issue #92	Performance benchmarking, concurrency testing, query optimization and reliability assessment	Multiple change requests → Approved; 2,063 tests passed, lint and formatting checks passed	5c1e20e

Backend	PR #108 — DDP-085, Issue #91	OWASP security hardening, login lockout, dependency remediation and security logging	Changes requested → Approved; 2,075 tests passed, ESLint, Prettier and security audit checks passed	0881662
Frontend	PR #46 — DDP-042, Issue #22	Customer checkout interface and backend Order integration	Changes requested → Approved; final CI reported 346 tests passed	c376d4d
Frontend	PR #47 — DDP-047, Issue #24	Customer appointment booking, history, cancellation and API integration	Approved; 402 tests passed, ESLint, Prettier and production build passed	fddabda
Frontend	PR #68 — DDP-085, Backend Issue #91	Frontend dependency security remediation, browser-security headers and Node.js CI compatibility	Changes requested → Approved; 1,341 tests passed, ESLint, Prettier, build and CI passed	a21e1a0
Frontend	PR #77 — DDP-083, Infra Issue #10	Vercel SPA routing configuration for React Router	Approved; JSON validation, production build and frontend CI passed	81db413
Infrastructure	PR #19 — DDP-084, Issue #11	Playwright E2E testing, integration regression and execution evidence	Changes requested → Approved; 10 browser tests, 645 backend and 202 frontend regression tests passed	2b9fa78

Infrastructure	PR #22 — DDP-083, Issue #10	Vercel/Render deployment runbook, environment configuration, security and rollback procedures	Approved; Infra GitHub Actions passed	757f5b0
Infrastructure	PR #24 — DDP-090, Issue #12	Release coordination, quality gates, provenance manifest and rollback framework	Approved; 42 automated tests passed and GitHub Actions checks completed	f473291
Infrastructure	PR #25 — DDP-091, Issue #13	Final UAT report, demonstration plan and deployment limitations	Changes requested → Approved; Infra CI reported passing. Hosted payment verification remains pending	a788ef5
Documents	PR #53 — DDP-088, Issue #47	Final implementation-aligned SRS and SDS documentation	Changes requested → Approved; document consistency reviewed, Git diff validation passed	27b2605
Documents	PR #54 — DDP-087, Issue #46	Final test coverage report and test case register	Multiple change requests → Approved; reports document 3,416 passing tests across backend/frontend suites	00a26dc

H.3 Sprint Management

The development work was organized into multiple sprints.

Sprint-related information, task allocation, completion evidence, and progress records are maintained through the project management process.

The Darshana Opticals OSMS project followed an iterative, sprint-based development approach. Tasks were assigned and tracked using GitHub Issues and Pull Requests.

- **Sprint 1:** Project setup, requirements, architecture, and core system foundations.
- **Sprint 2:** Implementation and integration of major frontend and backend functionalities.
- **Sprint 3:** Final testing, security verification, deployment, documentation, and release preparation.

Sprint progress, task completion, code reviews, and milestone achievements were recorded through GitHub Issues, Pull Requests, and supporting project documentation.

H.4 Development Standards

The project follows documented coding and review standards.

Source: `documents/standards/coding-standards.md`

The standards support consistent implementation practices, source-code review, and maintainability.

Appendix I — User Acceptance and Client Feedback Evidence

I.1 User Acceptance Testing

Related Issue: DDP-091 — Final User Acceptance Testing and Demonstration.

User acceptance evidence should identify the evaluated application version, selected scenarios, participants or stakeholder roles, outcomes, and outstanding issues.

[Insert the finalized UAT evidence when verified.]

I.2 Client Feedback

Client feedback should be included only when supported by actual recorded responses or approved meeting evidence.

Pending

I.3 Final Acceptance or Sign-Off

The Darshana Opticals OSMS project underwent final testing, internal reviews, and User Acceptance Testing (UAT) to evaluate system functionality and release readiness.

Acceptance Evidence:

- Final UAT results and demonstration plan documented under DDP-091.
- Relevant Pull Requests reviewed, approved, and merged.
- Testing and deployment verification evidence recorded in project repositories.

Sign-Off Status: Formal client acceptance or supervisor sign-off documentation has not been provided for inclusion in this report. Therefore, final acceptance remains **pending formal confirmation**.

Reference: DDP-091 — Final UAT Report

Appendix J — Supplementary Limitations and Outstanding Work

The final SRS and project evaluation identify several outstanding requirements and operational verification gaps.

Table J.1: Outstanding Requirements and Limitations

Area	Limitation or Outstanding Activity
Product Catalogue	Separate style-based filtering remains unverified

Inventory	Automated inter-branch transfers unavailable
Inventory	Full real-time synchronization not independently verified
Loyalty	Point redemption and tier-based rewards unavailable
Reporting	Historical stock movement and inventory turnover reporting unavailable
Appointment Communication	Automated reminders excluded by approved decision
Session Security	Separate 30-minute inactivity timeout not verified
Usability	Original staff transaction-time criterion not verified
Reliability	Sustained 99.9% uptime not verified
Scalability	Original 50% growth criterion not fully verified
Portability	Required browser compatibility testing not verified
Operations	Backup restoration and long-term retention guarantees require verification

These limitations are discussed in Chapter 6 and retained here as supplementary evidence for future system evaluation.

J.1 Outstanding Quality Assurance Verification

Table J.2: Final QA Verification Gaps

Area	Outstanding Verification
API Integration	Complete method/path-specific integration evidence for all applicable endpoints
Critical Business Logic Coverage	Verify the required 90% branch coverage against the designated critical modules
Hosted Payment	Complete successful hosted PayHere payment-confirmation E2E verification
System E2E	Reconcile DDP-084 scenario results with their original browser execution artifacts
CI Traceability	Preserve relevant workflow run URLs and test-coverage artifacts
Release Traceability	Confirm parity between tested source revisions and deployed release revisions
Security	Reconcile remaining risks and detailed DDP-085 findings
Client Acceptance	Obtain and document formal UAT or client acceptance evidence where required

The existence of these outstanding items does not automatically indicate a functional defect.

Rather, they represent areas where the available evidence is insufficient to assert complete acceptance or release readiness.

9.2 Appendix Summary

The appendices provide supplementary documentation and technical evidence supporting the Final Development Report.

The original and implementation-aligned SRS/SDS documents establish requirements and architectural traceability, while source-code references and Architectural Decision Records provide evidence of the implemented design.

The testing appendices include the Final Test Case Register, Test Coverage Report, and Performance Test Results Report.

The documented local automated executions recorded 3,416 passing tests, while controlled performance and concurrency assessments provided evidence of selected system quality characteristics.

The appendices also identify the limitations of the available verification evidence, including incomplete endpoint integration confirmation, outstanding critical-module branch coverage assessment, hosted payment verification, and release-source traceability.

These supporting materials provide an evidence-based foundation for the project evaluation while distinguishing verified results from outstanding acceptance activities.